

2026 年 7 月 3 日 Version 1.6

ハッカソン ～計画書と設計書～

チーム 40

24G1021 : 宇井 祐真
24G1026 : 梅野 大誠
24G1136 : 山口 侑希
25G1020 : 薄井 悠希
25G1041 : 菊池 侑人
25G1059 : 佐藤 涼菜

目次

第 1 章	プロジェクトの計画書	1
1.1	プロジェクトの概要	1
1.1.1	プロジェクトの題目	1
1.1.2	プロジェクトの目的	1
1.1.3	既存の技術とプロジェクトの独自性	2
1.2	スコープ・マネジメント	3
1.2.1	成果物	3
1.2.2	プロジェクトの対象範囲と非対象範囲	3
1.2.3	機能分解構造 FBS	4
1.2.4	製品分解構造 PBS	4
1.2.5	FBS と PBS の対応関係	5
1.3	作業計画	5
1.3.1	作業分解構造 WBS と管理番号	5
1.3.2	アローダイアグラムとクリティカルパス	6
1.3.3	役割分担	7
1.3.4	ガントチャート	7
1.4	検証と妥当性確認: V&V 計画	11
1.4.1	プロジェクトの成果物に対する有効指標 MOE	11
1.4.2	有効指標 MOE に対応する性能指標 MOP	11
1.4.3	システムテストで評価する性能指標 MOP	13
1.4.4	結合テストで評価する性能指標 MOP	14
1.4.5	単体テストで評価する性能指標 MOP	14
1.4.6	プロジェクト中に監視する技術性能指標 TPM	14
1.4.7	プロジェクト中に監視する指標 TRL	15
1.5	資源・調達管理	16
1.6	リスク管理	16
第 2 章	システムの基本設計	17

2.1	機能一覧	17
2.2	システム構成図	18
2.2.1	システム全体の構成図	18
2.2.2	指揮デバイスの構成図	18
2.2.3	中継機の構成図	19
2.2.4	楽器デバイスの構成図	19
2.3	操作インターフェース設計	19
2.3.1	指揮デバイスの設計	20
2.3.2	指揮デバイスの演奏開始トリガー送信機構の設計	20
2.3.3	指揮デバイスの BPM 変更機構および音量変更機構の設計	20
2.3.4	中継機デバイスの表示	21
2.4	状態遷移図	21
2.4.1	全体の状態遷移図	21
2.4.2	指揮デバイスの状態遷移図	22
2.4.3	楽器デバイス受光機構の状態遷移図	23
第 3 章	システムの詳細設計	27
3.1	部品一覧	27
3.2	ハードウェア設計	29
3.2.1	指揮デバイス	29
3.2.2	観覧車型中継機デバイス	29
3.2.3	レーザー光送信装置	34
3.2.4	楽器デバイス受光部装置	34
3.3	Processing	34
3.3.1	システムアーキテクチャとファイル構成	34
3.3.2	通信プロトコルとエラーハンドリング	38
3.3.3	システム統括とデータ構造	40
3.3.4	各楽器の実装仕様 (Instrument 系クラス)	61
3.3.5	デバッグおよび単体テスト機能	76
3.4	通信設計	77
3.4.1	通信の概要	77
3.4.2	ファイル構成	77
3.4.3	通信に使用する変数・関数	78

3.4.4	クラス図	84
3.4.5	ボタン入力の判定	85
3.4.6	加速度センサ制御	86
3.4.7	可変抵抗器を用いた BPM・音量読み取り機能	89
3.4.8	処理ロジック	92
3.4.9	回転速度同期機構の設計	94
3.4.10	LED マトリクスでの BPM 表示	99
3.4.11	関数設計	101
3.4.12	Tick 管理およびシリアル・同期通信	102
3.5	処理フローの設計	118
3.5.1	全体のフローチャート	118
3.5.2	LEDMatrix の処理フローチャート	120
3.6	テストの設計	121
3.6.1	音色の再現度テスト	121
3.6.2	ボタン判定の制御	122
3.6.3	加速度センサの制御	122
3.6.4	BPM・音量読み取り機能	122
3.6.5	BPM・音量変更機能	122
3.6.6	観覧車の制御	122
3.6.7	BPM 変更機能	123
3.6.8	同期精度テスト	123
3.6.9	負荷テスト	123

第 1 章 プロジェクトの計画書

1.1 プロジェクトの概要

1.1.1 プロジェクトの題目

マジカル☆観覧車

1.1.2 プロジェクトの目的

近年、私たちは動画サイトやストリーミングサービスによって、誰でも手軽に音楽コンテンツを楽しむことができるようになった。特に、日本独自の文化として、ゲームやアニメといったサブカルチャー発の音楽コンテンツが盛んである。このようなコンテンツは、上記のようなデジタル技術の発展の恩恵を受ける一方で、実際の体験に重きを置くライブイベントやコンサートなどとの相乗効果を得て、市場規模の拡大に大きく貢献している [1][2]。実際に、一般社団法人コンサートプロモーターズ協会が行ったライブ市場調査によると、2025 年にはライブの年間売上として 6443 億円を記録しており、入場者数は 5999 万人を記録した [3]。これらのイベントが市場シェアを拡大する理由として、ライブ演出における音響や照明、身体で感じる振動などは、その演奏状態を複数人へ同時に共有可能であり、空間全体としての一体感を形成する役割を持っている点が挙げられる。つまり、ユーザが音楽を聴くだけにとどまらず、音響や照明などの物理的、視覚的な技術をその場で一体となって楽しむといった、没入感の高い体験を得られるという点が理由の一つである。

ここで、没入感はどのように創出されるのだろうか。Natalia Cooper ら (2018) は、VR 環境においてのシミュレーションの充実度を高めるため、再現の難しい感覚を別感覚による代替フィードバックとして与える実験を行った。その結果、VR 上の視覚的な刺激のみよりも、音や触覚といった異なる刺激を追加することが、ユーザのシステムに対する没入感・関与感を向上させることを明らかにした [4]。このことから、複数の刺激による多感覚的な情報の提示は、ユーザ体験における没入感の向上に

有効であるとわかる。

また、HCI 分野において、ユーザの主体的な動作がシステムに対する没入感を高めることができることが示されている。Martin ら (2024) によると、ライブコンサート環境において、観客は受動的な聴取をするのではなく、能動的な参加を行うことが、ライブの活気や観客の感情性を形成すると結論づけた [5]。よって、能動的な活動参加もまた、システムの没入感を高める要因となるとわかる。

一方、現状の自動演奏技術分野については、デジタル技術の発達により高精度な同期や楽曲再現を行うことができる。例えば、yamaha よりリリースされている Disklavier ENSPIRE シリーズは、光学センサやサーボモータを用いることで、演奏情報の高精度再現を行うことが可能であり、ユーザに対して、物理的な鍵盤の動きを用いての視覚的フィードバックも可能としている [6]。しかしながら、このフィードバックは多くの場合、利用者1人に対してのものとなり、空間全体を考えると、他に与える刺激は限定的になる場合がある。よって、現在の自動演奏技術分野は、視覚的なフィードバックを追加し没入感を高めているものの、その実、空間的フィードバックは限定的であると言える。

従って、本プロジェクトではこの問題を解決するため、自動演奏技術に空間的フィードバックを追加した楽器間通信システムであるマジカル☆観覧車を提案する。

1.1.3 既存の技術とプロジェクトの独自性

本節では、プロジェクトの設計計画にあたり、既存技術の分析を通して本プロジェクトの独自性について検討する。まず、音楽に視覚的なフィードバックを追加した例として、DMX512 信号 (以下、DMX 信号と示す) を用いたシステムが挙げられる。DMX 信号とは、操作卓と調光機の制御に用いられ、現在では舞台やコンサートなどの場面で演出に合わせた光量制御に用いられている [7]。このシステムは、音楽だけでなく光といった異なる刺激を与えることで、没入感を高めることができる。しかしながら、利用されるシチュエーションを考えると、システムの対象である観客は現象を受動的に認識する 경우가多く、観客が演奏に参加できない場合が考えられる。ここで、本提案では空間フィードバックの一環として指揮デバイスを導入している。このデバイスを用いることで、ユーザが主体的に演奏に参加することができ、身体的な動作関与による没入感の向上を期待することができると考えられる。

また、物理的な操作による音楽の制御を行うシステムの例として、ReacTable が挙げられる。ReacTable とは、卓型のディスプレイと複数の物理モジュールによって構

成されるシンセサイザの一種で、身体的な物理動作を通して演奏の制御、ディスプレイによる視覚的フィードバックが可能になる [8]。しかしながら、ディスプレイ内に視覚的なフィードバックが行われる関係上、空間全体へのフィードバックを与えるインターフェースとして扱われてはいない。ここで、本提案では、指揮デバイスに加え観覧車型デバイスを導入している。このデバイスを用いることで、ユーザ以外からも演奏状態の把握や共有ができ、空間全体への多感覚的フィードバックを可能にすることが期待できると考えられる。

以上より、本提案は既存研究の空間共有性と主体的な演奏制御技術を組み合わせることで、空間全体への感覚フィードバックを行い、広く没入感や関与感を高めることができると考えられる。

1.2 スコープ・マネジメント

1.2.1 成果物

本プロジェクトで製作する成果物は、同期開始に指揮棒、中継機に観覧車を用いた楽器演奏システムである。指揮動作によって中継機と楽器デバイスの開始と終了、BPM と音量情報を送信する指揮デバイス、指揮デバイスから情報を受け取り、BPM に応じて回転する観覧車型中継機デバイス、光信号によって BPM を感知し、その速さで音楽を再生する楽器デバイスの 3 つに分割し製作する。楽器デバイスは、Arduino で楽譜情報を作成し、Processing から楽器音声を再生する動作と、BPM 変化や音声変更の処理を行い演奏情報を変更するシステムを制作する。また、中継機では LEDMatrix に現在の BPM 数を表示するように製作を行う。そして、3D プリンタを用いた指揮デバイス機構、観覧車機構、レーザー光送信機構、Arduino を用いたモータ制御機構を成果物とする。

1.2.2 プロジェクトの対象範囲と非対象範囲

本プロジェクトの対象範囲は、センサーや UDP を用いた無線通信、processing を用いた楽器演奏、3D プリンタを用いた観覧車製作や中継機デバイス、指揮デバイスのハードウェア製作である。非対象範囲は、有線による情報転送、Arduino からの音源再生、信号の暗号化である。

1.2.3 機能分解構造 FBS

機能分解構造 FBS は、以下の図 1.1 の通りである。観覧車型中継機デバイスを用いた楽器間通信デバイスの作成にあたって、ユーザ視点の機能として三つに分けられる。一つ目は楽器の演奏、二つ目に通信同期関係、三つ目に見た目の華やかさである。その三要素を詳細に分割すると、楽器演奏に関しては多彩な楽器の音色や高度な音色再現が挙げられる。通信同期に関しては、ズレの少ない同期や変化する速度が挙げられ、見た目の華やかさに関しては、視覚的に伝わりやすい表示や見て興味を誘うような動作が挙げられる。

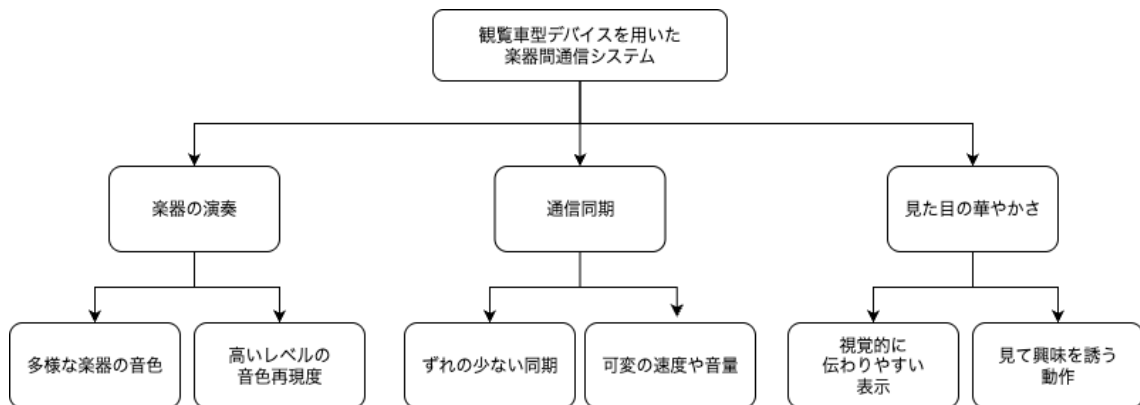


図 1.1 FBS 図

1.2.4 製品分解構造 PBS

次に製品分解構造 PBS について説明する。PBS は以下の図 1.2 の通りである。観覧車型中継機デバイスを用いた楽器間通信デバイスをサブ成果物に分解すると、Processing、楽器、楽曲表現、通信の四つに分割される。Processing に関しては、音声波形の生成と音声の再生に分割され、音声波形の生成は基礎波形情報と適切なエフェクトが成果物としてあげられ、音声の再生は発音機構が成果物として挙げられる。楽器に関しては、同期通信の開始信号が必要であり、そこから照度センサと開始信号からの BPM 計算処理プログラムが成果物で挙げられる。楽曲表現に関しては、演奏情報や回転デバイスに挙げられ、それぞれ LED マトリクスとモーター、観覧車本体が成果物として挙げられる。通信に関しては、指揮デバイスに関する通信、中継機に関する通信、同期通信の三つに分割される。指揮デバイスについては加速度の設計と演奏情報の更新機構が成果物としてあげられ、中継機については演奏情報の一斉

送信のプログラム設計，同期通信については光学設計とモータ回転速度の設計が成果物として挙げられる。

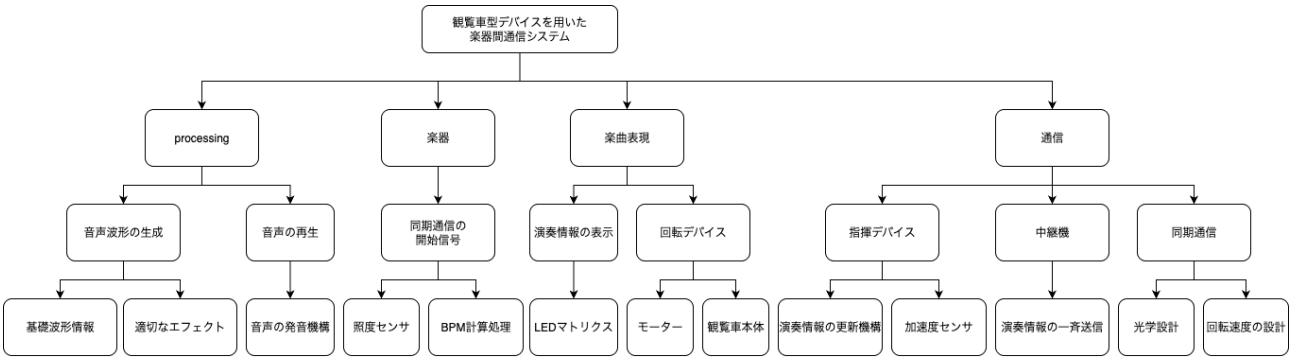


図 1.2 PBS 図

1.2.5 FBS と PBS の対応関係

FBS と PBS の対応関係について説明する。関係図は以下の図 1.3 の通りである。矢印の左側が FBS 要素であり，右側が PBS 要素である。多様な楽器の音色の要素には基礎波形情報と，音色を鳴らすための発音機構が対応する。高度な音色再現には適切なエフェクトが対応し，視覚的に伝わりやすい表示には LED マトリクスが対応する。ズレの少ない同期では，演奏情報の一斉送信や光学設計，BPM 計算処理や照度センサの成果物が対応する。速度変化の実現には，モータ回路や回転速度の設計，演奏情報の更新機構が対応し，興味を誘う動作には加速度センサや観覧車本体の成果物が対応する。

1.3 作業計画

1.3.1 作業分解構造 WBS と管理番号

作業分解構造 WBS と管理番号について説明する。WBS は設計フェーズ，実装フェーズ，テストフェーズに分けて説明する。それぞれ図 1.4，1.5，1.6 の通りである。図 1.4 について，1100 は基本設計についてであり，指揮デバイス，中継機デバイス，楽器デバイスの設計をそれぞれ割り当てる。1200 では詳細設計を割り当て，

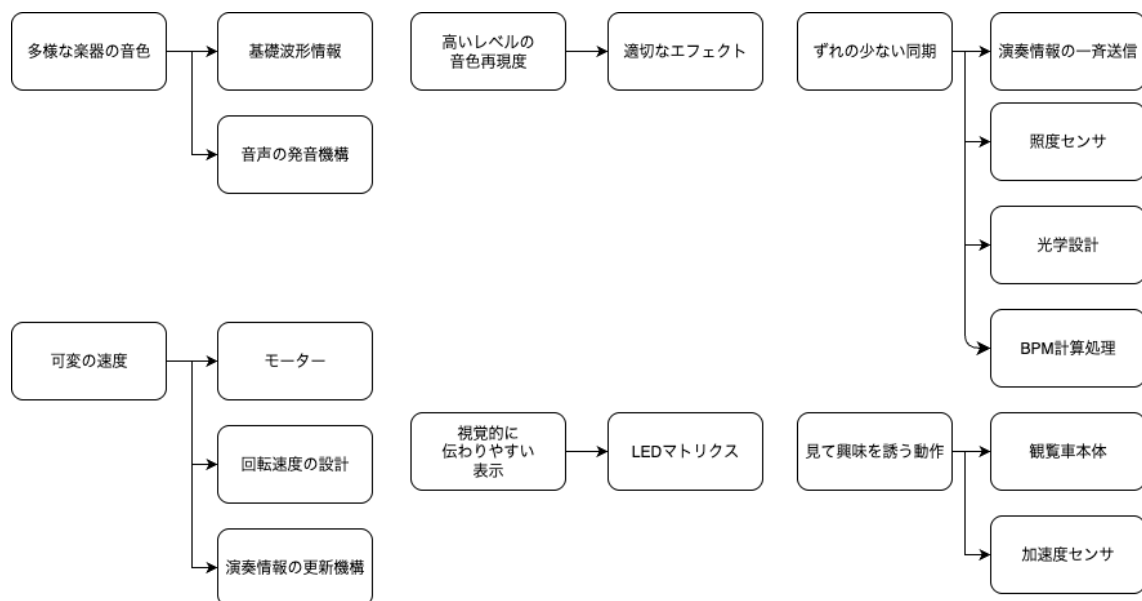


図 1.3 FBS と PBS の対応関係図

中継機デバイス、指揮デバイス、楽器、通信タイミング制御のハードウェア・ソフトウェア面の内容を作成する。図 1.5 について、2100 はソフトウェア、2200 はハードウェアとし、実際に作業する内容を詳細に分割する。図 1.6 について、実装フェーズで作成したものを単体でテストする内容を 3100 に、そしてそれらを結合して行うテストを 3200 に分割する。

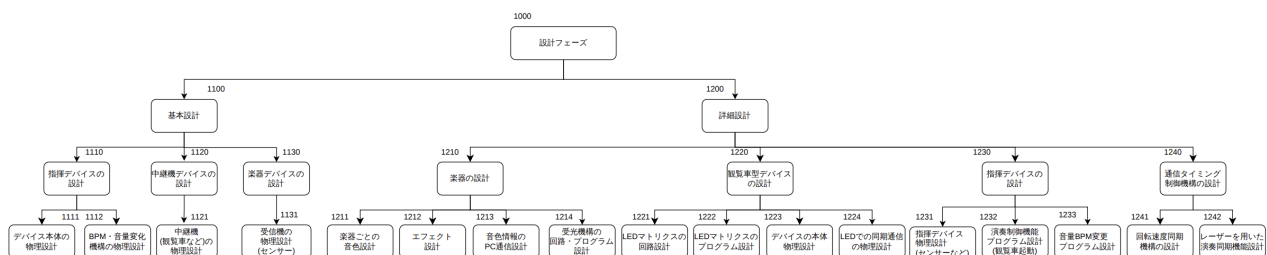


図 1.4 設計フェーズの WBS

1.3.2 アローダイアグラムとクリティカルパス

成果物の完成までのアローダイアグラムは図 1.7 の通りである。実装フェーズからテストフェーズまで消費する日付を考慮してつなげた結果、クリティカルパスは 2131, 2132, 2135, 3141, 2224, 3121, 3211, 3212, 3231, 3242 という順

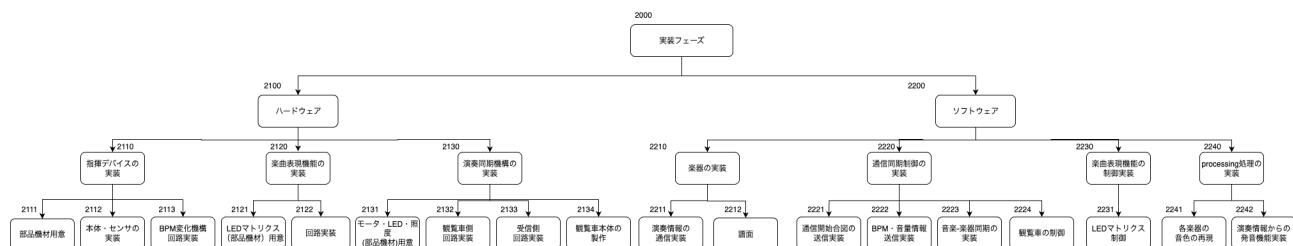


図 1.5 実装フェーズの WBS

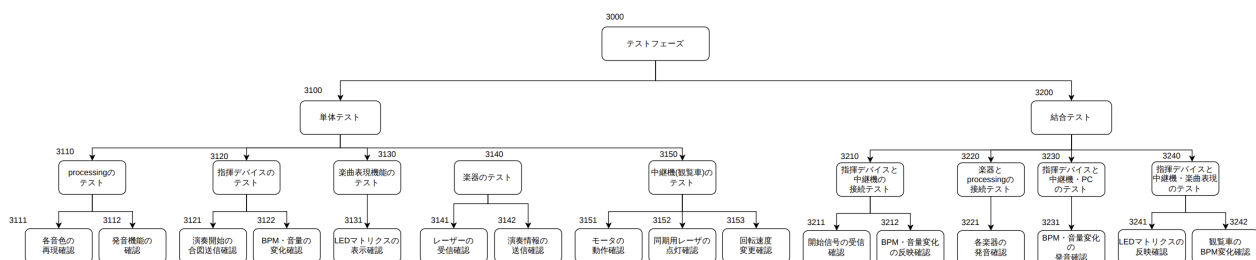


図 1.6 テストフェーズの WBS

序となり、日数は 40 日となった。この日数は制作期間である 47 日を下回る日数であるため、妥当な計画だとわかる。

1.3.3 役割分担

WBS を基に役割分担を作成する。役割分担表は、以下の 1.8 の通りである。分担の振り分けは主に楽曲表現型、楽曲系、通信系に分担し、それぞれハッカソン 1 と 2 でペアとなり製作を行う。楽曲表現系は山口と佐藤、楽曲系は梅野と薄井、通信系は宇井と菊池が行う。

1.3.4 ガントチャート

ガントチャートは図 1.9、1.10 の通りである。図 1.9 は実装フェーズのガントチャートであり、八週目までに基本的にそれぞれの単体作業を完成させる流れとなっている。また、各楽器の音色については十週目まで継続して音声の再現度を向上させる。図 1.10 はテストフェーズのガントチャートであり、単体のテストはそれぞれの項目が完成次第テストを行って確認し、その後八週目以降から結合テストで動作するのを確認するという計画となっている。十週目の授業日までにすべての結合テストを

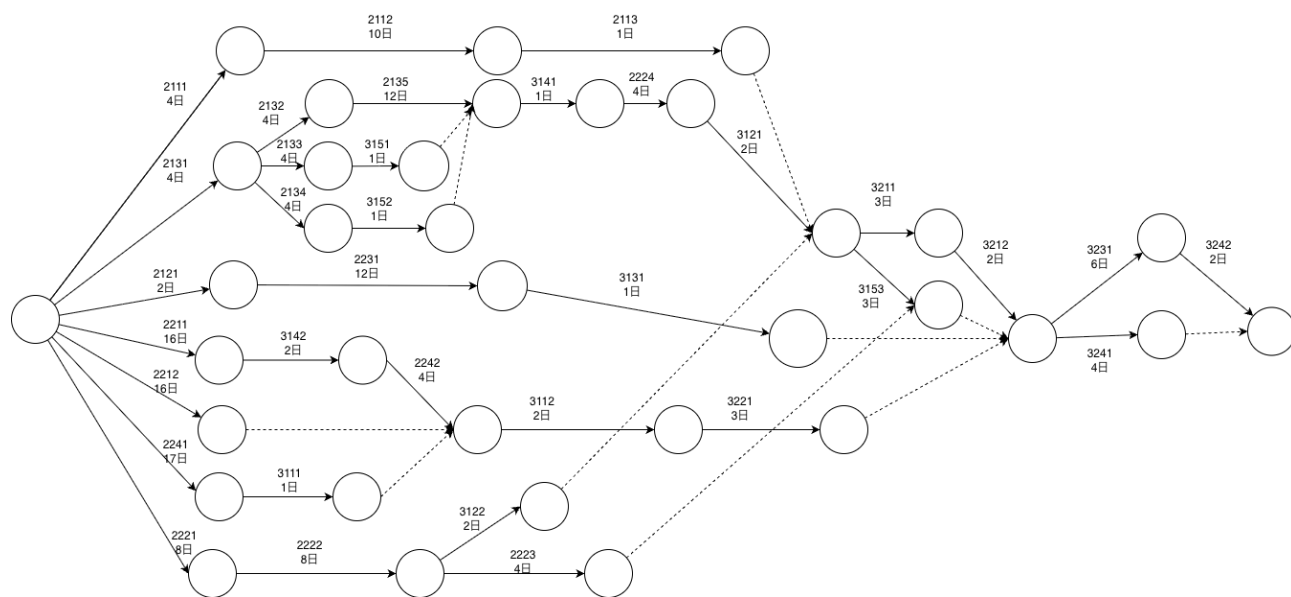


図 1.7 アローダイアグラム

実施し、十週から十一週目は主に予備日として間に合わなかった際に用い、結合テストまで行えるようにする。

観覧車型デバイスを用いた楽器間通信システム									
時間的フェーズ		要素成果物		活動タスク				担当者	
1000	設計フェーズ	1100	基本設計	1110	指揮デバイスの設計	1111	デバイス本体の物理設計	山口	佐藤
						1112	BPM・音量変化機構の物理設計	山口	佐藤
				1120	中継機デバイスの設計	1121	中継機の物理設計	山口	佐藤
				1130	楽器デバイスの設計	1131	受信機の物理設計	山口	佐藤
		1200	詳細設計	1210	楽器の設計	1211	楽器ごとの音色設計	梅野	薄井
						1212	エフェクト設計	梅野	薄井
						1213	音色情報のPC通信設計	梅野	薄井
						1214	受光側の設計(回路)	山口	佐藤
				1220	観覧車型デバイスの設計	1221	LEDマトリクスの設計(プログラム)	梅野	薄井
						1222	デバイス本体の設計(回路・見た目)	山口	佐藤
						1223	レーザーでの同期通信送信側の物理(回路)設計	山口	佐藤
				1230	指揮デバイスの設計	1231	指揮デバイス物理(回路)設計	山口	佐藤
						1232	演奏制御機能設計(観覧車起動プログラム)	宇井	菊池
						1233	BPM・音量変更設計(プログラム)	宇井	菊池
				1240	通信タイミング制御機構の設計	1241	回転速度同期機構の設計(プログラム)	宇井	菊池
						1242	レーザーを用いた演奏同期機能設計(プログラム)	宇井	菊池
2000	実装フェーズ	2100	ハードウェア	2110	指揮デバイスの実装	2111	部品機材用意	山口	佐藤
						2112	本体・センサの実装	山口	佐藤
						2113	BPM・音量変化機構の実装	山口	佐藤
				2120	楽曲表現機能の実装	2121	LEDマトリクスの用意	山口	佐藤
						2131	部品機材用意	山口	佐藤
						2132	3Dプリンタによる観覧車部品用意	山口	佐藤
				2130	演奏同期機構の実装	2133	観覧車回路実装	山口	佐藤
						2134	受信側回路実装	山口	佐藤
						2135	観覧車(中継機)本体の製作	山口	佐藤
		2200	ソフトウェア	2210	楽器の実装	2211	演奏情報の通信実装	梅野	薄井
						2212	譜面実装	梅野	薄井
						2221	通信開始合図の送信実装	宇井	菊池
				2220	通信同期制御の実装	2222	BPM・音量情報送信実装	宇井	菊池
						2223	音楽・楽器同期の実装	宇井	菊池
						2224	観覧車の制御	宇井	菊池
				2230	楽曲表現機能の制御実装	2231	LEDマトリクス制御	山口	佐藤
						2241	各楽器の音色の再現	梅野	薄井
				2240	processing処理の実装	2242	演奏情報からの発音機能実装	梅野	薄井
3000	テストフェーズ	3100	単体テスト	3110	processingのテスト	3111	各音色の再現確認	梅野	薄井
						3112	発音機能の確認	梅野	薄井
						3121	演奏開始の合図送信確認	宇井	菊池
				3120	指揮デバイスのテスト	3122	BPM・音量の変化確認	宇井	菊池
						3131	LEDマトリクスの表示確認	山口	佐藤
						3141	レーザー受信確認	宇井	菊池
				3130	楽曲表現機能のテスト	3142	演奏情報の送信確認	梅野	薄井
						3151	モータ動作確認	山口	佐藤
				3140	楽器のテスト	3152	同期用レーザー点灯確認	山口	佐藤
						3153	回転速度変更確認	山口	佐藤
		3200	結合テスト	3150	中継機(観覧車のテスト)	3211	開始信号の受信確認	宇井	菊池
						3212	BPM・音量変化の反映確認	宇井	菊池
						3221	各楽器の発音確認	梅野	薄井
				3210	指揮デバイスと中継機の接続テスト	3231	BPM・音量変化の発音確認	梅野	薄井
						3241	LEDマトリクスの反映確認	山口	佐藤
						3242	観覧車によるBPM変化確認	山口	佐藤
				3220	楽器とprocessingの接続テスト				
				3230	指揮デバイスと中継機・PCのテスト				
				3240	指揮デバイスと中継機・楽曲表現のテスト				

図 1.8 WBS を基にした役割分担表

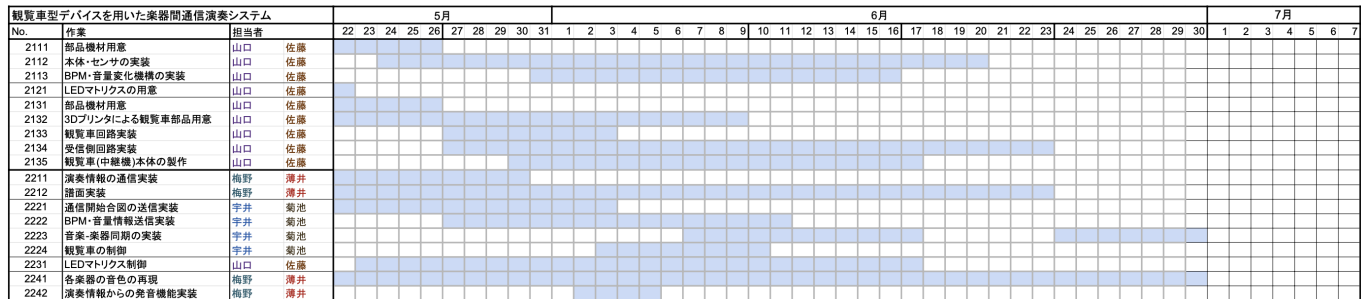


図 1.9 実装フェーズのガントチャート

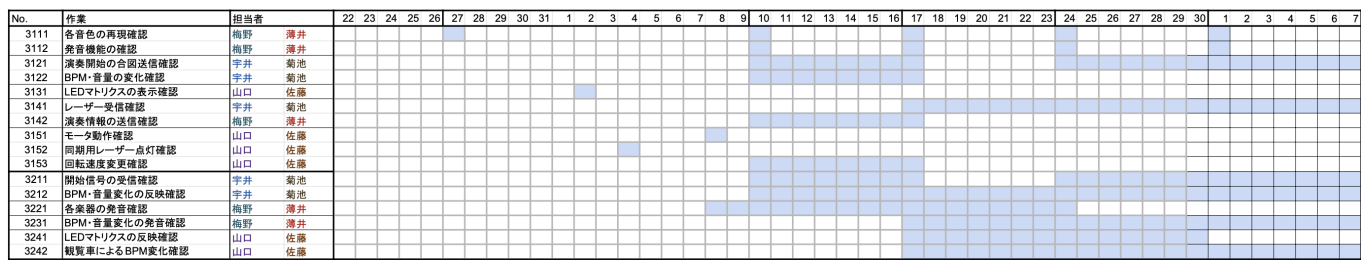


図 1.10 テストフェーズのガントチャート

1.4 検証と妥当性確認: V&V 計画

1.4.1 プロジェクトの成果物に対する有効指標 MOE

本プロジェクトの成果物に対する効果指標 MOE(Measure of Effectiveness) は以下の 4 点とする。作成するシステムは、「指揮デバイスと観覧車を用いた、空間的フィードバックにより高い没入感を実現する」という目的をもつため、人が知覚や体験が期待通りの効果を発揮するかを評価軸とする。

- ・ 輪唱のタイミングに対して不快に感じる程度の遅延がないか
- ・ ユーザの操作が正しく反映されるか
- ・ 音色の再現ができていないか
- ・ エンターテインメント性があるか

1.4.2 有効指標 MOE に対応する性能指標 MOP

- ・ 遅延時間が 50ms 未満 (輪唱のタイミング)
- ・ 99%の処理成功率 (ユーザの操作)
- ・ 音色の対照実験でスコア 4.0 以上 (音色の再現ができていないか)
- ・ アンケートから CSAT スコア 75%以上 (エンターテインメント性があるか)

効果指標 MOE に対応する性能指標 MOP として、以下の 4 点を示す。

まず、輪唱のタイミングに対して不快に感じる程度の遅延がないかを評価する MOP として、遅延時間が 50ms 未満であることを目指す。50 ms 未満という数値を決定するにあたり、ハース効果を根拠として採用する。ハース効果とは、類似した二つの音が時間差で知覚された際、その時間差が一定時間内であれば、聴覚がそれらを一つの音として知覚し、先に到達した音源から発生した音だと判断する現象である [9]。そのため、作成するシステムの輪唱において、それぞれの楽器の演奏に 50ms を超えるズレが発生すると人は演奏がずれていると知覚すると考えられる。そのため、本システムの輪唱の遅延時間は 50ms 未満を技術的目標とする。

次に、ユーザの操作が正しく反映されるかを評価する MOP として、99%の処理成功率を目指す。ここで処理成功率の目標を 99%(ツーナイン) に設定した根拠は、一

一般的な web サービスなどの開発環境において目標とされる数値のためである [10]. 本プロジェクトで作成するシステムは、ブレッドボード上で稼働する一つのシステムであるため、テスト環境での運用に近い. そのため、ツーナインを参考に、作成するシステムの全体の処理成功率は 99%を目標とする.

次に、音色の再現ができているかを評価する MOP として、複数人に作成した音色を聞いてもらい主観評価で再現ができていると評価されることを目指す. 具体的な評価方法としては、複数人に対して、実際の楽器音と作成した楽器音を聴かせて、それらがどれだけ似ていたかを以下に示す 1~5 で回答してもらい評価する.

- ・ 1: 全く似ていない
- ・ 2: あまり似ていない
- ・ 3: どちらともいえない
- ・ 4: ある程度似ている
- ・ 5: 非常に似ている

提示するサンプルと作成した楽器音は音色のみが異なる対照実験を行うようにし、それぞれピアノ、ストリングス、フルート、ドラムセット (キックドラム、スネアドラム、ハイハット) に対して評価を行わせる. 具体的な目標値としては、それぞれの音色についての回答結果に対して似ているとされるスコア 4.0 以上と設定する. ここで、上記に示すように本評価はリッカート尺度のため、回答の有効処理範囲を考えそれぞれ中央値と最頻値のスコアを用いる.

次に、エンターテインメント性があるかに対応する MOP として、複数人にアンケートを取り、顧客満足度 CSAT で 75%以上の範囲を目指す. 本プロジェクトで作成するシステムでは、エンターテインメント性を評価するために一般的な顧客満足度 CSAT(Customer Satisfaction Score) を導入して、独自の評価軸に基づいたアンケート調査を実施する. エンターテインメント性の評価をするため、以下に示す質問を設計した. 各質問を 5 段階 (1: 大変不満, 2: 不満, 3: どちらともいえない, 4: 満足, 5: 大変満足) のリッカート尺度で回答させる.

- ・ 観覧車の回転と楽器が演奏をしている様子を見て、空間的な楽しさや驚きに満足したか?
- ・ 指揮棒を振ったり、可変抵抗器を操作して、演奏のテンポなどがリアルタイムに変化する操作感到満足したか?

- ・光と音の動きが一体になったアトラクションとして、総合的な楽しさに満足したか？

アンケートの回答の評価としては、各質問において 4(満足) または、5(大変満足) と回答した回答者の割合を算出し、これを各項目の CSAT スコア (%) とする。CSAT スコアの計算式は (1.1) 式の通りである。

$$CSAT \text{スコア} = \left(\frac{\text{4または5の回答者数}}{\text{回答者数}} \right) \times 100 \quad (1.1)$$

算出された CSAT スコアは、一般的な企業のスコアの平均が 76.7%であることから、この数値と同等の 75%を目標値として設定する [11]。

1.4.3 システムテストで評価する性能指標 MOP

システムテストフェーズでは、すべてのコンポーネントを統合した「システム全体」の完成状態で、最終的な MOP および MOE の評価実験を実施する。評価項目および具体的な手法は以下の通りである。

- ・操作の反映度およびエンターテインメント性の主観評価 (MOP 目標：処理成功率 99.0%以上, CSAT スコア 75.0%以上)

開発メンバー以外の第三者を被験者（複数人）として、実際にシステムを操作してもらうユーザーテストを実施する。操作ログから入力に対するシステムの処理成功率を算出すると同時に、体験後に 5 段階リッカート尺度のアンケート調査を行い、式 (1.1) を用いて算出された CSAT スコアが目標値に達しているかを評価する。

- ・音色の再現度評価 (MOP 目標：中央値および最頻値が 4.0 以上)

自作した音源（ピアノ、ストリングス 4 種、フルート、ドラム）の単体出力音を録音する。複数人の被験者に対し、実際の楽器音と自作音を聴取させる。

各楽器の音色について、1～5 の 5 段階評価で回答を得る。集計データの中央値および最頻値が「ある程度似ている」に相当する 4.0 以上を達成しているかを評価する。

1.4.4 結合テストで評価する性能指標 MOP

結合テストフェーズでは、観覧車本体のハードウェア実装と制御プログラム、および BPM 指令を行う通信部を結合した段階で、主に物理的な同期性能の検証を行う。

- ・ 輪唱タイミングの遅延計測 (MOP 目標：50ms 未満)

演奏開始信号が送信されてから各楽器が音を出すまでの時間、および楽器間の演奏ズレを millis 関数などを用いてミリ秒単位で計測し、ハース効果の許容限界である 50ms 未満を満たしているかを検証する。

- ・ 演奏中の BPM および音量変化対応 (MOP 目標：処理成功率 99.0%以上)

演奏中に指揮機を操作し、実際に 5 段階の BPM に対応し発音ができるかを検証する。また、音量に対しても 5 段階で変化させ、発音される音量が変化しているかを検証する。

1.4.5 単体テストで評価する性能指標 MOP

単体テストフェーズでは、システムを結合する前の、各要素単体での基本性能を個別に評価する。

- ・ 各種センサー・入力デバイスの基礎精度検証

指揮棒の判定プログラム、可変抵抗器のアナログ読み取り値、および照度センサによるレーザー光読み取りを、シリアルモニタ等を用いて単体で計測し、処理精度を確認する。

- ・ 音色の再現度評価 (MOP 目標：中央値および最頻値が 4.0 以上)

自作した音源（ピアノ、ストリングス 4 種、フルート、ドラム）の単体出力音を発音する。プロジェクトのチーム内で、実際の楽器音と自作音を聴取させる。各楽器の音色について、1～5 の 5 段階評価で回答を得る。集計データの中央値および最頻値が「ある程度似ている」に相当する 4.0 以上を達成しているかを評価する。

1.4.6 プロジェクト中に監視する技術性能指標 TPM

プロジェクトの実行中に監視する技術性能指標 TPM(Technical Performance Measure) として、以下の 4 項目を定義する。これらは本システムの核心である「演

奏情報通信機能」「観覧車の同期動作」「音響制御」の技術的リスクを追跡するための定量的な指標である。

- ・ **1. 入力・判定精度 (%)**

指揮棒を振った際の判定精度，および BPM・音量変更用可変抵抗器や照度センサの計測精度をそれぞれ計測する。

- ・ **2. 通信・制御遅延時間 (ms) とパケット到達精度 (%)**

指揮棒操作や音量変更，演奏開始信号が送信されてから，観覧車および各楽器用の Arduino で受信・反映されるまでの遅延時間 (ms) とデータ受信成功率 (%) を計測する。

- ・ **3. 輪唱演奏の同期時間差 (ms)**

演奏開始信号が送信されてから楽器が実際に音を出すまでの時間，および各楽器が演奏を開始した時間と，輪唱として理想的な開始時間との時間差を計測する。

- ・ **4. 音色の再現度 (類似度)**

自作した音色が，自作の音に対して実際の楽器音との類似度の主観評価を 5 段階で計測する。なお，計測方法としてはチーム内での聴取実験から算出する。

これらの定量的評価を毎週の授業での報告会で計測することで，技術的リスクの早期発見，定量的な進捗管理を行う。これにより，最終的なシステム性能要求 MOP の達成を確実なものとする。

1.4.7 プロジェクト中に監視する指標 TRL

本プロジェクトの開発プロセスにおいて，技術的リスクの早期発見と進捗状況の客観的な可視化を目的として，技術成熟度レベル TRL(Technology Readiness Level)を導入する。各開発フェーズにおける技術の成熟度は，以下の 4 段階に基づいて監視を行う。

- ・ 1. 概念レベル：アイデアや原理が確立されており，説明できるレベル
- ・ 2. 基本動作レベル：単体で動作確認が可能なレベル
- ・ 3. 結合動作レベル：機能を結合し，実際の使用環境に近い環境で動作確認が可能なレベル
- ・ 4. 実運用レベル：実際の使用環境で安定的な稼働が確認できるレベル

本プロジェクトでは，毎週の授業中の報告会で各自が担当するシステムの構成要素が

現在どの段階にあるかを報告し、進捗状況を管理する。これにより、主観的な進捗報告を排除し、定量的にプロジェクトを監視する。

1.5 資源・調達管理

資源・調達管理について説明する。今回作成するにあたってハードウェアで使用する電子部品は貸し出し機材と各自で購入し、作業開始日の5月26日までにすべて必要な機材を揃える。また、観覧車型中継機デバイスや指揮棒デバイスの工作は3Dプリンタを用いて製作を行う。この工作のための機材も作業開始日の5月26日までに作成し、開始日にすぐ作成に移れるような状態にする。

1.6 リスク管理

リスク管理について説明する。今回製作にあたって想定されるリスクとして、以下を挙げる。

- ・ 想定より実装に時間がかかる
- ・ ハードウェアがうまく動作しない
- ・ メンバー間で進捗に差が出る
- ・ グループ内の情報の伝達不備

それぞれのリスクに対して、以下の対策を考える。

- ・ 実装の際にバッファを設け、時間がかかることを前提としたマネジメントを行う。
- ・ データシート等から使用する素子の電気的特性を把握しておく。
- ・ 進捗が捗っているメンバーが、手助けを行う。また、確認した段階で現在の進捗と実行プロジェクトを見直す。
- ・ 連絡を密に行い、認識のすり合わせを行う。また、作業者自身に説明させることで、認識を確認する。

第 2 章 システムの基本設計

2.1 機能一覧

今回提案するものは、一つ目に信号を送る指揮デバイスの設計、二つ目に指揮デバイスから信号を受け取って動作する中継機デバイス、三つ目に楽器デバイスの設計である。本節では指揮デバイスと中継機デバイス、楽器デバイスの機能について説明する。

始めに指揮デバイスについて説明する。指揮デバイスとは演奏全体を統括するデバイスであり、主に以下の三つの機能を有する。第一の機能は、演奏開始トリガーの送信である。ユーザが指揮デバイスに取り付けられたボタンを押下すると、観覧車デバイスと楽器デバイスへ演奏開始トリガーを送信する。

第二の機能は、楽曲の BPM 変更である。ユーザが可変抵抗器を操作した後、指揮棒を振ると新しい設定 BPM 情報が中継機デバイスである観覧車へ送信される。これにより、観覧車のモータ回転速度がこの値に追従して変化する。そして、その回転に伴うレーザーの通過光を介して各楽器デバイスに情報が伝達され、リアルタイムに楽曲の BPM が変更および同期される。

第三に、楽曲の音量変更である。ユーザが可変抵抗器を操作した後、指揮棒を振ると音量情報が各楽器デバイスへマルチキャスト送信されることで楽曲の音量がリアルタイムに変更される。

次に、中継機デバイスについて説明する。中継機デバイスとは、指揮デバイスからの情報を受け取り観覧車を作動させ、現在の BPM を表示するデバイスであり、主に以下の二つの機能を有する。第一の機能は、上記で書いた BPM に合わせて回転を行い、楽曲の BPM を制御する機能である。観覧車に設置してあるレーザー受光のテンポで BPM を管理し、楽曲のテンポを視覚的にも理解できる機能である。第二に、LEDMatrix の表示である。ここに指揮デバイスから受け取った BPM を表示し、視覚的に現在流れている曲のテンポを理解できるような機能である。LEDMatrix は Arduino Uno R4 WiFi に付属したもので表示する。

最後に、楽器デバイスについて説明する。楽器デバイスは観覧車デバイスの回転状態を基準として、自律的に演奏を行うデバイスであり、主に以下の四つの機能を有する。第一の機能は、演奏開始の検知である。指揮デバイスが振り動作を検知すると、無線通信で楽器デバイスへと演奏開始情報が送信され楽曲の再生を開始する。第二の機能は、楽曲の BPM リアルタイム同期である。演奏開始後もレーザー光の通過間隔を常時計測し、その間隔から現在の BPM を逐次算出する。第三の機能は、楽器の音色再生である。Arduino と USB ケーブルで接続された PC 上の Processing を用いることで、各楽器の音色を出力し、演奏を再現する。第四に、楽曲の音量リアルタイム反映である。指揮デバイスから受け取った音量情報を Processing へ送信し、再生中の出力音量へ反映する。

2.2 システム構成図

2.2.1 システム全体の構成図

今回提案する成果物全体の構成図は、図 2.1 の通りである。指揮棒の振動を加速度センサーで感知した際に、Arduino1 から無線で演奏情報を送信する。その後、情報を受け取った Arduino2 が中継機である観覧車デバイスを作動させる。演奏の開始には指揮デバイスのボタンが押下された時、Arduino3～6 の楽曲演奏を開始する。観覧車デバイスが作動した時に、指揮デバイスから受け取った情報を基に BPM を LEDMatrix で表示を行う。観覧車が作動した時、観覧車の 8 箇所を設置しているレーザーが楽器演奏デバイスにある cds セルに光を送信し、光を受信すると楽曲が PC の Processing から流れる仕組みである。そして、流れてくるレーザーのテンポに従って BPM が変化するという仕組みである。

2.2.2 指揮デバイスの構成図

指揮デバイスのシステム構成図を図 2.2 に示す。本デバイスは内蔵の加速度センサーによりデバイスの振り動作を検知すると、観覧車デバイスと楽器デバイスにそれぞれに演奏情報が送信され、観覧車デバイスの回転が開始される。演奏の開始は、指揮デバイスのボタンを押すことで行う。また、ユーザによる操作により楽曲の BPM と音量を変更できる。

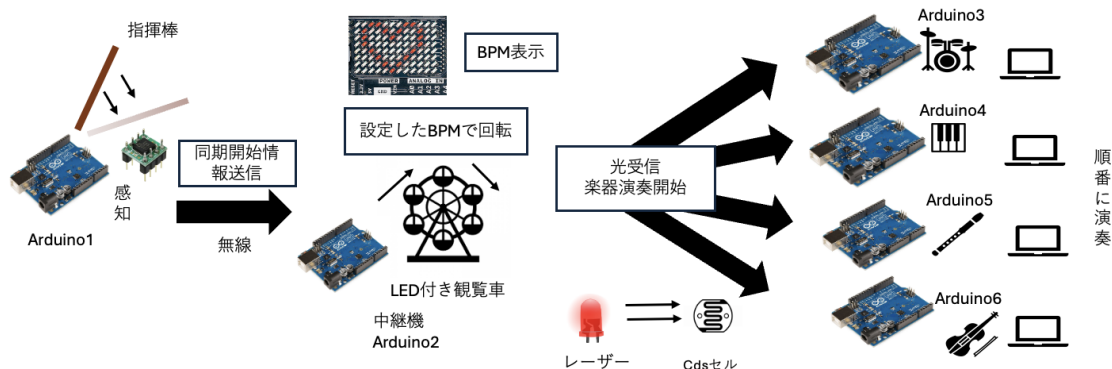


図 2.1 全体の構成図

2.2.3 中継機の構成図

中継機デバイスの構成図は、図 2.3 の通りである。指揮デバイスから信号を受け取った際に、中継機である観覧車デバイスが作動する。指揮デバイスから送信された BPM を元にして観覧車は回転し、LEDMatrix にはその BPM が表示される仕組みである。

2.2.4 楽器デバイスの構成図

最後に、楽器デバイス受光機構のシステム構成図を図 2.4 に示す。本機構は、回転している観覧車デバイスのレーザー光を検知することで楽器の同期を維持する。あらかじめ設定した位置に各楽器デバイスを設置し、観覧車デバイスのレーザーが通過した瞬間に光センサがこの光を検知することで、演奏の同期および BPM の変更が行われる。

2.3 操作インターフェース設計

本節では指揮デバイスの設計および指揮デバイスの演奏開始トリガー送信機構と BPM 変更機構および音量変更機構の操作インターフェース設計、中継機デバイスの表示設計について説明する。なお、楽器デバイスの受光部はユーザの直接操作が存在しないため、操作インターフェース設計の対象外とする。

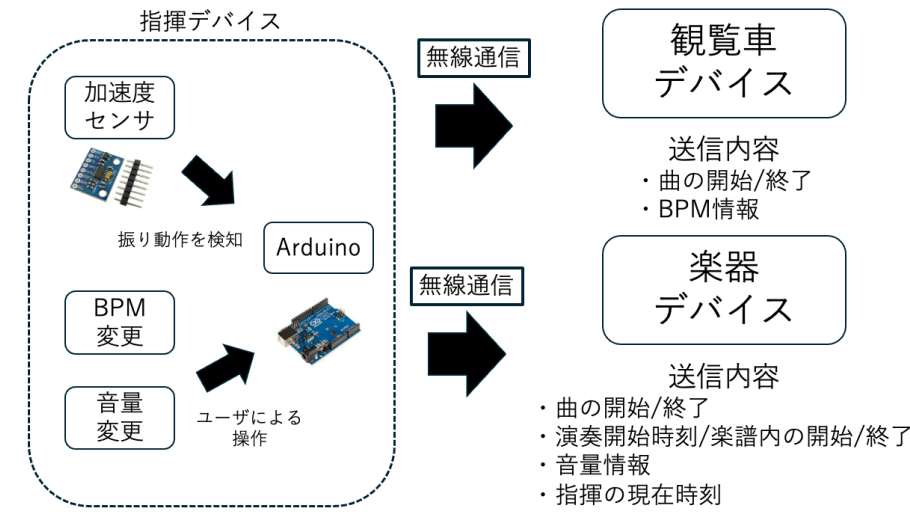


図 2.2 指揮デバイスのシステム構成図

2.3.1 指揮デバイスの設計

指揮デバイスの設計について図 2.5 に示す。本デバイスは、実際の指揮棒のようにユーザが片手で振って操作できるよう、細長い筒状の外装を採用する。また、内部のブレッドボードに加速度センサ、可変抵抗器、ボタンを設置し、ユーザが操作しやすいようスライダー部分は外部へ露出させる構造とする。さらに、指揮デバイスの先端部に加速度センサを設置することで、操作時の重心を安定させユーザビリティを向上させる。

2.3.2 指揮デバイスの演奏開始トリガー送信機構の設計

次に、指揮デバイスの演奏開始トリガー送信機構の設計について図 2.6 に示す。本機構は、ユーザがボタンを押下することで演奏開始トリガーを送信する。ボタンの押下によって変化する電圧を指揮デバイスで検知し、観覧車デバイスへ送信をし観覧車デバイスが動作を開始する。

2.3.3 指揮デバイスの BPM 変更機構および音量変更機構の設計

次に、指揮デバイスの BPM 変更機構および音量変更機構の設計について図 2.7、図 2.8 に示す。本機構は、ユーザがスライド式可変抵抗器を操作することで楽曲の BPM および音量が変更できる。ユーザがスライド式可変抵抗器をスライドし、指揮

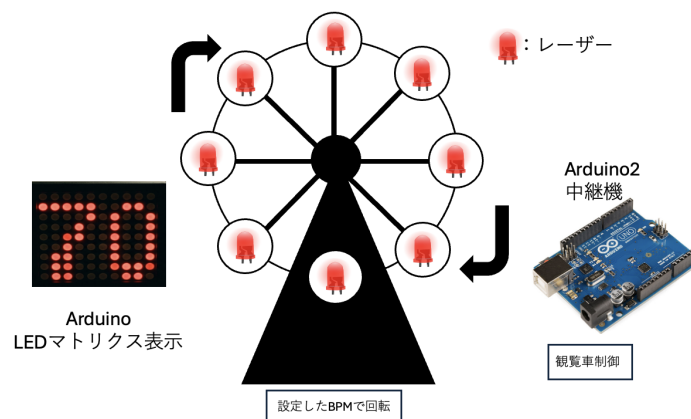


図 2.3 中継機の構成図

棒を振ると新しい設定 BPM 情報が観覧車デバイスへ無線送信される。これにより、観覧車デバイスの回転速度がこの値に追従して変化し、その回転を介して各楽器デバイスへ情報が伝達されることで、楽器の BPM が変更される。この一連の処理によって観覧車デバイスの回転速度と楽曲の BPM が常に同期される仕様である。また音量変更は、BPM と同様にユーザがスライド式可変抵抗器をスライドすると各楽器用 Arduino に情報が送信され変更される。また、楽曲の BPM および音量は五段階に分けて変更できる。

2.3.4 中継機デバイスの表示

最後に中継機デバイスの表示設計について説明する。中継機デバイスでは、受信した BPM 情報に従って観覧車を回転させる。また、LEDMatrix を用いて現在の BPM 値を表示することで、ユーザがテンポを視覚的に確認できるようにする。動作に関しては、上記の図 2.3 の通りである。

2.4 状態遷移図

2.4.1 全体の状態遷移図

今回提案するシステムの状態遷移図を説明する。状態遷移図は図 2.9 の通りである。電源を入れ、ボタンを押下すると観覧車が作動する状態になる。その後回転する際に観覧車にあるレーザーから光を受光し、メロディが順に流れていき演奏する状態

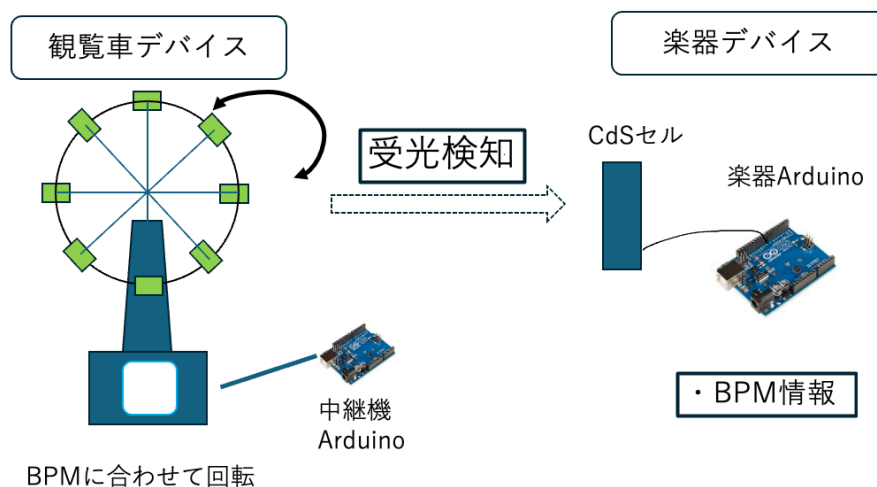


図 2.4 楽器デバイスの受光機構のシステム構成図

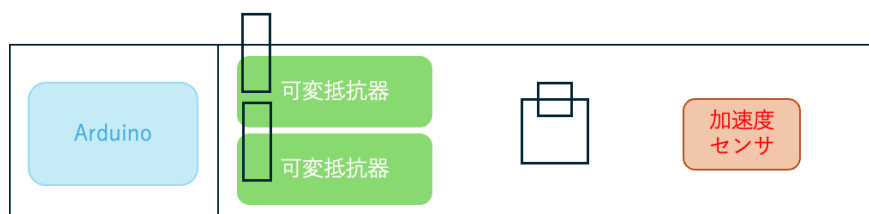


図 2.5 指揮デバイス設計

になる。その間に BPM 変更を行った場合、指揮デバイスを振動させると BPM を更新し、回転速度が変化する。演奏終了すると待機状態に戻り、そのまま電源が切れる流れである。

2.4.2 指揮デバイスの状態遷移図

次に、指揮デバイスにおける状態遷移について説明する。まず、指揮デバイスのシーケンス図を図 2.10 に示す。ユーザが指揮デバイスを上下に振ると加速度センサが動作を検知し Arduino に送信される。この時、値が閾値を超過した場合、観覧車デバイスと無線通信を開始し観覧車デバイスが送信した演奏情報を基に回転する。また、ボタンを押下した時、各楽器デバイスへ楽曲の初期 BPM および音量情報が送信されることで演奏が開始される。

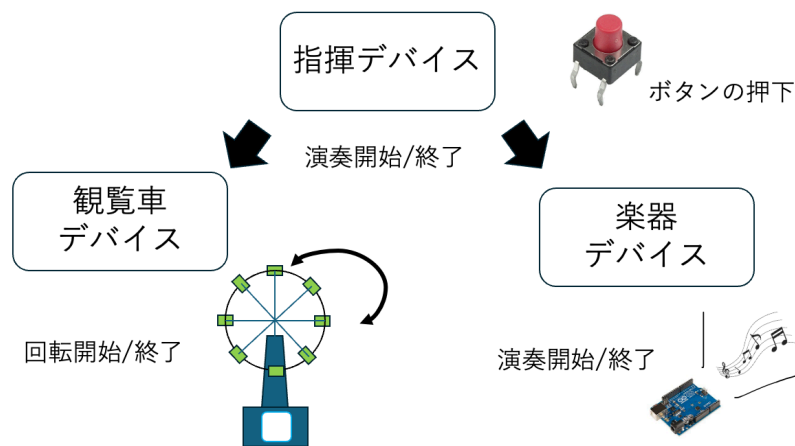


図 2.6 演奏開始トリガー送信機構設計

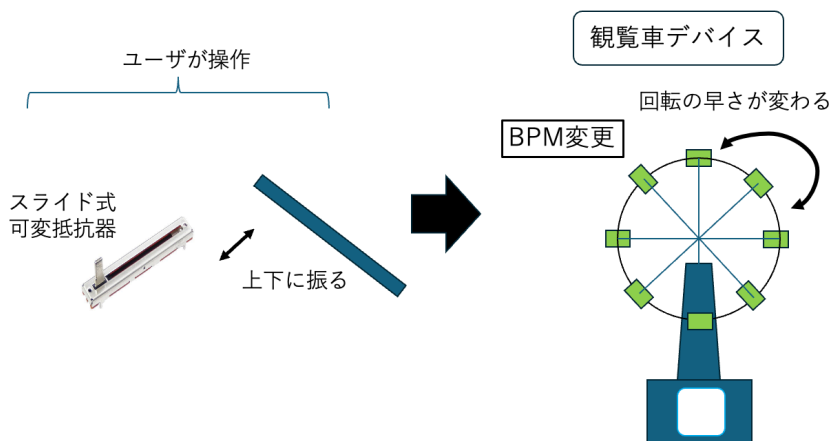


図 2.7 BPM 変更機構設計

2.4.3 楽器デバイス受光機構の状態遷移図

次に、楽器デバイスの受光機構のシーケンス図を図 2.11 に示す。観覧車デバイスのレーザー光が初回に通過した際、光センサが楽器デバイス内の Arduino ヘデジタル信号を送信し、PC から楽器の音色が再生される。演奏中は観覧車デバイスのレーザー光が楽器デバイスの受光部を通過するごとに光センサが Arduino ヘデジタル信号を送信する。この通過間隔から BPM を算出することで、最新の BPM 値を PC へ送信する。

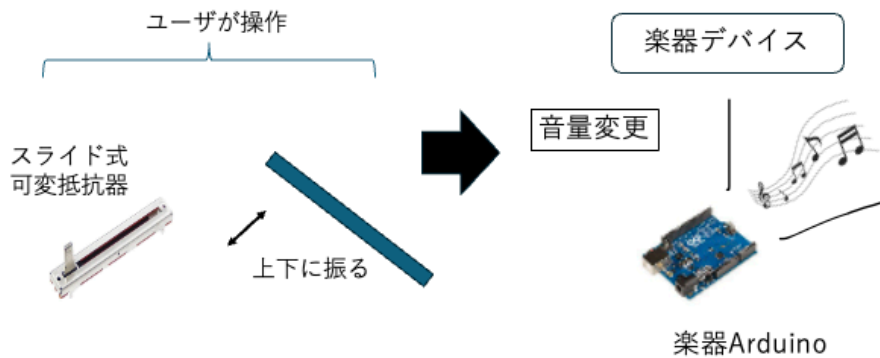


図 2.8 音量変更機構設計

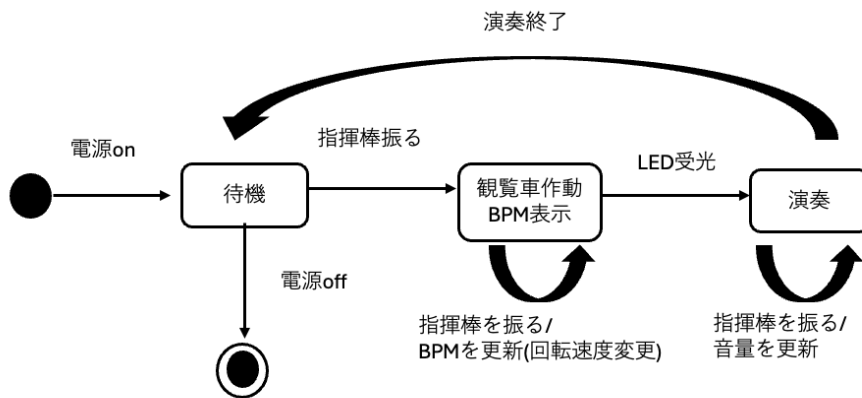


図 2.9 状態遷移図

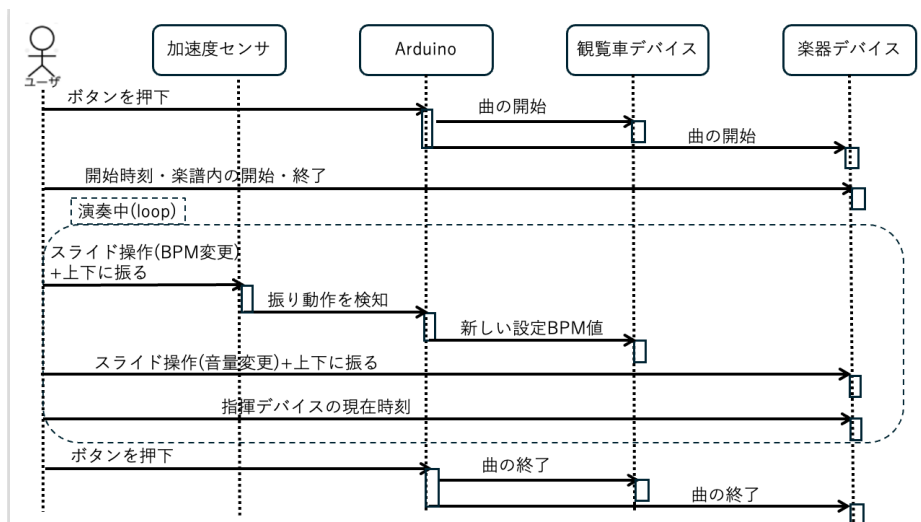


図 2.10 指揮デバイスのシーケンス図

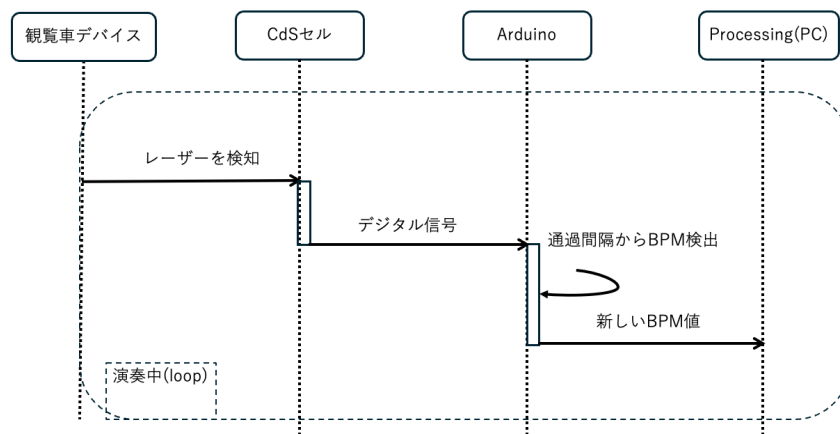


図 2.11 楽器デバイスの受光機構のシーケンス図

第 3 章 システムの詳細設計

3.1 部品一覧

本節では、指揮デバイスおよび中継機デバイス、楽器デバイスの受光部を構成する部品について説明する。指揮デバイスを構成する部品について表 3.1 に示す。デジタル 3 軸加速度センサは指揮デバイスの動きの検知に使用し、スライド式可変抵抗器は BPM および音量変更に使用される。

表 3.1 指揮デバイスに使用な機器

Arduino UNO R4 WiFi	-	1
スライド式可変抵抗器 10 k Ω	-	2
デジタル 3 軸加速度センサ	GY-291	1
タクトスイッチ	DTS-63	1
抵抗器 10k Ω	-	1
ジャンパーワイヤー	-	適宜

中継機デバイスで使用する部分を表 3.2 にまとめる。この内、個別で購入する必要があるものはハイパワーギヤーボックス、ブレッドボード用 DC ジャック DIP 化キット、ボタン電池、ボタン電池ホルダー、QI ケーブル、有極コンデンサーである。

次に、楽器デバイスが受光するための部品について表 3.3 に示す。光センサモジュールとして CdS セルを使用する。CdS セルにレーザー光が照射された際の出力を Arduino のアナログピンへ接続することで、観覧車デバイスのレーザー光の通過を検知する。また、今回はドラム、ピアノ、ストリングス、フルートの 4 種類の楽器の音色を再現するためそれぞれに Arduino を使用し、光センサおよびブレッドボードも 4 台用意する。

表 3.2 中継機デバイスに必要な機材

機材	品番	個数 [個]
Arduino Uno R4 WiFi	-	1
ルーター	-	1
ブレッドボード	-	1
赤色ドットレーザーモジュール	FU650AD5-C6	8
ボタン電池基板取付用ホルダー (CR2477 用)	CH031-2477LF	4
ボタン電池	CR2477	4
スライドスイッチ	SS-12D00-G5	4
モータードライバーモジュール	AE-DRV8835-S	1
ハイパワーギヤーボックス HE	72003	1
ジャンパーワイヤー	-	適宜
ブレッドボード用 2.1mm 標準 DC ジャック DIP 化キット	AE-DC-POWER- JACK-DIP	1
超小型スイッチング AC アダプ ター 5 V 1 A	-	1
QI ケーブル 2P-1Px2	311-262	1
抵抗器 10 Ω	-	1
コンデンサー 0.01 μF	-	1
コンデンサー 0.047 μF	-	1
有極性コンデンサー 100 μF	-	1

表 3.3 楽器デバイスに必要な機器

機材	品番	個数 [個]
Arduino UNO R4	-	4
CdS セル	GL5516	4
ブレッドボード	-	4
ジャンパーワイヤー	-	適宜

3.2 ハードウェア設計

3.2.1 指揮デバイス

指揮デバイスの回路図を図 3.1 に示す。電源仕様は、PC との USB 接続から電力を供給する。また、Arduino 内蔵の電圧レギュレータにより 5V および 3.3V を生成し、各モジュールへ安定した電力を供給する。

加速度センサには GY-291 (ADXL345) を用いており、I2C 通信により Arduino と接続する。CS ピンを Arduino の 3.3V に接続することで I2C モードに設定し、さらに SDO ピンを GND に接続することで I2C アドレスを 0x53 に固定している。また、SDA ピンおよび SCL ピンはそれぞれデータ通信線およびクロック信号線として Arduino の A4, A5 ピンに接続している。加えて、加速度が内部設定した閾値を超過した場合には、INT1 ピンから Arduino の D2 ピンへ割り込み信号が出力される。これにより、指揮デバイスの振り動作を検知し、モータ制御用 Arduino との UDP 通信を開始する。

さらに、演奏開始および終了信号の送信に用いるボタン入力判定回路を実装する。ボタンの誤判定を防止するため 10k Ω のプルアップ抵抗を用いた回路を構成する。ボタンが押されていない状態では、入力ピン D3 はプルアップ抵抗を介して 5V に接続されるため HIGH 状態となる。一方、ボタン押下時には回路が GND に短絡され、入力電位が LOW に遷移する。指揮デバイスのプログラムでは D3 ピンの状態変化を常時監視することで、演奏開始および終了操作を判定する。

3.2.2 観覧車型中継機デバイス

中継機の詳細設計について説明する。図 3.2 について、モータードライバーに Arduino D5, D6 ピンを接続し、PWM 制御を行えるようにする。モータードライバーへの電力供給は Arduino の 5V からと、DC ジャックからの AC アダプター 5V 1A を用いる。モータードライバー及び DC ジャックはピンヘッダーのはんだ付けが必要なため、はんだ付けを行う。ギアボックスは 0.01 μ F のコンデンサーと直接はんだ付けを行う。

次に、観覧車の設計について説明する。作成するために、3D プリンタを用いて作成する。実際に設計に使用するモデルデータを図 3.3 から図 3.8 に示している。観覧

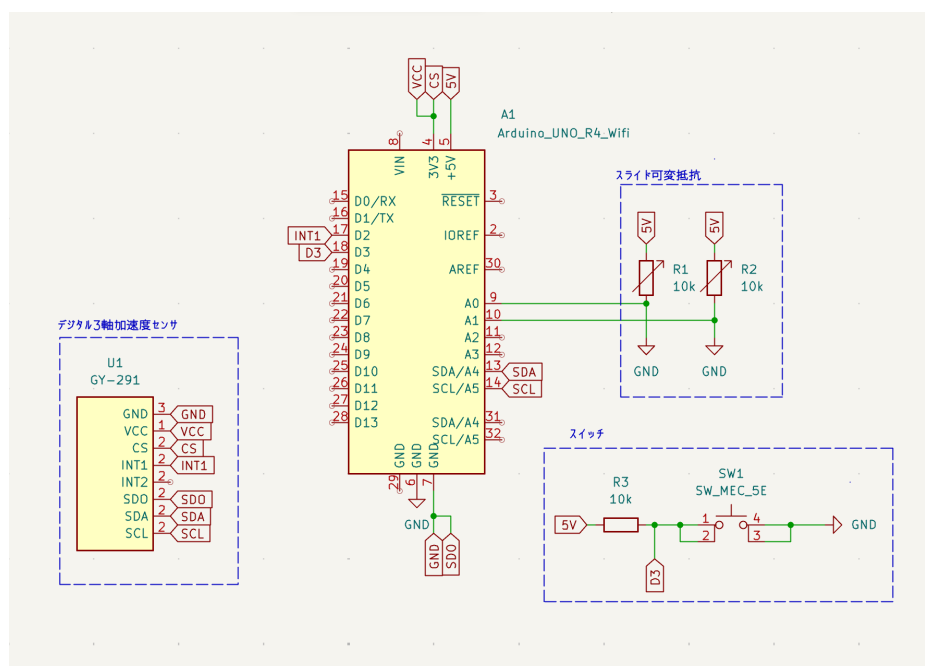


図 3.1 指揮デバイスの回路図

車の直径は約 16cm とし、ボタン電池付きレーザーを 45 度間隔で配置する。そして、観覧車を支える左右の支柱および土台を設計する。観覧車本体は、以下のパーツに分割して設計を行う。

- ・土台 観覧車全体を支える部分であり、内部を空洞化させる。また、支柱を土台に差し込み、固定する役割も持つ。実際に設計する部品は図 3.3 の通りである。
- ・支柱 観覧車の回転軸を左右から支える部品である。回転軸を安定して保持し、観覧車が滑らかに回転できるようにする。回転軸を上から挿入する形であり、軸が折れないようギアボックスモータに繋ぐ側の支柱の支える穴を大きくしている。実際に設計する部品は図 3.4 の通りである。
- ・回転軸 ギアボックスモータの回転を観覧車本体へ伝達する部品である。モータと接続し、観覧車全体を回転させる役割を持つ。支柱に固定できるよう、支柱の位置の軸部をへこませて固定できるようにしている。実際に設計する部品は図 3.5 の通りである。
- ・回転リング 観覧車の円形部分であり、レーザー側とその反対側で 2 個作成する。モータによって回転し、レーザーによる光信号送信を行う部分でもある。角度 45 度おきにレーザーを設置する穴を用意し、そこにレーザーをはめ込み固定する。もう一つの円盤に電池基板を設置し電力供給する。実際に設計する部品は

図 3.6, 3.7 の通りである.

- ・ 受光部 レーザーの延長線上に設置する照度センサーの受光部である. 回転リングから平行な位置に円盤を設置し, 中央にギアボックスモータを設置する台を設ける. 90 度おきに円盤にくぼみを作り, センサーを設置できるようにした. 受光部の円盤によりレーザー光が後ろに漏れないという設計となっている. 実際に設計する部品は図 3.8 の通りである.

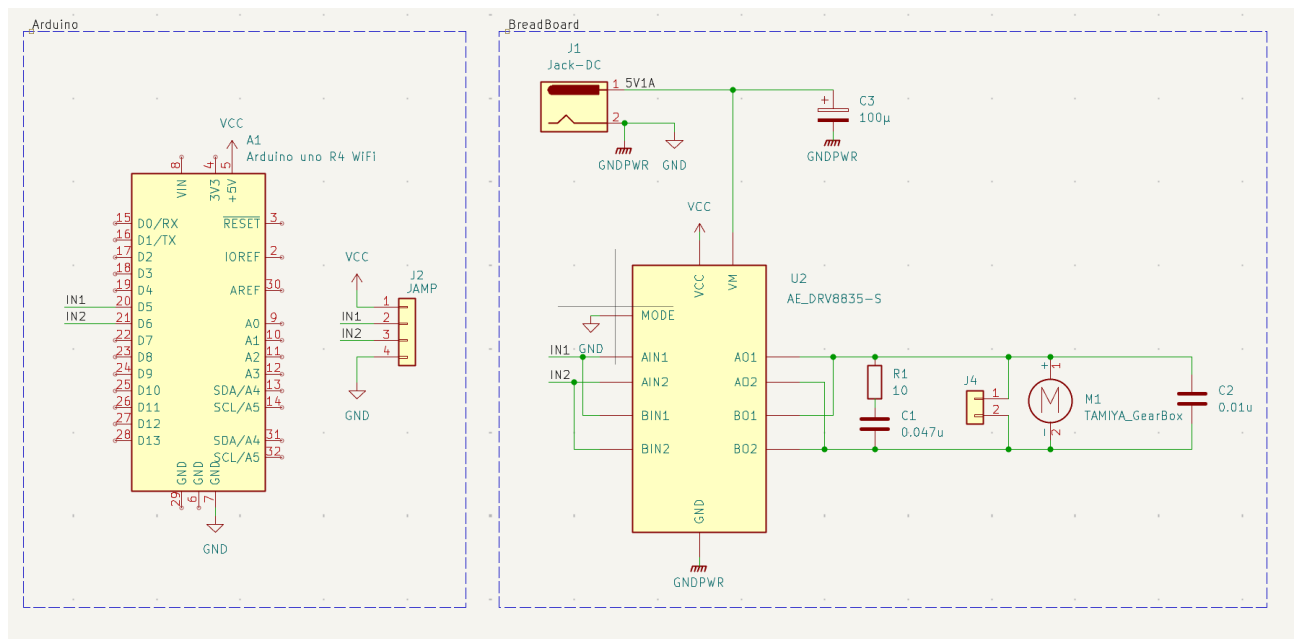


図 3.2 観覧車型中継機デバイス回路図

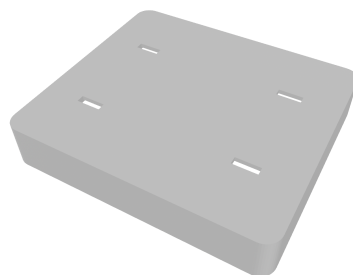


図 3.3 設計に使用するモデルデータ (土台部)

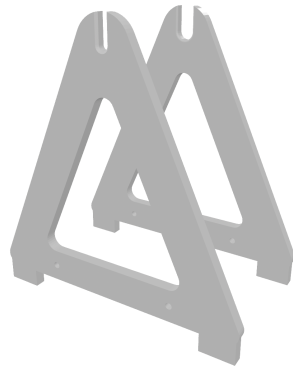


図 3.4 設計に使用するモデルデータ (支柱部)

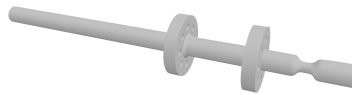


図 3.5 設計に使用するモデルデータ (回転軸部)

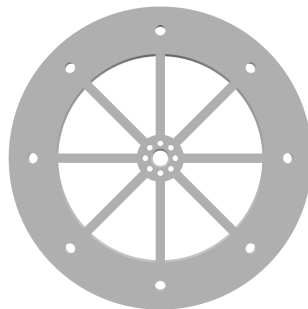


図 3.6 設計に使用するモデルデータ (回転リング部 1)

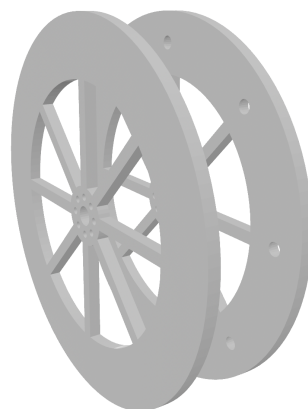


図 3.7 設計に使用するモデルデータ (回転リング部 2)

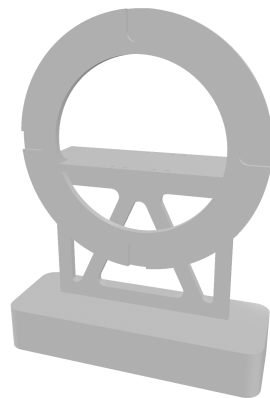


図 3.8 設計に使用するモデルデータ (受光部)

3.2.3 レーザー光送信装置

レーザー光送信装置の詳細設計について説明する。図 3.9 について、回路図の通りにボタン電池とレーザーをはんだ付けする。これらを観覧車のリングに 45 度間隔に装着する。電源としては図 3.9 に示すようにボタン電池からの 3V 給電を行う。ここで、レーザーについて、データシートより定電流源回路を組み込んであるため、現在の設計であれば外部抵抗は必要とせず、素子を安全に利用することができる [?].

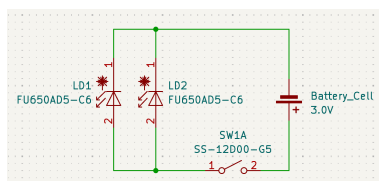


図 3.9 レーザー回路図

3.2.4 楽器デバイス受光部装置

楽器デバイスの受光部の回路図を図 3.10 に示す。電源仕様は、PC との USB 接続から電力を供給する。光センサは Arduino の 5V ピンから給電されるため、外部電源は不要である。また、光センサが光を受光した瞬間に A0 のアナログピンにセンサ値を出力する。このセンサ値の変化を Arduino で処理することで光を検知する仕様である。

3.3 Processing

本節では、音色の波形生成と発音を担うソフトウェア（Processing）の詳細設計について述べる。

3.3.1 システムアーキテクチャとファイル構成

Processing のプログラムは可読性を考慮し、機能ごとにファイルを分割している。ファイル構成を表 3.4 に示す。

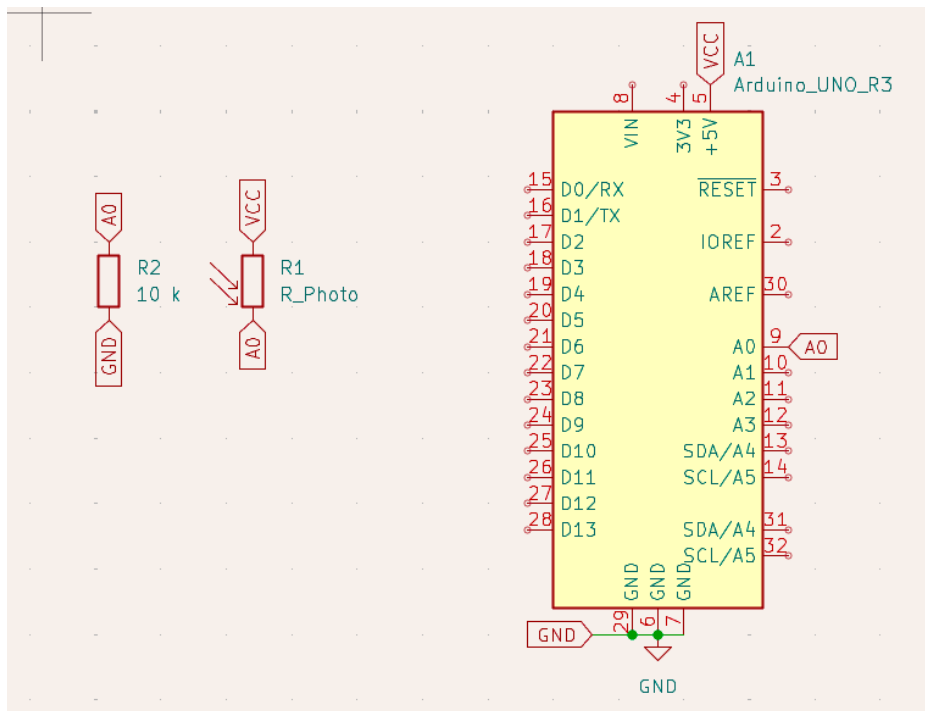


図 3.10 楽器デバイスの受光部の回路図

表 3.4 Processing のファイル構成

ファイル名	説明
MainCode.pde	プログラムを総轄する SystemController.
DataUtils.pde	波形生成に使用する JSON ファイルのデータを扱う DataUtils.
InstrumentPlayer.pde	発音命令を委譲し出力する InstrumentPlayer.
AddEffect.pde	特定の楽器群に空間的な反響音を付加するエフェクトクラス.
NoteSampler.pde	デバッグおよびシステム検証用として、Processing 内で完結して楽譜データを自動再生するクラス.
(楽器名).pde	各楽器の波形を生成する.
data / xxx.json	JSON ファイルは data ディレクトリに格納する.

MainCode.pde では、Processing のプログラム全体を統括し、setup メソッドや serialEvent メソッドなどを記載する。DataUtils.pde では、data ディレクトリに格納された JSON ファイルを読み込み、配列として保持する。この配列は、

GetDynamicProfile メソッドや、文字列でデータ種別を指定する統合ゲッター GetLUT メソッドを介してのみ取得できる設計とする。InstrumentPlayer.pde では、発音命令を受けて (楽器名).pde にある各楽器の波形生成用クラスを呼び出し、生成した波形を出力する。また、AddEffect.pde では、特定の楽器専用の出力バスに空間的な残響を付与し、複数人がホールで奏でるようなリアルなアンサンブル感を再現する。開発および検証段階においては、NoteSampler.pde を利用することで、Arduino からのシリアル通信を介さずに内蔵の楽譜データを用いた自動演奏テストを可能としている。

本システムは複数のクラス群によって構成されており、その全容を図 3.11 に示す。

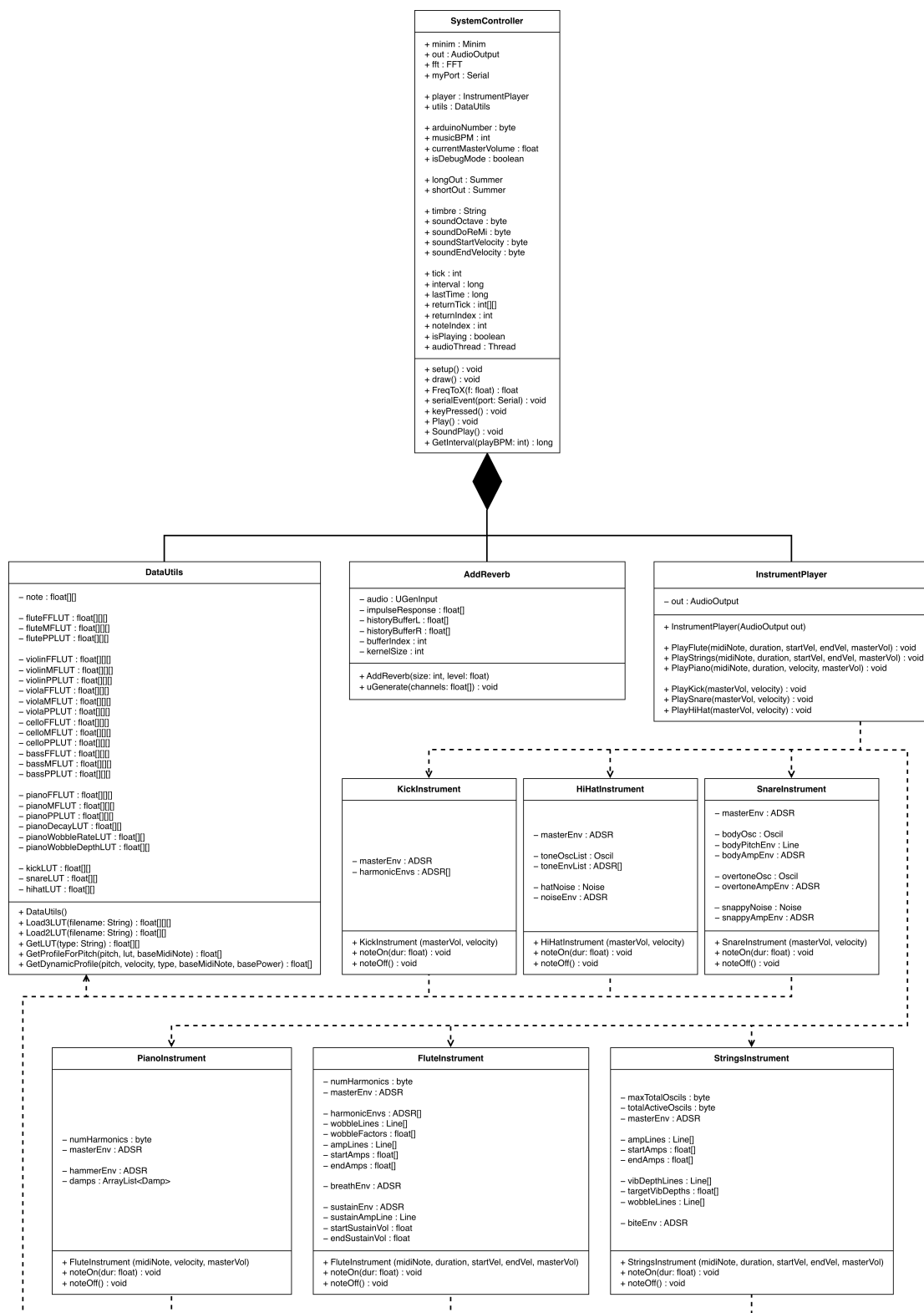


図 3.11 Processing のクラス図

全体の制御を担う `SystemController` がシリアル通信の受信や描画処理を統括し、発音命令は `InstrumentPlayer` クラスを経由して各楽器固有のクラス (`StringsInstrument`, `FluteInstrument` など) へ委譲される。合成に必要なデータなどは `DataUtils` クラスに集約されており、最終的な音声信号は必要に応じて空間エフェクトを経由して出力される設計となっている。

3.3.2 通信プロトコルとエラーハンドリング

Arduino から PC へのデータ送信には有線のシリアル通信を採用するものとし、その形式と送信タイミングを表 3.5, 表 3.6, 表 3.7 に示す。

表 3.5 発音指示パケット (ヘッダ N)

形式	N,pitch1,duration1,startVel1,endVel1, pitch2,duration2,startVel2,endVel2,... \n
例	N,60,48,80,112,64,48,80,112\n (ドとミの和音を長さ 48Tick で、共に mf から ff へ変化させながら同時に鳴らす)
送信タイミング	Arduino の現在 Tick が、楽譜の timing に到達した瞬間.

表 3.6 BPM 変更指示パケット (ヘッダ B)

形式	B,bpmValue\n
例	B,130\n
送信タイミング	指揮から BPM の変更を受信した瞬間.

表 3.7 ボリューム変更指示パケット (ヘッダ V)

形式	V,volumeValue\n
例	V,99\n
送信タイミング	指揮から全体のマスターボリューム変更を受信した瞬間.

先頭にデータの種類を表すヘッダ文字を付与したカンマ区切りの文字列とし、ヘッダ文字が N の場合は発音指示、B の場合は BPM 変更指示、V の場合はボリューム変

更指示であると判別する。和音の同時発音性を保証するため、同一タイミングの音符は 1 行のパケットにまとめて送信する。

表 3.8 エラーおよび境界値の処理仕様

対象項目	判定条件	処理内容
パケット (N)	要素数からヘッダを除いた値が 4 の倍数でない	破棄（欠損とみなす）
パケット (B, V)	ヘッダを含む要素数が 2 でない	破棄（欠損とみなす）
各パラメータ	定められた許容範囲（0～127 等）を逸脱	constrain() による補正

本システムでは、通信環境によるパケットの欠損や想定外の数値入力に対処するため、表 3.8 に示す通りのエラーハンドリングを実装する。シリアル通信においては、ノイズ等の影響によりデータの一部が欠損する可能性がある。特に発音指示パケット（ヘッダ N）は、ノート数に応じてデータ長が可変となるため、分割後の要素数を確認することで、受信したデータがプロトコルに準拠しているかを判定している。不整合が検知された場合は、誤作動を防止するために当該パケットを即座に破棄する設計とした。

具体的な処理手順として、シリアル通信で受信した文字列は、末尾に含まれる改行コード（\n や \r）が後のデータ変換処理において数値変換エラーを引き起こす原因となる。そのため、パケットを受信した直後に trim 関数を用いて空白文字や改行コードを確実に除去した上で、split 関数によるカンマ分割を行う仕様としている。分割後の要素数からヘッダを除いた値が規定数（発音指示であれば 4 の倍数）に合致するかを確認することで、不完全なパケットを安全に弾くことができる。BPM やボリュームなどの数値データについては、指揮側からの入力値がソフトウェア上の許容範囲を超過する場合が考えられる。これに対し、単にエラーとして捨てるのではなく、constrain 関数を用いて上限・下限値に丸め込みを行うことで、システムの動作継続性を優先させる。

3.3.3 システム統括とデータ構造

3.3.3.1 SystemController (システム統括・状態管理)

SystemController のフローチャートを図 3.12, 3.13, 3.14, 3.15, 3.16 に示す.

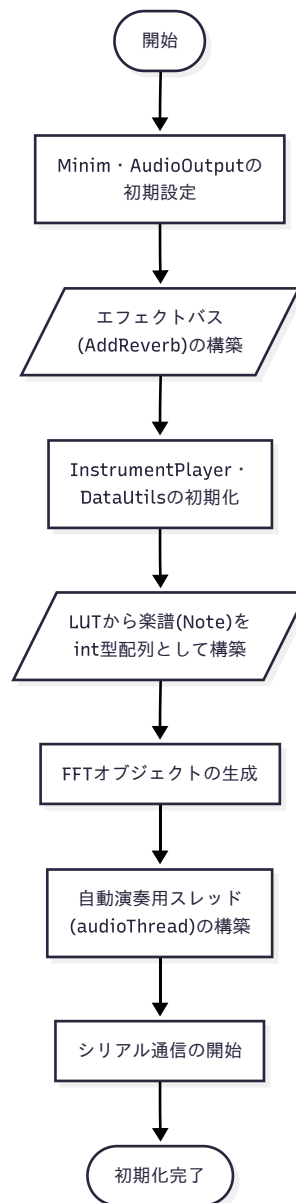


図 3.12 SystemController のフローチャート (setup)

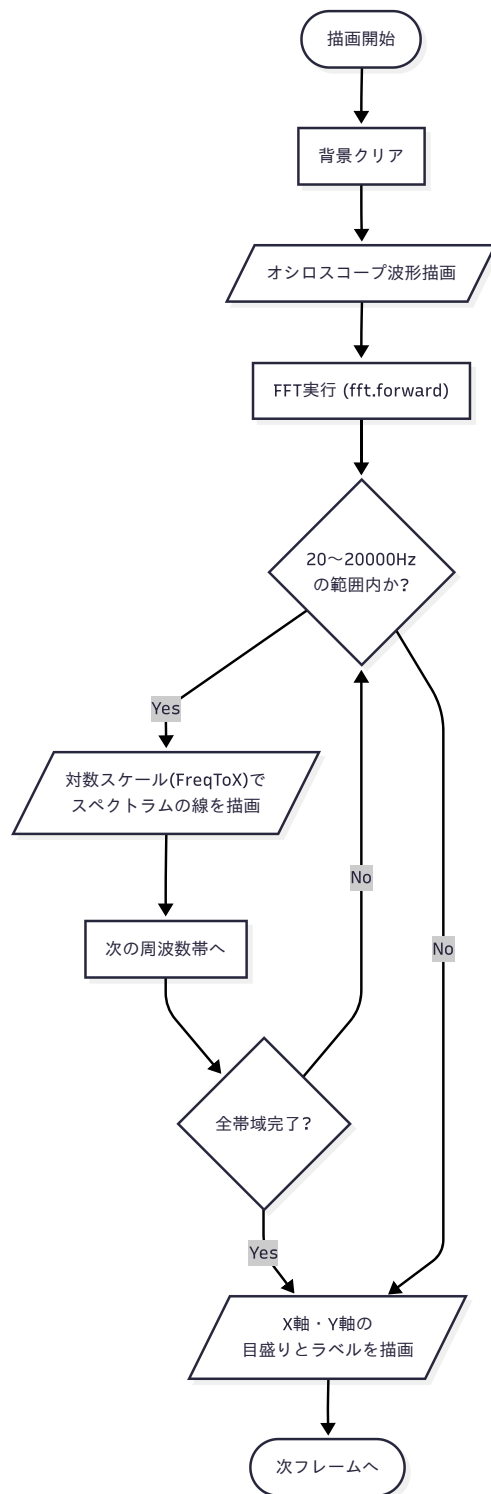


図 3.13 SystemController のフローチャート (draw)

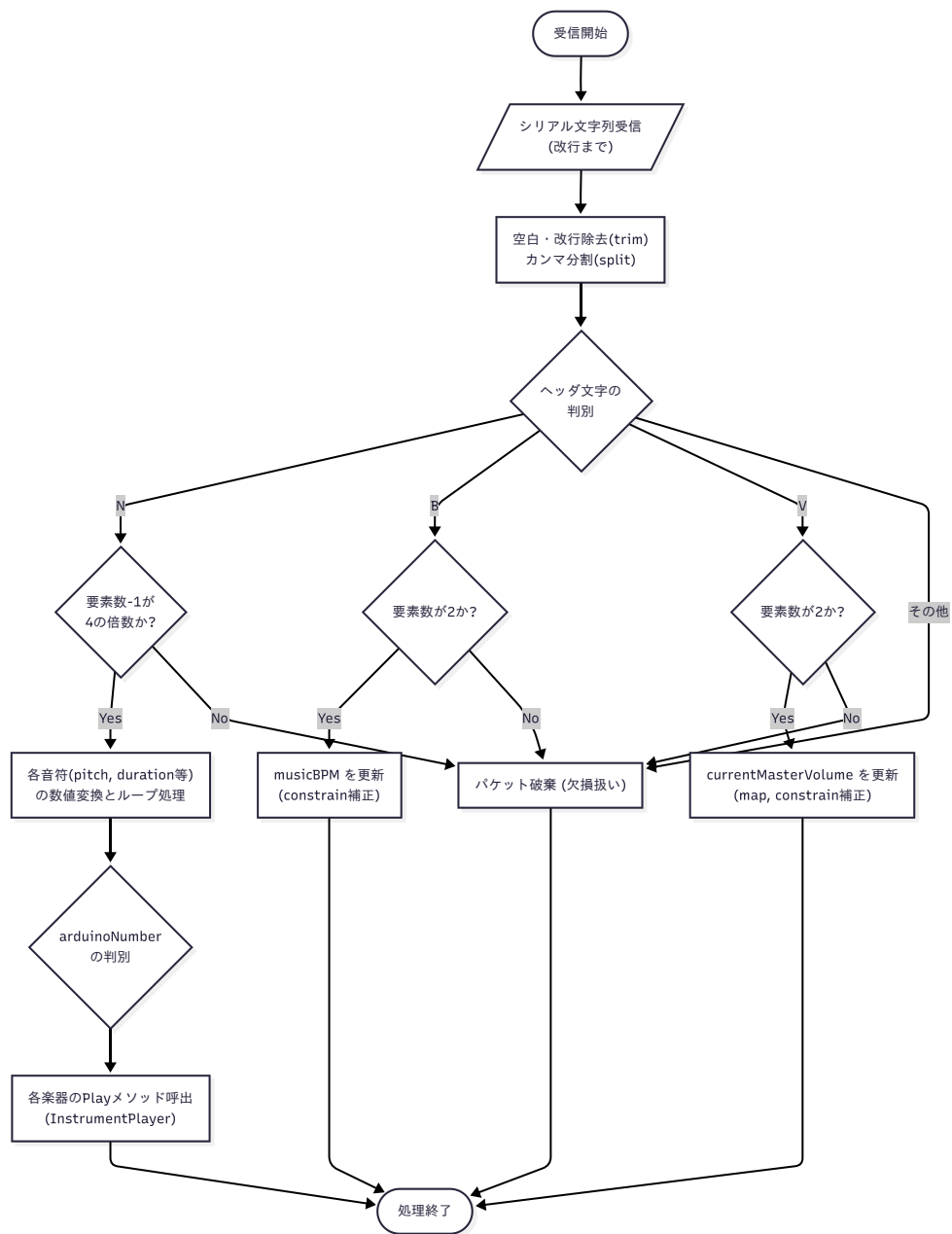


図 3.14 SystemController のフローチャート (serialEvent)

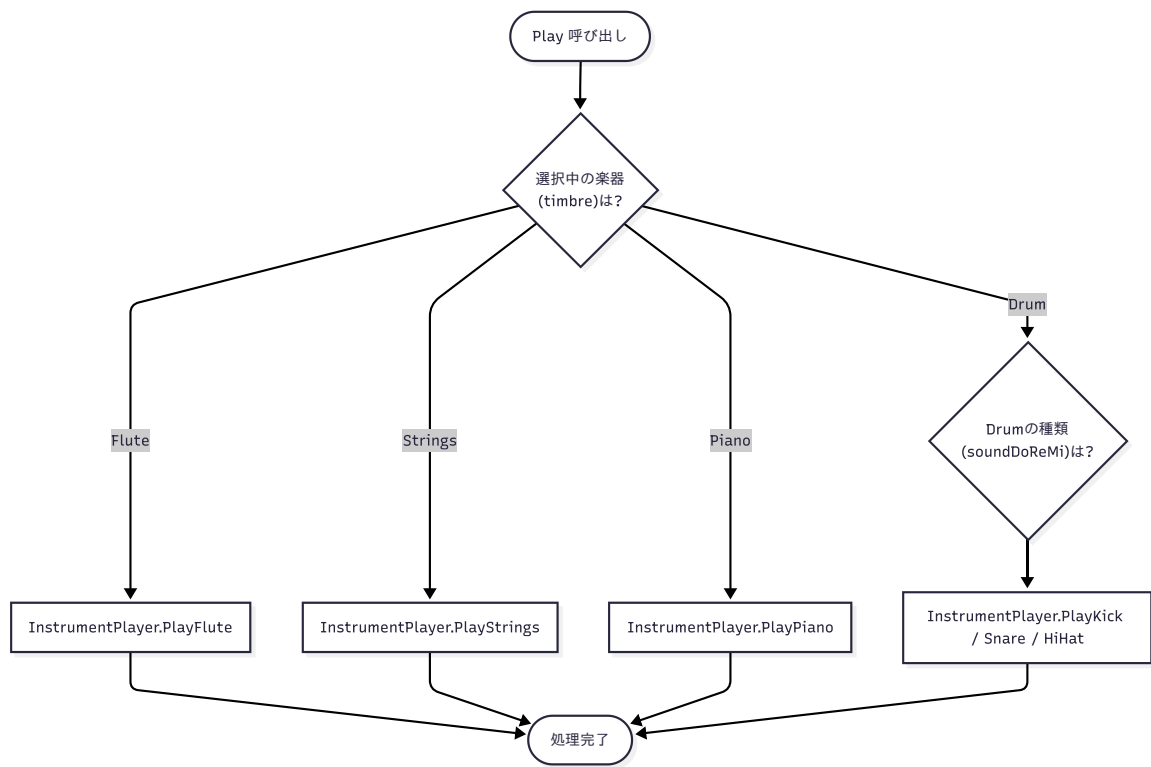


図 3.15 SystemController のフローチャート (Play)

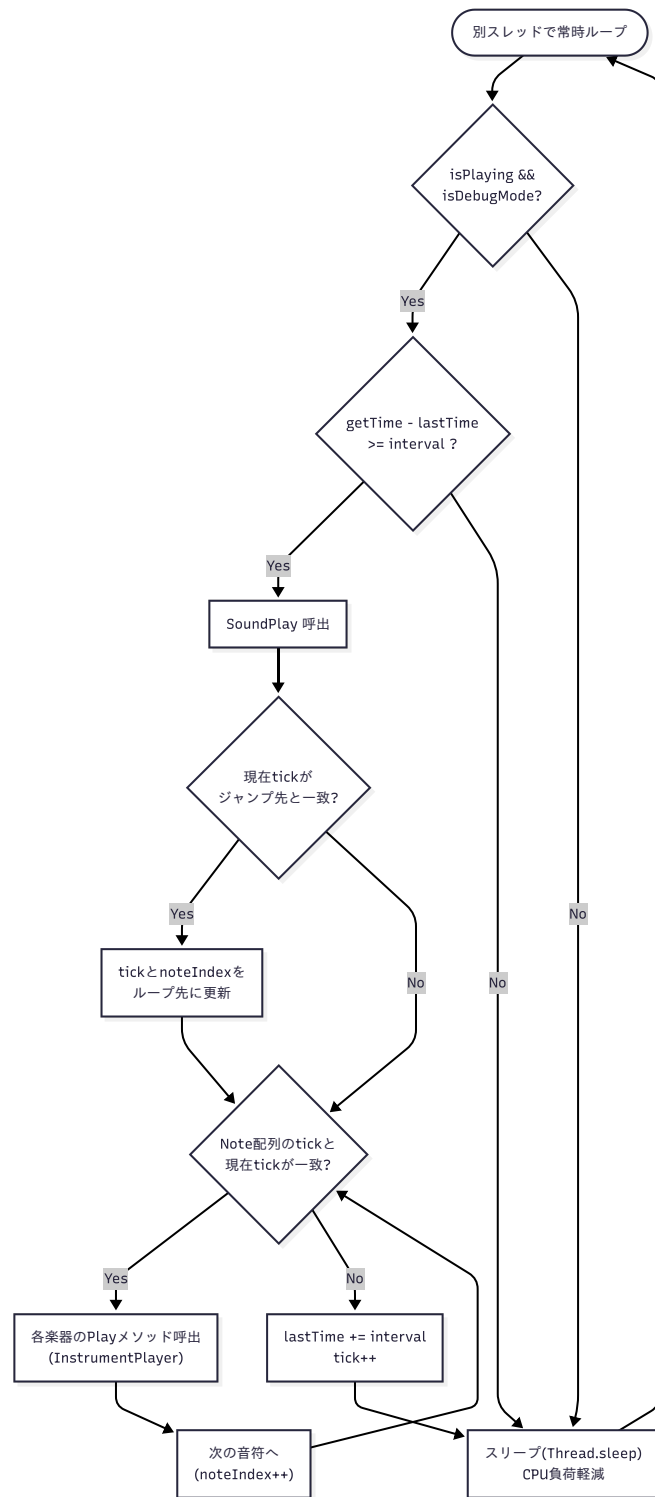


図 3.16 SystemController のフローチャート (SoundPlay)

システムの根幹となるメインクラスであり、Arduino からのシリアル通信の受信、Minim ライブラリを用いた音響処理の初期化、エフェクトバスの構築、および FFT（高速フーリエ変換）による波形解析の管理を行う。本クラスで定義されている全属性（変数）を表 3.9 に、全メソッド（関数）を表 3.10 に示す。

表 3.9 SystemController の属性 (変数)

変数名	型	説明
myPort	Serial	Arduino とのシリアル通信を管理するオブジェクト.
minim	Minim	音響合成ライブラリ Minim のメインインスタンス.
out	AudioOutput	Minim の最終的な音声出力先.
fft	FFT	出力波形を周波数スペクトルに変換する解析オブジェクト.
musicBPM	int	現在の楽曲テンポ. Tick から秒数への変換基準となる.
timbre	String	デバッグモード時における発音テスト用の楽器名.
soundOctave, soundDoReMi, soundStartVelocity, soundEndVelocity	byte	デバッグモード時における手動発音用のパラメータ群.
currentMasterVolume	float	現在の全体音量 (0.5 ~ 1.0). 最終出力時の振幅に乗算される.
arduinoNumber	byte	接続する Arduino の番号 (1 ~ 4). 演奏する楽器の識別に用いる.
isDebugMode	boolean	単体テスト用フラグ. true 時に PC キーボード入力を受け付ける.
longOut, shortOut	Summer	エフェクト (反響音) を経由させるためのミキサーオブジェクト.
player	InstrumentPlayer	発音命令を各楽器クラスへ委譲するための仲介インスタンス.
utils	DataUtils	波形生成用の LUT データを保持・提供するユーティリティ.

表 3.10 SystemController のメソッド

メソッド名	戻り値	引数	説明
setup	void	なし	Minim, エフェクト, シリアル通信, 自動演奏スレッドの初期化を行う.
draw	void	なし	オシロスコープおよびスペクトラムアナライザをリアルタイム描画する.
FreqToX	float	float f	周波数を対数スケールに変換し, アナライザの X 座標を算出する.
serialEvent	void	Serial p	Arduino からのパケットを受信・分割し, 発音やパラメータ変更処理へ分岐する.
Play	void	なし	デバッグモード時に, 選択中の楽器・音程のテスト発音を実行する.
keyPressed	void	なし	デバッグ時の PC キーボード入力を検知し, パラメータ変更や自動演奏をトリガーする.

開発および音色調整段階において, Arduino や指揮機が未接続の状態でも PC 単体で検証を行えるよう, NoteSampler による自動演奏機能を実装する. 本機能に関する全属性 (変数) を表 3.11, 全関数 (メソッド) を表 3.12 示す.

表 3.11 NoteSampler の属性

変数	型	説明
tick	volatile int	演奏中の現在の時間 (Tick).
interval	volatile long	1Tick あたりに必要な待機時間 (ナノ秒).
lastTime	volatile long	前回発音をチェックしたシステム時間.
returnTick	volatile int [] []	繰り返し演奏用のトリガー時間とジャンプ先を管理する配列.
returnIndex, noteIndex	volatile int	配列の読み込み現在位置.
isPlaying	volatile boolean	自動演奏の実行状態フラグ.
audioThread	Thread	描画 (draw) の遅延に影響されない独立したタイマースレッド.
Note	int [] []	JSON から変換された, Tick・音高・ベロシティを含む楽譜データ.

表 3.12 NoteSampler のメソッド

メソッド名	戻り値	引数	説明
GetInterval	long	int playBPM	指定された BPM から 1Tick あたりのナノ秒を算出する.
SoundPlay	void	なし	現在の tick と楽譜データを照合し, 一致した場合に発音命令を発行する.

Processing の標準の描画ループ (draw) に依存すると, 波形描画の負荷によって演奏テンポが揺らいでしまう問題が発生する. そのため, Thread を用いた別プロセスの無限ループ内で時間を監視する設計とし, 各変数には volatile 修飾子を付与している.

3.3.3.2 DataUtils (データ構造とルックアップテーブル)

DataUtils のフローチャートを図 3.17, 3.18, 3.19, 3.20, 3.21 に示す.

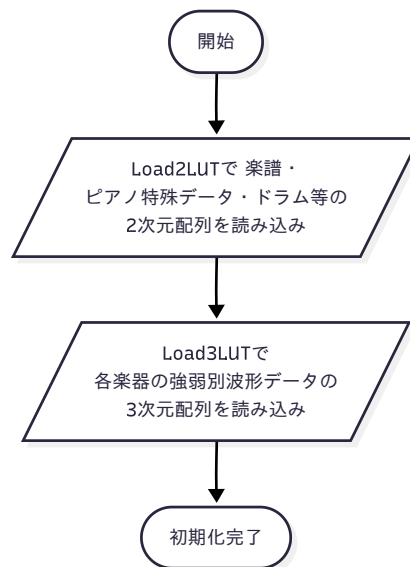


図 3.17 DataUtils のフローチャート (コンストラクタ)

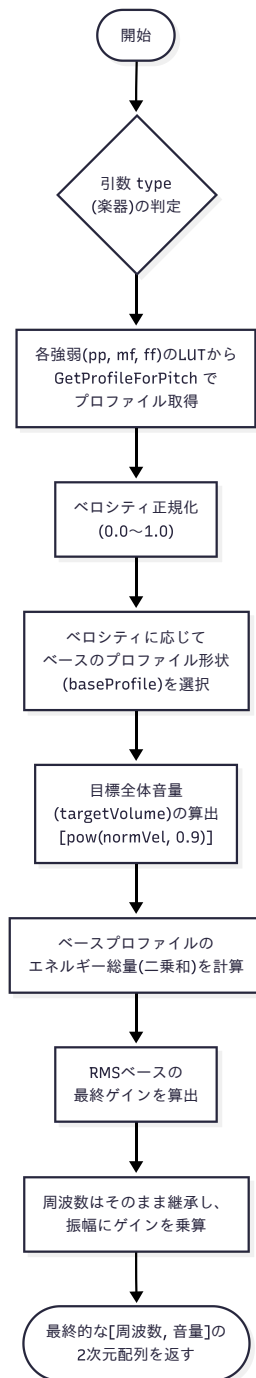


図 3.18 DataUtils のフローチャート (GetDynamicProfile)

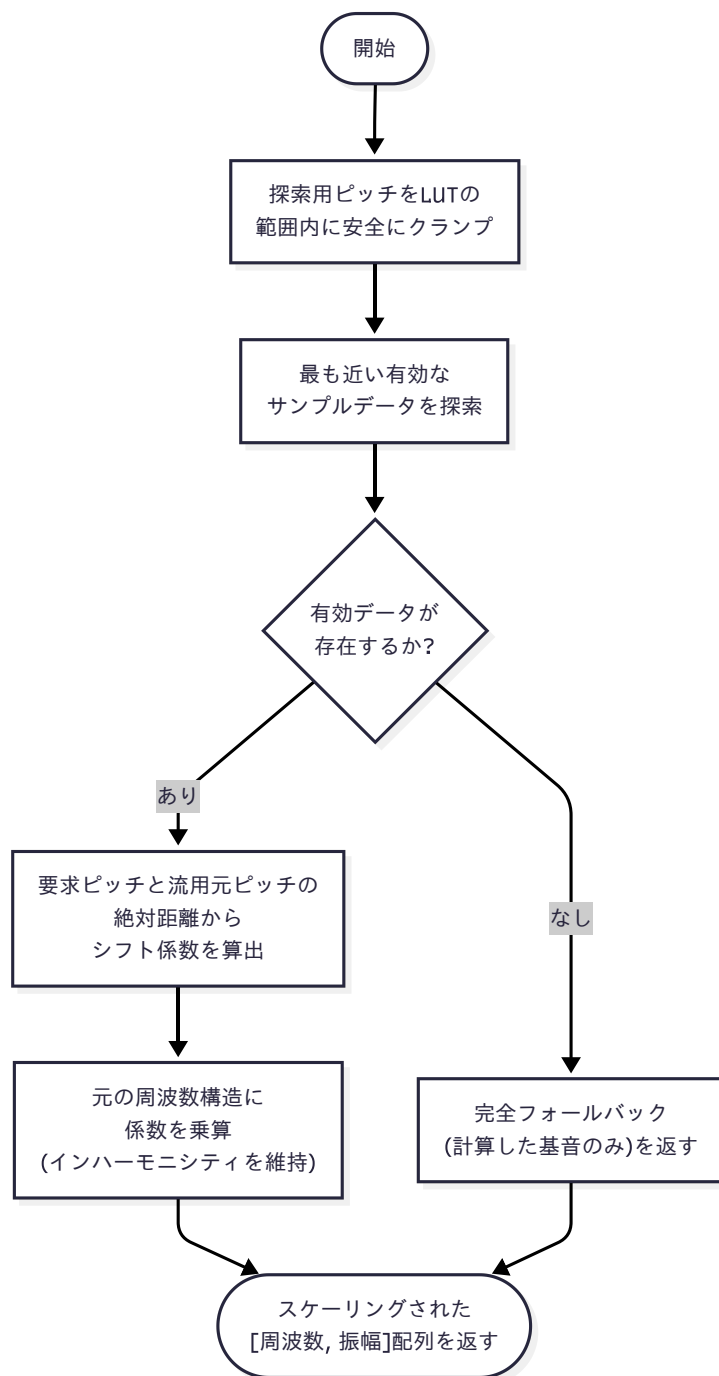


図 3.19 DataUtils のフローチャート (GetProfileForPitch)

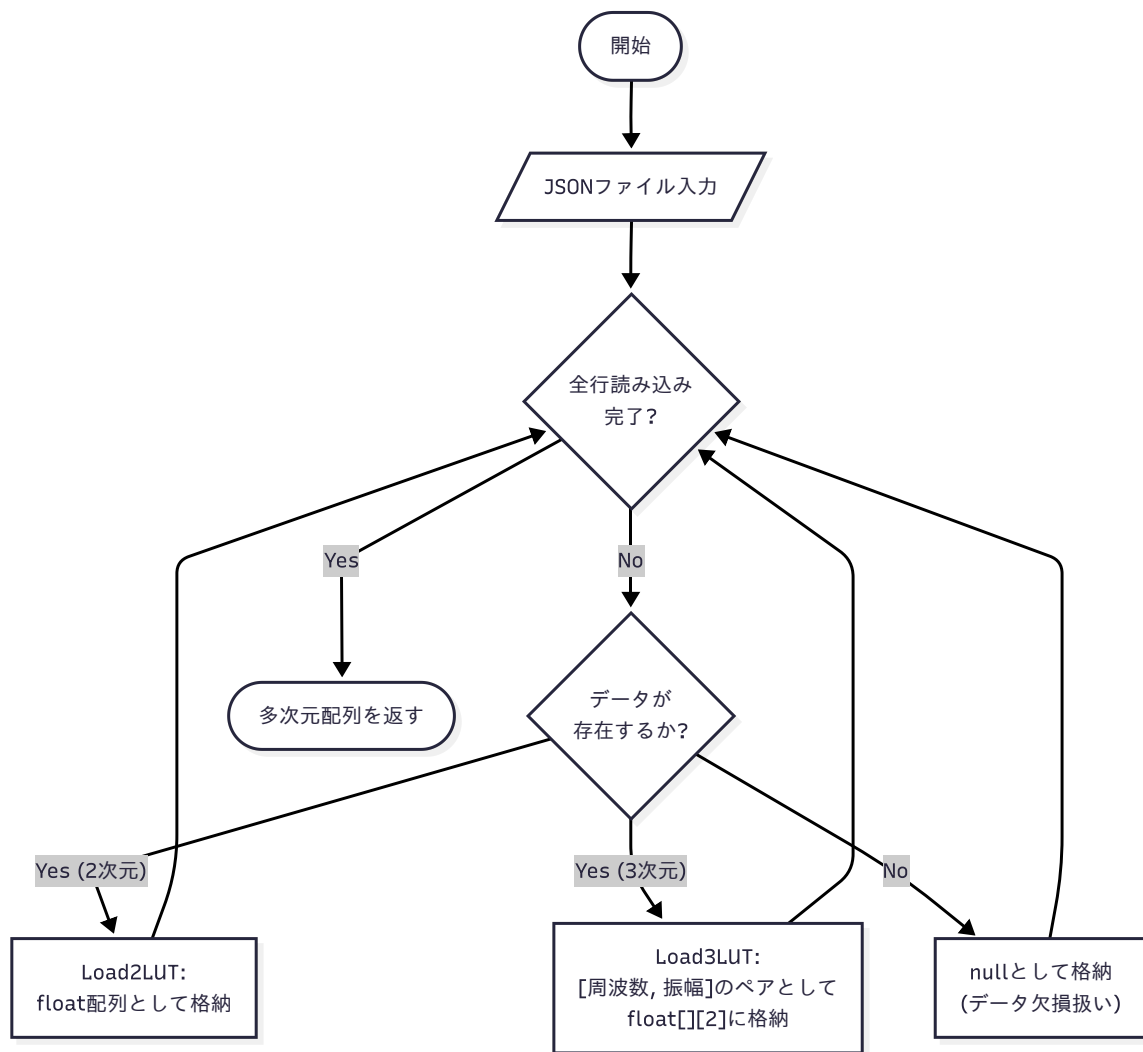


図 3.20 DataUtils のフローチャート (Load3LUT, Load2LUT)

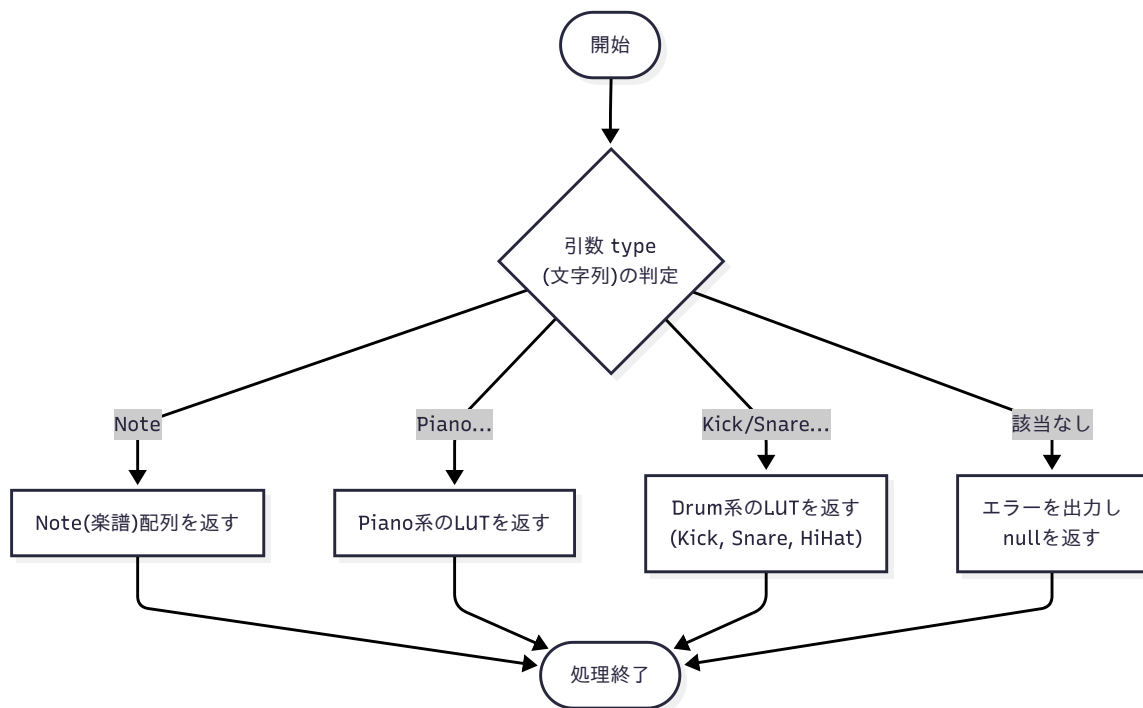


図 3.21 DataUtils のフローチャート (GetLUT)

本システムにおける音色合成の基礎となるのは、Python を用いた実波形解析 (FFT[16]) によって構築されたルックアップテーブル (LUT) である。本クラスの全属性を表 3.13 に、全メソッドを表 3.14 に示す。

表 3.13 DataUtils の属性 (カプセル化された LUT データ群)

変数名	型	説明
note	float [] []	JSON から読み込んだ自動演奏用の楽譜データ配列.
fluteFFLUT, fluteMFLUT, flutePPLUT	float [] [] []	フルートの強弱 (ff / mf / pp) ごとの倍音プロファイル.
violinFFLUT, violinMFLUT, violinPPLUT	float [] [] []	ヴァイオリンの強弱ごとの倍音プロファイル.
violaFFLUT, violaMFLUT, violaPPLUT	float [] [] []	ビオラの強弱ごとの倍音プロファイル.
celloFFLUT, celloMFLUT, celloPPLUT	float [] [] []	チェロの強弱ごとの倍音プロファイル.
bassFFLUT, bassMFLUT, bassPPLUT	float [] [] []	コントラバスの強弱ごとの倍音プロファイル.
pianoFFLUT, pianoMFLUT, pianoPPLUT	float [] [] []	ピアノの強弱ごとの倍音プロファイル.
pianoDecayFFLUT 等 (Decay, WobbleRate, WobbleDepth の各強弱)	float [] []	ピアノ専用の減衰時間, うなり速度, うなり深度を格納した 2 次元配列群.
kickLUT, snareLUT, hihatLUT	float [] []	各ドラムの周波数・強度・減衰時間を格納した 2 次元配列群.

表 3.14 DataUtils のメソッド

メソッド名	戻り値	引数	説明
DataUtils	コンストラクタ	なし	data ディレクトリから全 JSON ファイルを読み込む.
Load2LUT	float[] []	String fileName	ピアノの減衰やドラム等, 要素数が固定の 2 次元配列を展開する.
Load3LUT	float[] [] []	String fileName	ピーク数が音階によって変動する 3 次元配列を展開する.
GetProfileForPitch	float[] []	float pitch, float[] [] lut, int baseMidiNote	ピッチに最も近い実測データを抽出し, 正確なピッチヘシフト演算を施して返す.
GetDynamicProfile	float[] []	float pitch, float velocity, String type, 等	ピッチとベロシティの 2 軸から線形補間を行い, 最終的な周波数・振幅のペアを生成する.
GetLUT	float[] []	String type	文字列指定によりドラムやピアノの各パラメータ配列を安全に返す.

本設計では, 管弦楽器やピアノについて, 演奏の強弱 (ff / mf / pp) ごとに音高・周波数・振幅の三次元可変長配列 (float[] [] []) を保持する. 要求されたピッチがデータと完全に一致しない場合, GetProfileForPitch メソッドにおいて最も近い実測のプロファイルを取得し, 流用元の基準 MIDI ノートと要求ピッチとの半音単位の絶対距離から, 式 (3.1) に基づくピッチシフト係数を算出する.

$$\text{ピッチシフト係数} = 2^{(\text{originalPitch} - \text{sourceMidi})/12} \quad (3.1)$$

この係数をプロファイル内の全周波数成分に乗算することで, 実波形が本来持つ波

形構成を維持したまま、全音域への自然なスケーリングを可能にする。発音時は、GetDynamicProfile メソッドがベロシティを基に強弱間の線形補間を行い、最終的な波形構成レシピを生成する。

また、ドラムやピアノの各パラメータについても、専用の LUT を二次元配列として定義し、波形合成の基礎データとして利用する。外部クラスからこれらのデータへアクセスする際は、引数の文字列によって条件分岐し、目的の配列を返す GetLUT メソッドを実装している。

3.3.3.3 InstrumentPlayer (発音命令の委譲とスケジューリング)

InstrumentPlayer のフローチャートを図 3.22 に示す。

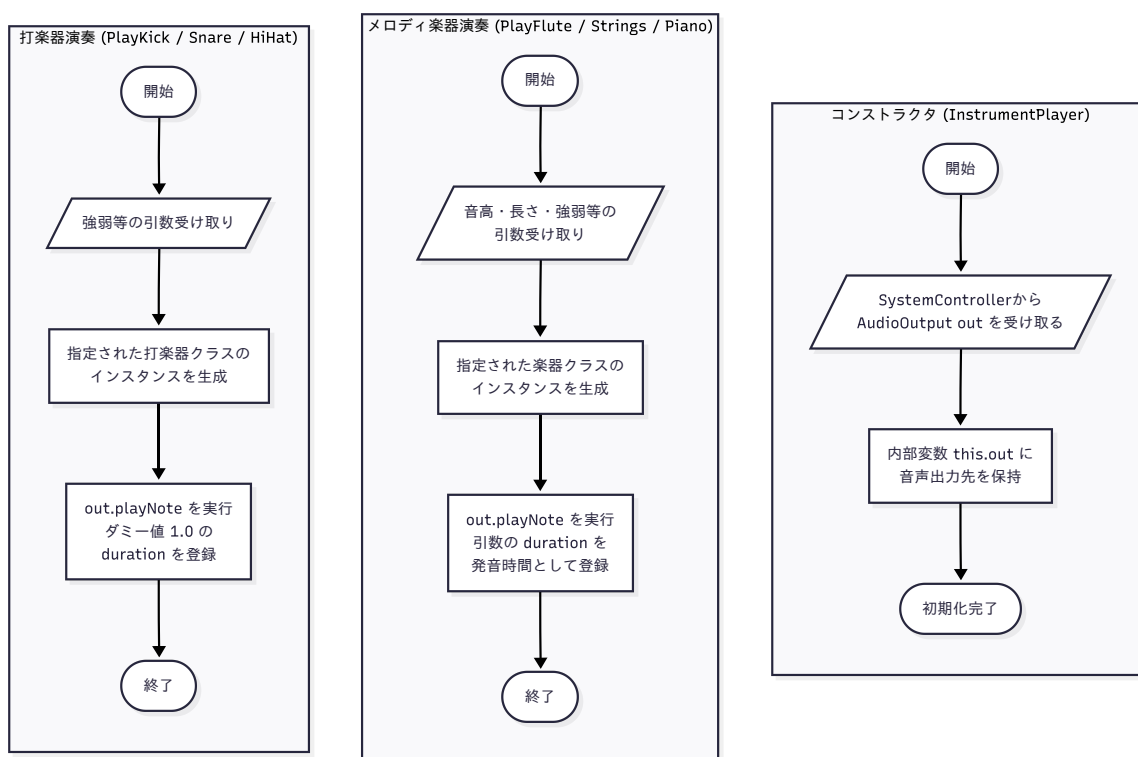


図 3.22 InstrumentPlayer のフローチャート

SystemController と各楽器の波形生成クラス (Instrument 系クラス) を仲介し、発音のスケジューリングを担う。本クラスの全属性および全メソッドを表 3.15、表 3.16 に示す。

表 3.15 InstrumentPlayer の属性

変数名	型	説明
out	AudioOutput	SystemController から受け取った Minim の音声出力オブジェクト.

表 3.16 InstrumentPlayer のメソッド

メソッド名	戻り値	引数	説明
InstrumentPlayer	コンストラクタ	AudioOutput out	メイン出力オブジェクトを保持する.
PlayFlute, PlayStrings, PlayPiano	void	float midiNote, float duration, float startVel, (float endVel),	各楽器のインスタンスを生成し, out.playNote を通じて指定長での発音をスケジュールする.
PlayKick, PlaySnare, PlayHiHat	void	float masterVol, float velocity	ドラム群のインスタンスを生成する. 長さは内部エンベロープに完全に依存するため, ダミー値を用いて発音をスケジュールする.

全体の出力を管理する SystemController から出力オブジェクト (AudioOutput out) を受け取り, 内部で保持する.

キーボード入力やシリアル通信から得られたパラメータ (音高, 長さ, ベロシティなど) を受け取ると, 本クラスに定義された PlayFlute や PlayPiano といった専用メソッドが呼び出される. メソッド内部では, 指定されたパラメータを持つ楽器クラスのインスタンスを動的に生成し, out.playNote を通じて Minim の内部スケジューラに登録・発音させる設計とする.

特筆すべき仕様として, 管弦楽器やピアノが受信した音符の長さ (duration) を直接 playNote の発音時間として渡すのに対し, キックやスネア等の打楽器群 (PlayKick 等) については, 発音の長さが楽器本来の物理的減衰に完全に依存する. そのため, 打楽器の呼び出しにおいては第 2 引数の duration にシステム上のダミー値を与え,

実際の消音タイミング等の制御は各楽器クラス内部のエンベロープ（ADSR）へ完全に一任することで、不要な処理の競合を防ぐ。

3.3.3.4 AddReverb（空間エフェクト）

AddReverb のフローチャートを図 3.23 に示す。

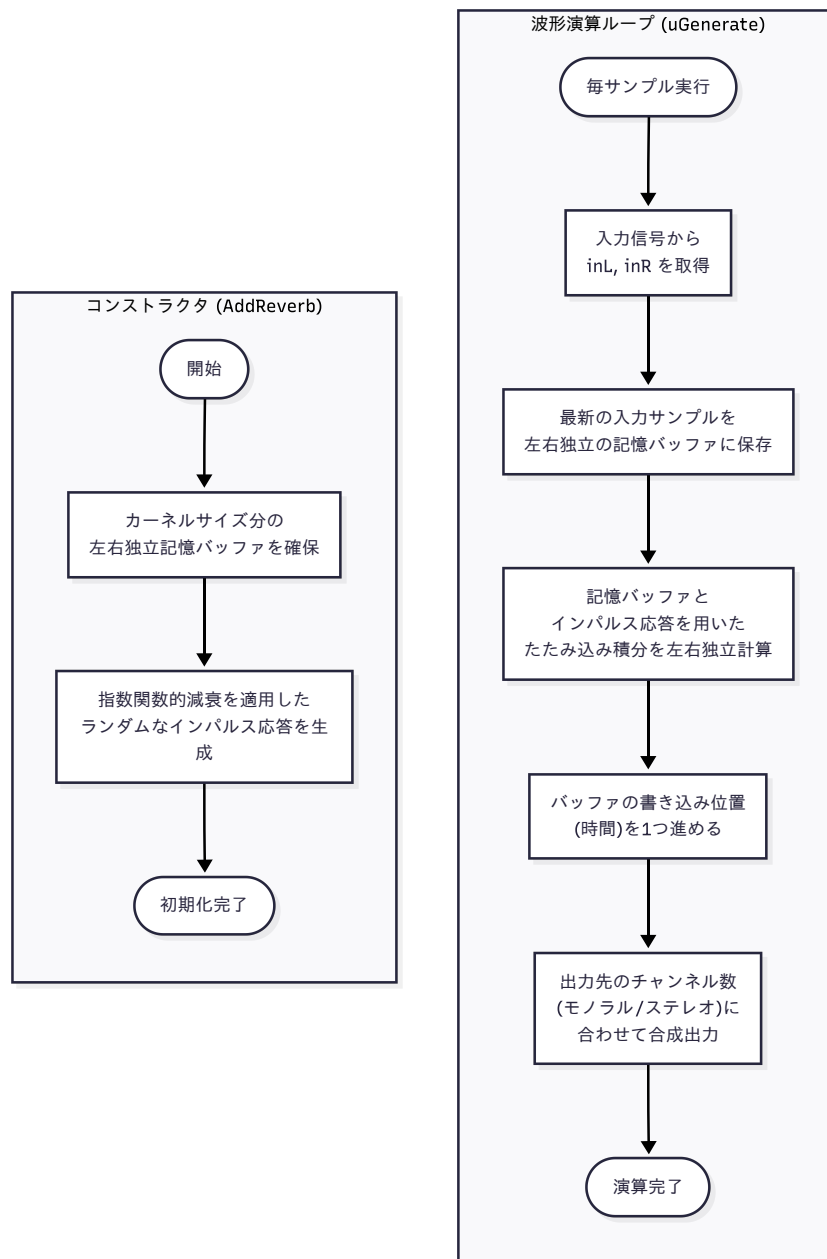


図 3.23 AddReverb のフローチャート

本システムでは、複数人がホールで奏するようなリアルな余韻を再現するため、空間的な残響を付加するエフェクトクラス `AddReverb` を実装する。本クラスは `Minim` の `UGen` を継承して設計されており、その全属性を表 3.17、全メソッドを表 3.18 に示す。

表 3.17 AddReverb の属性

変数	型	説明
audio	UGenInput	前段のモジュールから入力される音声信号の受け皿.
impulseResponse	float []	減衰特性を計算済みのインパルス応答配列.
historyBufferL, historyBufferR	float []	ステレオ処理を行うため, 左右独立して過去の波形を記憶するリングバッファ.
bufferIndex	int	リングバッファの現在の書き込み位置.
kernelSize	int	記憶する過去のサンプル数 (反響の長さ).

表 3.18 AddReverb のメソッド

メソッド名	戻り値	引数	説明
AddReverb	コンストラクタ	int size, float level	指数関数的減衰を適用したランダムなインパルス応答を生成する.
uGenerate	void	float [] channels	audio からの信号と過去の波形との畳み込み積分を, 毎サンプル左右独立して演算・出力する.

波形合成の出口において, 楽器の信号に対して式 (3.2) で示される畳み込み積分 (Convolution) を実行する.

$$y[n] = \sum_{k=0}^{K-1} x[n-k] \cdot h[k] \quad (3.2)$$

ここで, x は入力信号 (historyBuffer), h はインパルス応答, K はカーネルサイズ (kernelSize) である. AddReverb は特定の楽器クラス内部に実装されるのではなく, SystemController 内において独立したエフェクトバス (longOut, shortOut) として構築される. 各楽器クラスは自身の音を指定されたバスへ流すだけでよく, エフェクト処理と波形生成を完全に分離した設計とする.

3.3.4 各楽器の実装仕様 (Instrument 系クラス)

生成されたパラメータは `InstrumentPlayer` クラスに渡され、指定された楽器のコンストラクタが呼び出される。全楽器クラスは、発音処理を開始する `noteOn` と、消音処理を担う `noteOff` を共通して実装する設計とする。

本システムではデータ駆動型の波形生成プロセスを採用する。具体的には、Python を用いた実波形解析によって抽出された「周波数と振幅のペア」のデータを LUT から直接取得し、オシレーターの周波数および振幅として適用する。

3.3.4.1 フルートの実装 (FluteInstrument)

`FluteInstrument` のフローチャートを図 3.24 に示す。

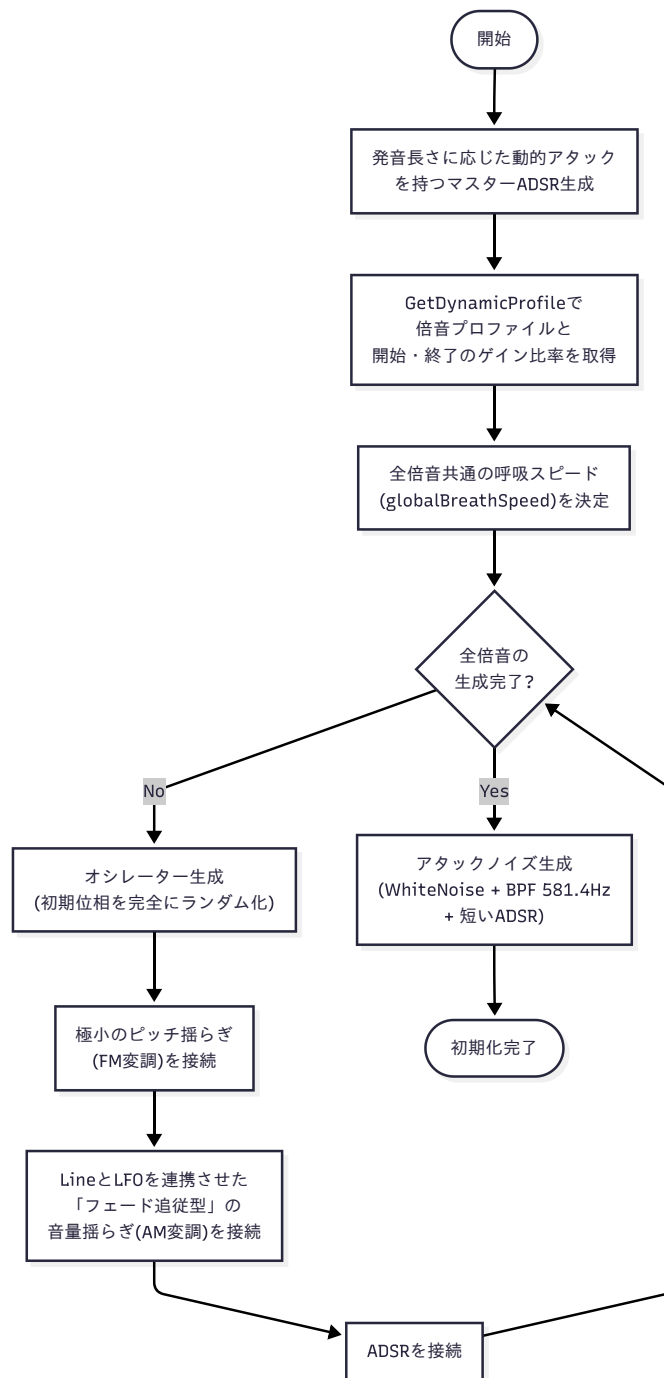


図 3.24 FluteInstrument のフローチャート

フルートクラスにおける全属性を表 3.19, 全メソッドを表 3.20 に示す.

表 3.19 FluteInstrument の属性

変数名	型	説明
numHarmonics	byte	生成する倍音の数を指定する.
masterEnv	ADSR	楽器全体の音の長さを管理するマスターエンベロープ.
harmonicEnvs	ADSR[]	各倍音独立のアタック・ディケイを管理する配列.
wobbleLines, wobbleFactors	Line[],float[]	音量揺らぎ (AM 変調) のフェードラインと, 倍音ごとの固有の揺らぎ比率を保持する配列.
ampLines, startAmps, endAmps	Line[],float[],float[]	倍音ごとのベース音量を管理し, 発音から消音までのフェード処理を行うための配列群.
breathEnv	ADSR	発音直後の破裂音 (アタックノイズ) の長さを管理する.

表 3.20 FluteInstrument のメソッド

メソッド名	戻り値	引数	説明
FluteInstrument	コンストラクタ	float midiNote, float duration, float startVel, float endVel, float masterVol	LUT データ取得, 音量スケールリング演算, ノイズ層生成, LFO 結線などの初期化を行う.
noteOn	void	float dur	各エンベロープおよび Line オブジェクトを一斉にトリガーし, 反響音バスに出力する.
noteOff	void	なし	鍵盤が離された際のリリース処理とバスからのアンパッチを行う.

フルート特有の息の揺らぎを再現するため, ランダムな周期を呼吸スピードとして

定義し、極小のピッチ揺らぎ（FM 変調）を与える。さらに実測データの変動率から導き出された「倍音ごとの自然な振幅揺らぎ（AM 変調）」の深度を配列として保持し、呼吸スピードに同期させて各倍音の音量を揺らすことで、人間らしい有機的な音色を再現する。

管楽器の物理特性を模倣するノイズとして、発音の瞬間にのみホワイトノイズにバンドパスフィルターをかけたアタックノイズを付加し、タンギング時に息がエッジに当たる破裂音の輪郭を表現する。また、短い音符の際にアタックが立ち上がり切る前に消音されてしまう現象を防ぐため、duration（発音長さ）とベロシティから動的にアタックタイムを算出・短縮するロジックを実装する。

3.3.4.2 スtringsの実装 (StringsInstrument)

StringsInstrument のフローチャートを図 3.25 に示す。

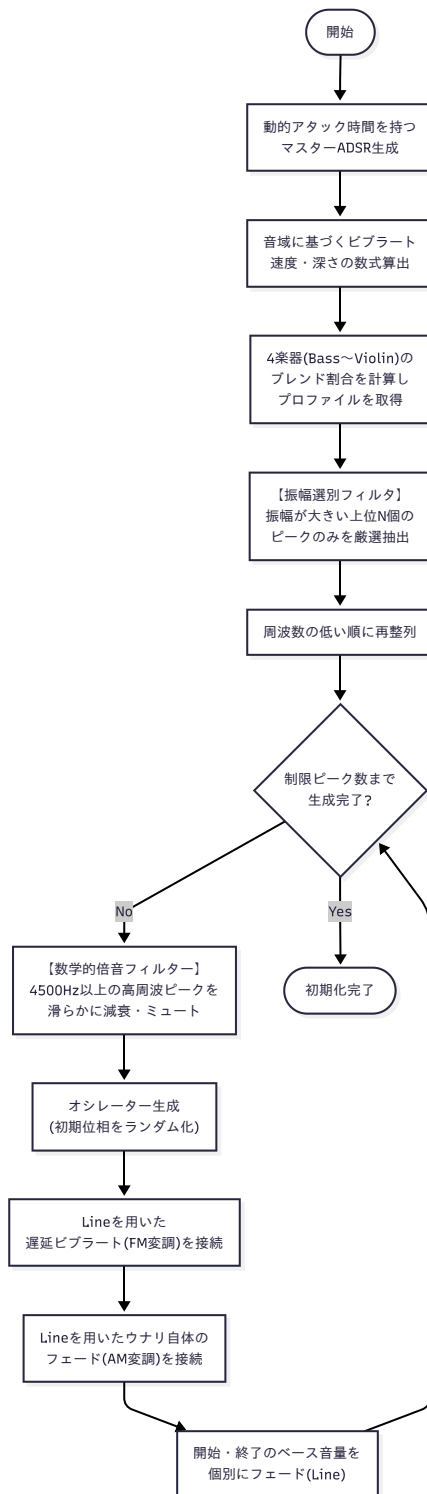


図 3.25 StringsInstrument のフローチャート

ストリングスクラスにおける全属性を表 3.21, 全メソッドを表 3.22 に示す.

表 3.21 StringsInstrument の属性

変数名	型	説明
maxTotalOscils, totalActiveOscils	byte	クロスフェード時に生成を許可するオシレーターの最大数と, 実際の稼働総数.
oscils	Oscil[]	複数楽器の倍音を合成するためのオシレーター配列.
ampLines, startAmps, endAmps	Line[],float[],float[]	倍音ごとのベース音量フェードを管理する配列群.
vibDepthLines, targetVibDepths	Line[],float[]	ビブラート (FM 変調) の深さを遅延させつつ徐々に増加させるための配列群.
wobbleLines	Line[]	音量揺らぎ (AM 変調) の深さ自体をフェードさせるための制御ライン配列.
masterEnv	ADSR	ストリングス全体の音の長さを管理するエンベロープ.

表 3.22 StringsInstrument のメソッド

メソッド名	戻り値	引数	説明
StringsInstrument	コンストラクタ	float midiNote, float duration, float startVel, float endVel, float masterVol	楽器割合の算出, 動的アタック時間の計算, および複数波形のクロスフェード初期化を行う.
createInstrument Oscillators	void	float [] [] profile, float blendRatio, int maxPeaks, float volFactor, float globalVibSpeed, float fmDepthCents, Summer mixer, float startGainFactor, float endGainFactor	各楽器のピークデータを振幅の強い順に抽出し, 数学的倍音フィルターを適用した上で安全にオシレーターを展開する.
noteOn	void	float dur	各エンベロープやビブラートの Line オブジェクトをトリガーし, 反響音バスに出力する.
noteOff	void	なし	消音処理を行い, エフェクトバスからアンパッチする.

ストリングスの広大な音域を自然に網羅するため, 音高 (MIDI ノート番号) に応じて 4 種類の弦楽器 (コントラバス, チェロ, ビオラ, ヴァイオリン) の LUT プロファイルを滑らかにクロスフェードさせる処理を実装する. 各楽器のデータは

`createInstrumentOscillators` メソッドへ渡され、振幅が強い上位成分が抽出されたのち、オシレーター群として動的にメモリ上に展開される。また、4500Hz を超える高次成分に対しては高音域特有の耳障りなピークを抑えるための数学的倍音フィルターを適用する。

さらに、短い音符の際にアタックが立ち上がり切る前に消音されてしまう現象を防ぐため、`duration`（発音長さ）とベロシティから動的にアタックタイムを算出・短縮するロジックを実装する。ビブラートについては、低音域は遅く浅く、高音域は速く深く揺れるよう音高に基づく数式モデルを組み込んでいる。

3.3.4.3 ピアノの実装 (PianoInstrument)

PianoInstrument のフローチャートを図 3.26 に示す。

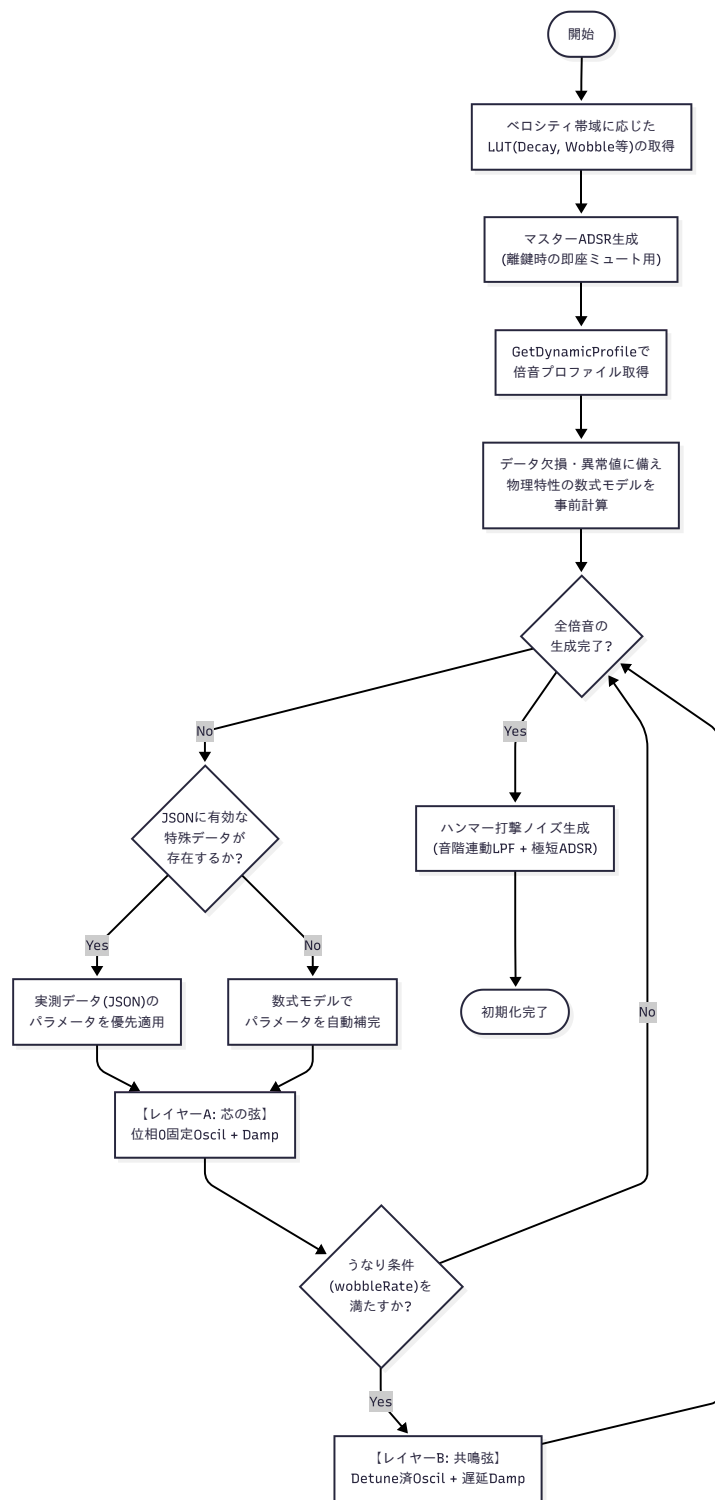


図 3.26 PianoInstrument のフローチャート

ピアノクラスにおける全属性を表 3.23、全メソッドを表 3.24 に示す。

表 3.23 PianoInstrument の属性

変数名	型	説明
numHarmonics	byte	生成する倍音の数を指定する。
masterEnv	ADSR	ダンパーフェルトの役割を果たし、離鍵時に即座にミュートする。
hammerEnv	ADSR	ハンマーが弦を叩く一瞬の打撃ノイズを管理する。
damps	ArrayList<Damp>	指数関数的減衰を表現する Damp オブジェクトのリスト。芯の弦と共鳴弦を動的に格納する。

表 3.24 PianoInstrument のメソッド

メソッド名	戻り値	引数	説明
PianoInstrument	コンストラクタ	float midiNote, float velocity, float masterVol	LUT データの取得、ハイブリッド減衰モデルの適用、デュアルストリング展開、およびハンマーノイズの構築を行う。
noteOn	void	float dur	準備された全 Damp オブジェクトをアクティベートし、反響音バスに出力する。
noteOff	void	なし	マスターエンベロープを用いてダンパーペダルが降りる挙動（即座のミュート）を再現する。

ピアノには最大 80 倍音の加算合成を採用し、打弦楽器としてのハンマーの物理挙動と弦の共鳴現象を再現する設計とする。管楽器等とは異なり発音後に自ら音量をコントロールすることは不可能であるため、音色の決定や音量制御には開始時の velocity のみを使用し、endVelocity は引数としない仕様とする。

減衰の表現においては、直線的な ADSR ではなく自然界のエネルギー減衰を再現する Damp クラス（指数関数的減衰）を用いる。各倍音の「減衰時間」「うなり速度」「うなり深度」は、専用の LUT データから実測値を優先して適用し、データ欠損時や異常値の場合は、Python の解析から導き出した物理特性の傾向に基づく数式モデルで自動補完する「ハイブリッド方式」を実装している。さらに、芯となる弦（Dry 弦）に加え、わずかに周波数をずらした共鳴弦を並列で合成するデュアルストリング構造により、ピアノ特有の複雑なうなりとリバーブ成分を生成する。

発音時のアタック成分として、打鍵の強さと音高に連動してカットオフ周波数が変化するローパスフィルター付きのホワイトノイズを付加し、ハンマーが弦を叩く瞬間の鋭い衝撃ノイズ（パーカッシブ成分）を再現する。離鍵時（noteOff）には、全体の出力を統括するマスターエンベロープが短いリリース時間で即座にミュート処理を行い、ダンパーが弦を押さえる挙動を再現する。

3.3.4.4 ドラムの実装 (Kick, Snare, HiHat)

ドラムパートは各楽器の物理特性に合わせ、LUT データを用いた加算合成と、複数のフィルターによる減算合成ノイズを組み合わせた構成とする。消音タイミング等の制御は各楽器内部のエンベロープへ完全に一任するため、発音命令時の `duration` にはシステム上のダミー値が与えられる。ドラムのフローチャートを図 3.27 に示す。

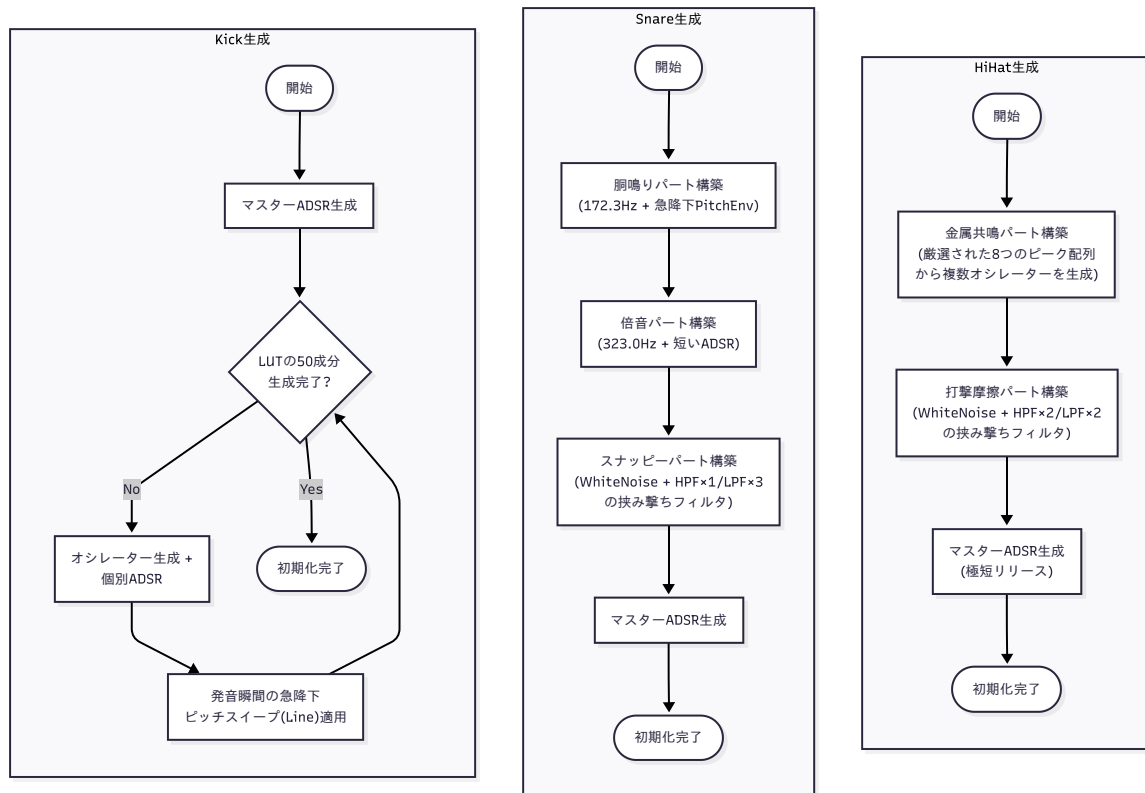


図 3.27 ドラムのフローチャート

キック (KickInstrument) キッククラスの全属性を表 3.25 に、全メソッドを表 3.26 に示す.

表 3.25 KickInstrument の属性

変数名	型	説明
masterEnv	ADSR	キック全体の余韻を管理するエンベロープ.
harmonicEnvs	ADSR[]	LUT から取得した各周波数成分の独立した減衰時間を管理する配列.

表 3.26 KickInstrument のメソッド

メソッド名	戻り値	引数	説明
KickInstrument	コンストラクタ	float masterVol, float velocity	50 成分のオシレーターを生成し、発音瞬間のピッチスweepを結線する.
noteOn	void	float duration	全エンベロープをトリガーし、エフェクトを経由しない出力バスへ接続する.
noteOff	void	なし	消音処理およびアンパッチを行う.

キックは、50 成分に及ぶ LUT 成分の加算合成によって構築される。発音の瞬間にのみ基音の周波数を 1.5 倍から元の周波数へ急激に降下させるピッチスweep (Line) を各オシレーターに適用する。

スネア (SnareInstrument) スネアクラスの全属性を表 3.27 に、全メソッドを表 3.28 に示す。

表 3.27 SnareInstrument の属性

変数名	型	説明
masterEnv	ADSR	スネア全体の出力を管理するエンベロープ.
bodyOsc, bodyPitchEnv, bodyAmpEnv, overtoneOsc, overtoneAmpEnv	Oscil, Line, ADSR, Oscil, ADSR	生成する波形のオシレーター, ピッチ急降下用ライン, および音量エンベロープ.
snappyNoise, snappyAmpEnv	Noise, ADSR	ホワイトノイズとそのエンベロープ.

表 3.28 SnareInstrument のメソッド

メソッド名	戻り値	引数	説明
SnareInstrument	コンストラクタ	float masterVol, float velocity	各波形とノイズを独立して構築し、ミキサーへ結線する.
noteOn	void	float duration	全エンベロープをトリガーし、エフェクトを経由しない出力バスへ接続する.
noteOff	void	なし	消音処理およびアンパッチを行う.

スネアは、特定の波形とホワイトノイズで合成される。ノイズには、複数のフィルターを用いて特定の音域のみを通過させる。

ハイハット (HiHatInstrument) ハイハットクラスの全属性を表 3.29 に、全メソッドを表 3.30 に示す。

表 3.29 HiHatInstrument の属性

変数名	型	説明
masterEnv	ADSR	極めて短い余韻を管理するハイハット全体のマスターエンベロープ.
toneOscList, toneEnvList	Oscil[],ADSR[]	金属共鳴の主要なピーク成分を個別に発振し制御するための配列群.
hatNoise, noiseEnv	Noise,ADSR	打撃摩擦を表現するノイズとそのエンベロープ.

表 3.30 HiHatInstrument のメソッド

メソッド名	戻り値	引数	説明
HiHatInstrument	コンストラクタ	float masterVol, float velocity	金属コアとなる複数パルス波の合成と、高帯域ハイパスフィルターを用いたノイズを構築する。
noteOn	void	float duration	全エンベロープをトリガーし、エフェクトを経由しない出力バスへ接続する。
noteOff	void	なし	消音処理およびアンパッチを行う。

ハイハットは、実測データから抽出された主要な成分を加算合成し、同時にノイズの減算合成を組み合わせている。ノイズ成分に対しては、ノイズには、複数のフィルターを用いて特定の音域のみを通過させる。

3.3.4.5 パラメータ変換仕様とシステム上の制約

Arduino から受信したデータを、Processing 内で音響処理を行うための物理量へと変換する共通ロジックについて述べる。1 小節を 96 Tick とした場合、四分音符は 24 Tick となる。受信した長さ (duration) は、現在の musicBPM を用いて式 (3.3) により実際の発音時間 (秒) へと変換される。

$$\text{発音時間(sec)} = \text{duration} \times \frac{60}{\text{musicBPM} \times 24} \quad (3.3)$$

また、受信した 0 ~ 127 の強さ (startVelocity および endVelocity) は、0.0 から 1.0 の振幅値へと変換され、前述の DataUtils を介した音色決定の演算基準として機能する。最終出力波形に対しては、指揮機からの全体音量指示 (マスターボリューム) が適用される。

各楽器クラスのインスタンス生成時において、引数として渡された発音時間 (秒) が Minim 内にスケジュールされる。そのため、長い音符の発音途中で BPM 変更指示を受信した場合、既に発音中の音符の終了タイミングは「変更前の BPM 基準」で維持されるというシステム上の制約が存在する。実運用上、BPM 変化後における「次の音符」の発音タイミングは Arduino の最新 Tick によって正確に制御されるため、

楽曲全体の同期は破綻しない。以上の理由から、本システムでは動作の安定性を優先し、発音開始時に一括で秒数変換を行う方式を基本仕様とする。

3.3.5 デバッグおよび単体テスト機能

開発および音色調整フェーズにおいて、Arduino とのシリアル通信が確立していない状態でも PC 単体で検証を行えるよう、SystemController 内部にデバッグモードを実装する。

isDebugMode が true の場合、keyPressed メソッドを介して PC のキーボード入力を検知し、各種検証機能を実行する。キーボードの文字キー（a～j 等）を各音階に、数字キー（1～7）をオクターブ指定に割り当て、Z, C, V, B キーによって対象楽器（フルート、ストリングス、ドラム、ピアノ）を動的に切り替えながら、手動での Play メソッド呼び出しによる単音テストを可能としている。さらに、各種記号キーを用いて発音時および消音時のベロシティ（強弱）をリアルタイムに変更できる設計とし、多様なパラメータにおける音色変化を網羅的に確認できる。

また、本システムでは複数音符の連続的な変化やアンサンブル時の処理負荷を検証するため、内蔵シーケンサである NoteSampler を用いた自動演奏機能を搭載している。キーボードの特定のキー（A, S, D, F）を押下することで、メモリ上に保持された楽譜データを用いた楽曲（「かえるのうた」「トルコ行進曲」等）の自動再生が開始される。この再生処理は、画面描画ループ（draw）の負荷によるテンポの揺らぎを完全に排除するため、独立したスレッド（Thread）上で時間を管理し、指定された BPM に対して正確なタイミングで InstrumentPlayer へ発音命令を委譲する。

画面表示においては、draw メソッド内にて Minim の出力波形（オシロスコープ）および FFT オブジェクトを用いた周波数スペクトラム（スペクトラムアナライザ）をリアルタイム描画する。特にスペクトラムアナライザでは、周波数軸（X 軸）を対数スケール（FreqToX メソッドによる演算）へと変換して描画し、合成された波形構造やエフェクトによる残響の推移などを視覚的かつ詳細に解析できる開発環境を構築している。

3.4 通信設計

3.4.1 通信の概要

本設計では、Wifi を用いた UDP による通信と、レーザーを用いた物理同期を行っている。通信の概要を図 3.28 に示す。通信の手順としては、まず指揮デバイスで BPM または演奏状態の操作が行われた場合は、観覧車デバイスに UDP を用いて情報を送信する。また、音量が操作された場合は、UDP を用いて楽器に情報を送信する。

次に、観覧車デバイスは指揮デバイスから送信された BPM を基にモータを回転させる。その回転周期を楽器デバイスの CdS セルで読み取り、BPM を算出する。

最後に指揮デバイスから各楽器に送る開始信号を送信し、それを受け取った楽器が演奏を開始する。1 台のドラム機（マスター）および複数台の他楽器機を WiFi および観覧車（レーザー）を介して連携させ、BPM および再生位置を同期して輪唱を行う。

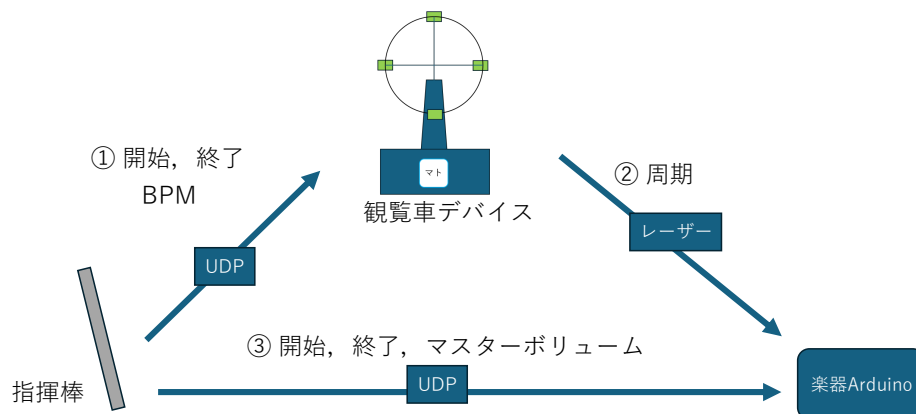


図 3.28 本システムの通信経路の概要。

3.4.2 ファイル構成

各役割ごとに、ファイルを表 3.31 に示すように定義する。FerrisWheel.ino, MotorControl.cpp, MotorControl.h は観覧車の動作に関する処理を記述したファイルであり、Controller.ino, SyncManager.cpp, SyncManager.h は指揮デバイスとの通信に関する処理を記述したファイルである。リファクタリングに伴い、センサー処理

をカプセル化した PotManager および AccManager を独立したモジュールとして追加した。

表 3.31 ファイル構成

ファイル名	内容
FerrisWheel.ino	観覧車の動作を担当する。UDP パケットの受信、モータ制御の呼び出しなどを行う。
MotorControl.cpp	モーターの制御を行う。ルックアップテーブルに基づく回転速度の切り替えや、起動時の加速を担当する。
MotorControl.h	MotorControl.cpp のヘッダーファイルを定義する。
Controller.ino	指揮デバイスのメインロジックおよび全体の司令塔を担当する。専門クラスへセンサー処理を委譲し、物理ボタンによる演奏状態判定と各関数の呼び出しを行う。
SyncManager.cpp	指揮デバイスからの通信を行う。Wi-Fi の接続処理および各デバイスへの UDP パケットの送信処理を担当する。
SyncManager.h	SyncManager.cpp のヘッダーファイルを定義する。
PotManager.cpp	可変抵抗器のアナログサンプリング、移動平均処理による平滑化、および BPM・音量の 5 段階範囲分割ロジックを担当する。
PotManager.h	PotManager.cpp のヘッダーファイルを定義する。
AccManager.cpp	3 軸加速度センサー（GY-291）の I2C 通信による制御、合成加速度の算出、および多重検知を防止した指揮棒のシェイク動作判定を担当する。
AccManager.h	AccManager.cpp のヘッダーファイルを定義する。

3.4.3 通信に使用する変数・関数

本設計に使用する定数は表 3.32 の通りである。これらは実装において、プログラム内で const を使って定義する。ただし、PotManager および AccManager クラス内の詳細な定数は 3.4.7 節に記載したため省略する。

表 3.32 設計に利用する定数一覧

型	名前	説明	所属	メモリ量 [byte]
char[]	SSID	Wi-Fi の ネットワーク名 (hackathon001-WPA2)	Main	17
char[]	PASS	その Wi-Fi の パスワード (hackathon001)	Main	12
IPAddress	ferrisWheelGroup	観覧車 (送信) の IP アドレス	Main	4
IPAddress	fluteIP	フルートの IP アドレス	SyncManager	4
IPAddress	pianoIP	ピアノの IP アドレス	SyncManager	4
IPAddress	stringsIP	ストリングスの IP アドレス	SyncManager	4
IPAddress	drumIP	ドラムの IP アドレス	SyncManager	4
IPAddress	instrumentGroup	楽器全体の IP アドレス	SyncManager	4
IPAddress	localIP	観覧車デバイス自身の静的 IP アドレス	Main	4
uint16_t	TARGET_PORT	通信に利用する相手側の待ち受けポート番号	Main	2
uint16_t	LOCAL_PORT	通信に利用する自身の UDP ローカルポート番号	Main	2
uint8_t	PIN_MOTOR_PWM	モータ制御用の出力ピン	MotorControl	1
uint8_t	bpmTable[5]	5 つの離散値 BPM 格納	MotorControl	5
uint8_t	pwmTable[5]	5 つの離散値 PWM 格納	MotorControl	5
float	VCC	Arduino への供給電圧 V_{cc}	MotorControl	4

また、システム内で利用する変数は表 3.33 の通りである。

表 3.33 システム内で利用する変数一覧

型	名前	説明	所属	メモリ量 [byte]
uint8_t	currentBPM	現在演奏中の BPM データ	MotorControl	1
float	RPM	BPM から計算され た目標回転数	MotorControl	4
float	omega	目標角速度 ω	MotorControl	4
uint8_t	pwmValue	analogWrite 用の 出力値	MotorControl	1
float	V_average	平均電圧 $V_{average}$	MotorControl	4
float	rpmTable[5]	rpm を格納する (計 算するため空にし ている)	MotorControl	5
bool	isButtonPlaying	開始・停止信号を 交互に切り替える フラグ	Main	1

次に Arduino のメモリの検証を行う。本設計がターゲットハードウェアのメモリ制約を満たすか検証を行う。使用するマイコンボードは Arduino UNO R4 WiFi である。データシートによれば、本機に搭載されているマイコンは、プログラムの実行中に変数や計算結果を保持する SRAM を 32 KB、プログラム本体を保存する Flash メモリを 256 KB 備えている [14]。表 3.32 および表 3.33 に示した通り、制御・通信用フラグの追加に伴う本システムの主要なメモリ使用量は 96 byte である。これは SRAM 全体の約 0.3% 程度に留まっており、無線通信ライブラリや制御スタックによる動的なメモリ消費を考慮しても、十分な容量を確保できていると判断した。

本設計において、指揮デバイスと観覧車間の演奏制御および同期処理を実現するために実装する関数を、表 3.34, 3.35, 3.36, 3.37, 3.38 に示す。

表 3.34 Main（メインルーチン）で実装する関数の一覧

戻り値	関数名	引数	概要
void	button	なし	物理ボタンの押下を検知し，開始信号・停止信号のトグル送信と演奏状態フラグの反転を一括管理する.

表 3.35 MotorControl クラスで実装する関数の一覧

戻り値	関数名	引数	概要
void	begin	なし	setup 内で一度だけ呼び出される関数であり，モーターピンの初期化を行う.
void	startRamp	uint8_t stepNumber	演奏開始時の加速を制御する. 初期の BPM に基づき一定時間をかけて徐々に目標速度（RPM）へ到達させる.
void	stopRamp	なし	演奏終了時の減速を制御する. 現在の BPM に基づき，時間をかけて徐々に速度を 0 に近づけていく.
void	changeBPM	uint8_t stepNumber	演奏中の BPM 変更を行う.
void	stepToNumber	uint8_t stepNumber	段階番号を BPM に変換する関数.
void	createRpmTable	なし	rpmTable を作成する関数.
void	updateStatus	なし	omega と V_average を作成する関数.

表 3.36 SyncManager クラスで実装する関数の一覧

戻り値	関数名	引数	概要
void	begin	const char ssid[], const char pass[]	setup 内で一度だけ呼び出される関数であり、Wi-Fi と UDP を準備している.
void	sendStart	IPAddress targetIP	演奏開始合図 ('S') を指定のアドレス宛てに送信するための関数.
void	sendBPM	uint8_t stepNumber	BPM 情報をマルチキャスト送信するための関数.
void	sendVolume	uint8_t stepNumber	音量情報をマルチキャスト送信するための関数.
void	sendEnd	IPAddress targetIP	演奏終了合図 ('E') を指定のアドレス宛てに送信する関数.
void	packet3	const uint8_t packet[], uint8_t b, IPAddress targetIP	パケットロス対策として 3 回冗長送信を行うための共通処理.

表 3.37 AccManager クラスで実装する関数の一覧

戻り値	関数名	引数	概要
void	setupADXL345	なし	ADXL345 (加速度センサー) の I2C 初期化および計測開始の準備を行う。
void	update	SyncManager& sync, PotManager& pot	20ms 周期の時間管理と、閾値判定による指揮棒のシェイク動作検知を行う。
void	updateShakedInfo	SyncManager& sync, PotManager& pot	pot の予約フラグを確認し、変化があれば sync を通じて BPM や音量の段階番号を送信する。
float	readAccMagnitude	なし	I2C で読み取った XYZ の 3 軸生データから合成ベクトル (加速度) を算出する内部計算関数。

表 3.38 PotManager クラスで実装する関数の一覧

戻り値	関数名	引数	概要
void	update	なし	20ms 周期の時間管理を行い、可変抵抗器の値のサンプリング処理を駆動する。
uint8_t	getBpmStep	なし	カプセル化された現在の BPM 段階番号 (1～5) を外部に返す。
uint8_t	getVolStep	なし	カプセル化された現在の音量段階番号 (1～5) を外部に返す。
int	calcStep	float val	平滑化された平均アナログ値を、設計書に基づく境界値に照らし合わせて 1～5 の段階番号に変換する。
bool	bpmUpdatePotentiometers	なし	BPM 用可変抵抗器の値を移動平均で処理し、前回値からの変化を検知してフラグ (真偽値) を返す。
bool	volUpdatePotentiometers	なし	音量用可変抵抗器の値を移動平均で処理し、前回値からの変化を検知してフラグ (真偽値) を返す。

3.4.4 クラス図

本設計におけるクラス図を図 3.29 のように定義する。独立した 2 つのハードウェア構成に合わせて、指揮デバイス (Controller.ino) と、観覧車デバイス (FerrisWheel.ino) の 2 つのメインルーチンに分離する構成とした。指揮デバイス側では、Controller.ino が全体の進行を管理し、通信・同期を担う SyncManager クラスに加えて、各センサー処理である PotManager クラスと AccManager クラスを集約 (所有) する構成としている。一方、観覧車デバイス側では、FerrisWheel.ino がパケットの受信管理を行い、モータ制御を担う MotorControl クラスを集約 (所有) する構

成とした。これら 2 つの独立したシステム間は、ネットワークをまたぐ UDP 通信によって相互に連動する。

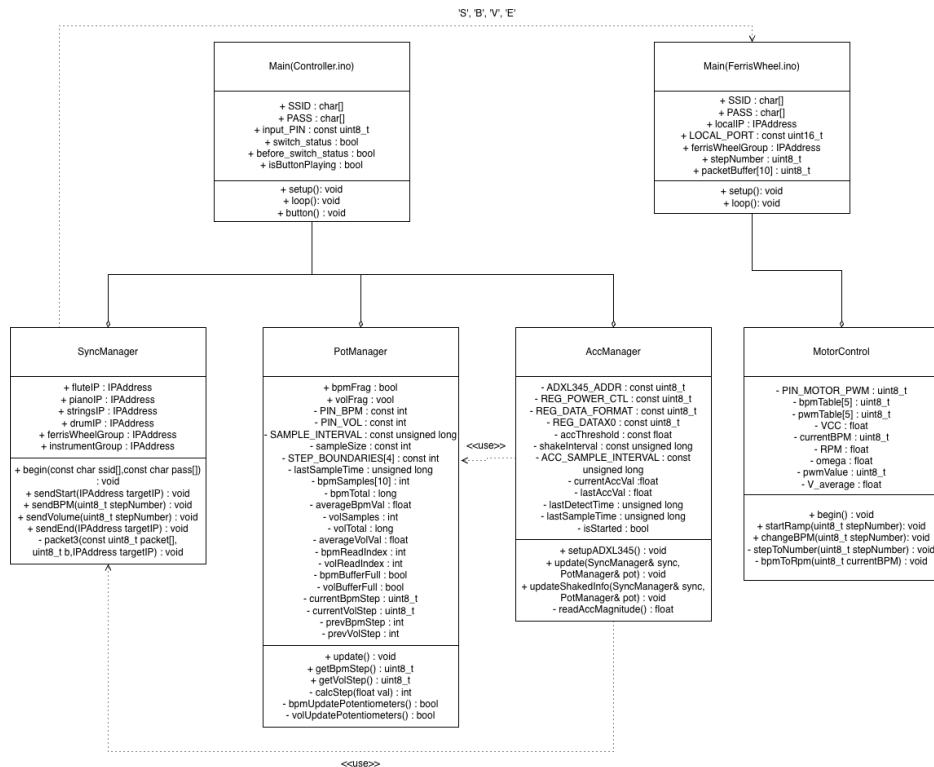


図 3.29 本システムのクラス図。

3.4.5 ボタン入力の判定

指揮デバイスにおいて、演奏の開始および終了信号の送信に使われるボタンの入力判定処理を実装する。

ボタンの誤判定を防ぐため、プルアップ抵抗を用いた回路を構成している。ボタンが押されていない状態では、回路は電源電圧にプルアップされているため、該当するデジタルピンには常に 5V が印加される。

一方、ボタンが押下された状態では、回路が GND に短絡されるため、ピンの電位は LOW へと遷移する。指揮デバイスのプログラムでこのデジタルピンの電圧状態の変化を常時監視・検知することにより、ボタンの押下を判定する。

さらに、チャタリングによる誤判定や冗長パケットによる誤作動を防止するため、押下検知後に一定時間の待機処理を設けるとともに、演奏状態管理フラグ

(isButtonPlaying) を用いたトグル制御を導入している。これにより、同一の物理ボタンのみで演奏の開始と終了を交互に切り替えることが可能となる。ボタン入力の判定で使用する変数の仕様を表 3.39 に示す。

表 3.39 ボタン入力の判定で使用する変数の仕様一覧

変数名	型	説明
input_PIN	const uint8_t	ボタンの状態を読み取るデジタルピン番号を定義
switch_status	bool	現在読み取ったボタンピンの状態を格納
before_switch_status	bool	前回読み取ったボタンピンの状態を格納（立下り検知に使用）
isButtonPlaying	bool	演奏状態管理フラグ。トグル制御により演奏中 (true) / 停止中 (false) を保持

表 3.40 ボタン入力の判定で使用する関数の仕様一覧

関数名	説明
button()	input_PIN の立下り (switch_status が 0 かつ before_switch_status が 1) を検知すると isButtonPlaying をトグルし、各楽器機・観覧車デバイスへ演奏開始 (sendStart) または終了 (sendEnd) 信号をユニキャストで送信する。送信後は 300ms の待機によりチャタリングを防止する

3.4.6 加速度センサ制御

指揮デバイスは、加速度センサを用いてユーザが棒を振ったことを検知し、演奏情報の更新を行う。センサ値の急激な変化を読み取ることで動作を判定し、誤検知防止機能をつけることで、安定した動作を実現する。

加速度センサの制御で使用する変数の仕様を表 3.41 に示す。アルゴリズムとしては、差分算出と判定、多重検知の防止、状態の反転、事後処理の 4 段階に分け

られる。差分算出と判定では、現在の加速度センサ値 (currentAccVal) と前回の値 (lastAccVal) の差の絶対値を算出する。ここで currentAccVal の値は、3 軸のセンサ値をそれぞれ 2 乗したものを加算し平方根をとった値とする。この差分が閾値 accThreshold を超え、かつ前回の検知から shakeInterval 以上の時間が経過している場合に、棒を振ったと判断する。

多重検知の防止では、一度振る動作を検知すると、lastDetectTime を現在の時刻に更新する。以降、shakeInterval が経過するまでは、加速度の変化を無視することで、一連の動作による複数回の誤検知を防止する。

状態の反転では、振る動作を検知するたびに、isStarted の bool 値を反転させる。これにより奇数回目の入力で開始 (true)、偶数回目の入力で停止 (false) という状態を表す。

事後処理としては、次回の判定に備えて、currentAccVal の値を lastAccVal に代入し、前回の処理時間を更新する。

また、振る動作の確定時には、可変抵抗器側で設定値の変更を検知したフラグ bpmFrag、またはフラグ volFrag が有効になっている場合、SyncManager クラスを介してその情報の送信指示を出す。これにより、可変抵抗器の操作で更新された BPM・音量の設定値は、次にユーザが棒を振ったタイミングでまとめて BPM は中継機に送信され、音量は各楽器に送信される。

さらに、I2C 通信の異常を検知した場合に自動復旧を行う。センサからのデータ取得時に通信異常を検知すると、SCL ピンを一定回数でクロッキングして STOP 条件を生成することでバスを解放し、Wire 通信および加速度センサの再初期化を行う。この間は直前の加速度値を返すことで、誤った検知の発生を防止する。

表 3.41 加速度センサの制御で使用する変数の仕様一覧

変数名	型	説明
currentAccVal	float	加速度センサから算出した合成加速度の値を格納
lastAccVal	float	変化量算出のための直前の加速度センサ値を格納
accThreshold	const float	振ったと判断するための変化量の閾値を定義
lastDetectTime	unsigned long	前回振ったことを検知した時間を ms 単位で格納
shakeInterval	const unsigned long	連続検知を防止するための待機時間を定義
lastSampleTime	unsigned long	前回センサ値をサンプリングした時間を ms 単位で格納
ACC_SAMPLE_INTERVAL	const unsigned long	加速度センサ値をサンプリングする周期を定義
isStarted	bool	楽器の演奏状態を保持 (true: 演奏中/false: 停止中)

表 3.42 加速度センサ制御 (AccManager クラス) で使用する関数の仕様一覧

関数名	説明
setupADXL345()	ADXL345 の電源投入と測定レンジの設定を行い、初期の加速度値を lastAccVal に格納する初期化関数
update(sync, pot)	ACC_SAMPLE_INTERVAL ごとに加速度値を取得し、差分算出・多重検知防止判定・状態の反転・事後処理までの一連の振る動作検知処理を行う。loop 内で毎回呼び出す
updateShakedInfo(sync, pot)	振る動作の確定時に呼び出され、PotManager の bpmFrag / volFrag が立っている場合に SyncManager 経由で BPM・音量の段階番号を送信する
readAccMagnitude()	ADXL345 から X/Y/Z 軸の生の値を取得し、2 乗和の平方根を返す。I2C 通信異常を検知した場合は直前の加速度値を返す
recoverI2C()	I2C の異常を検知した場合に、再設定を行う

3.4.7 可変抵抗器を用いた BPM・音量読み取り機能

指揮デバイスでは、演奏の要素である BPM と音量を、ユーザが操作できるように可変抵抗器を用いて 5 段階で調整する。アナログ入力値の特性を考慮したデータ処理を行うことで、安定した制御環境を構築する。この処理で使用する変数の仕様を表 3.43 に示す。

使用するパラメータは、可変抵抗器から得られる 0~1023 の値を 5 つの範囲に分割し、それぞれの段階を決定する。

アナログピンで取得される電圧値は、ノイズや電気的特性により細かな変動が生じやすい。この変動が段階の境界付近で発生すると、BPM や音量が意図せず頻繁に切り替わる問題が生じる。これを防止するために、移動平均を用いる。複数回の電圧値を常に平均化して評価することで、電圧値の推移を平滑化し、段階決定の安定性を確保する。

アルゴリズムとしては、サンプリングと移動平均化、範囲分割、パラメータの抽出と出力の 3 段階に分けられる。サンプリングと移動平均化はまず、SAMPLE_INTERVAL ごとに、BPM と音量変更用の可変抵抗器の電圧値をそれぞれ配列 bpmSamples, volSamples に sampleSize 分だけ読み取り保存する。このとき、配列の各要素は書き込み位置 (bpmReadIndex, volReadIndex) が指す最も古い値から順に上書きされ、上書きの直前に該当要素の値を合計 bpmTotal, volTotal からあらかじめ減算し、新たに読み取った値を加算することで、配列全体を読み直すことなく常に最新の合計値を保持する（移動合計）。配列が一巡し全要素が新しい値に更新された時点 (bpmBufferFull, volBufferFull が true になった時点) 以降、この合計値を sampleSize で除算した移動平均値を averageBpmVal, averageVolVal に格納する。これらの操作により、細かな変動を抑えた電圧値を取得する。

範囲分割では、移動平均化された 0～1023 の値が格納された変数 averageBpmVal, averageVolVal をあらかじめ表 3.45 で定義した 5 つの範囲に当てはめて、現在の段階を変数 currentBpmStep, currentVolStep に格納する。

パラメータの抽出と出力では、算出した段階が直前の段階 (prevBpmStep, prevVolStep) から変化した場合にのみ、公開フラグ bpmFrag, volFrag を立てる。外部からは getBpmStep(), getVolStep() を介して現在の段階を取得し、前述の加速度センサ制御における棒振り動作の確定時にこれらのフラグを参照して各楽器機への送信を行う。なお、音量用可変抵抗器は配線の都合上、回した方向とセンサ値の増減が逆になるため、getVolStep() は 6-currentVolStep を返すことで表示上の段階を反転させる。

表 3.43 BPM・音量読み取り機能で使用する変数の仕様一覧

変数名	型	説明
PIN_BPM	const int	BPM 変更用の可変抵抗器を接続するアナログ入力ピン番号を定義
PIN_VOL	const int	音量変更用の可変抵抗器を接続するアナログ入力ピン番号を定義
SAMPLE_INTERVAL	const unsigned long	可変抵抗器の値をサンプリングする周期を定義
sampleSize	const int	移動平均算出のためのサンプル数を定義
STEP_BOUNDARIES[]	const int	段階判定に用いる 4 つの閾値を保持する配列
lastSampleTime	unsigned long	前回サンプリングを行った時間を格納
bpmSamples[]	int	BPM 用のサンプル値を保持するための配列
volSamples[]	int	音量用のサンプル値を保持するための配列
bpmReadIndex	int	bpmSamples[] 内での現在の書き込み位置を管理するインデックス
volReadIndex	int	volSamples[] 内での現在の書き込み位置を管理するインデックス
bpmBufferFull	bool	bpmSamples[] が一巡し、移動平均の算出が有効になったかを保持
volBufferFull	bool	volSamples[] が一巡し、移動平均の算出が有効になったかを保持
bpmTotal	long	bpmSamples[] 内の累積合計を格納
volTotal	long	volSamples[] 内の累積合計を格納
averageBpmVal	float	算出された BPM 用のアナログ入力の移動平均値を格納
averageVolVal	float	算出された音量用のアナログ入力の移動平均値を格納
currentBpmStep	uint8_t	算出された BPM の 5 段階のレベル (1~5) を格納
currentVolStep	uint8_t	算出された音量の 5 段階のレベル (1~5) を格納
prevBpmStep	int	段階の変化検知のための直前の BPM 段階を格納
prevVolStep	int	段階の変化検知のための直前の音量段階を格納
bpmFrag	bool	BPM 段階が変化したことを示す公開フラグ. シェイク確定時の連動送信に使用
volFrag	bool	音量段階が変化したことを示す公開フラグ. シェイク確定時の連動送信に使用

表 3.44 可変抵抗器読み取り機能 (PotManager クラス) で使用する関数の仕様一覧

関数名	説明
update()	SAMPLE_INTERVAL ごとに BPM・音量それぞれの可変抵抗器の値をサンプリングし、移動合計・移動平均・段階算出を行う。loop 内で毎回呼び出す
calcStep(val)	0~1023 の移動平均値を STEP_BOUNDARIES[] と比較し、1~5 のいずれかの段階を返す
bpmUpdatePotentiometers()	BPM 用可変抵抗器の値を読み取り移動合計・移動平均を更新し、段階が変化した場合に true を返す
volUpdatePotentiometers()	音量用可変抵抗器の値を読み取り移動合計・移動平均を更新し、段階が変化した場合に true を返す
getBpmStep()	現在の BPM 段階 (currentBpmStep) を外部に返すゲッター
getVolStep()	現在の音量段階を外部に返すゲッター。配線特性による値の反転を補正し、6-currentVolStep を返す

3.4.8 処理ロジック

表 3.34 で定義した関数を用いて処理ロジックについて説明する。無線通信は、利用するマイコンである Arduino UNO R4 WiFi に合わせるため WiFiS3.h ライブラリを使用する。指揮デバイス (指揮棒) を送信元とし、中継機へは BPM の段階番号を、楽器へはタイミングをずらしながら開始信号 (ヘッダ) と音量をユニキャストで個別に送信する仕様とする。なお、終了時の演奏終了ヘッダは両方に送信する。指揮デバイスに搭載された可変抵抗器 (BPM, 音量をユーザーが操作する用) および指揮棒 (加速度センサを搭載した開始・終了検知用) の状態に基づき、表 3.45 のように電圧値から変換された 1~5 の段階番号や制御コマンドを受信する。受信した段階番号を元に、中継機および楽器デバイスは表 3.46 のような変換をする。

UDP で送信するパケットの構造は表 3.47、表 3.48 のように、識別用のヘッダとデータ本体からなる合計 2 バイトの固定長ペイロードとして定義する。ここで UDP

は到達保証がないプロトコルなので、システムの信頼性を高めるために冗長送信と送信待機を行う。まず、冗長送信では、パケットロスによる制御の失敗を防ぐため、BPM 変更、演奏開始・終了の各イベント発生時に、同一パケットを 3 回連続して送信する。また、送信待機では指揮デバイスの状態が変化した時だけ、通信を行うことでネットワークの帯域の占有を最小限に抑える。

表 3.45 段階と入力値範囲の対応表

段階	入力値の範囲
1	0 – 300
2	301 – 500
3	501 – 650
4	651 – 818
5	819 – 1023

表 3.46 各段階に対応する BPM および音量設定値

段階	BPM 設定値	音量設定値
1	60	0
2	90	64
3	120	128
4	150	192
5	180	255

表 3.47 通信の識別ヘッダ (第 1 バイト:char)

ヘッダ	値 (ASCII コード)	内容
“B”	0x42	BPM データ
“V”	0x56	音量データ
“S”	0x53	演奏開始 (Start)
“E”	0x45	演奏終了 (End)

表 3.48 各ヘッダに対応したペイロードの内容 (第 2 バイト:uint8_t)

ヘッダ	ペイロード内容
"B"	BPM の段階番号 (1~5) を受信. 中継機内部の bpmTable[] を参照し, 設定値へ変換.
"V"	音量 の段階番号 (1~5) を受信. 楽器ユニットへ送信される.
"S"	演奏開始コマンド. 各楽器ユニットが受信し, このパケットの到達をトリガーとして即座に演奏を開始.
"E"	演奏終了コマンド. 第 2 バイトにはダミーデータ (0) が格納されており, 受信後は停止処理へ移行.

3.4.9 回転速度同期機構の設計

モータ制御の原理と BPM の変換について説明する. 本システムでは, Arduino の PWM (パルス幅変調) 機能を用いてモータの平均印加電圧を制御する. 供給電圧を V_{cc} (定数 vcc), プログラム上の出力値を $pwmValue$ とすると, モータに加わる平均電圧 $V_{average}$ は (3.4) 式で決定される.

$$V_{average} = V_{cc} \times \frac{pwmValue}{255} \quad (3.4)$$

次にルックアップテーブルによる速度制御について説明する. 一般的な DC モータの回転速度は供給電圧に比例する性質があるが [15], 実際には摩擦や機構の負荷などによる非線形な挙動を示す事が多い. そのため, 当初検討していた「4 拍で 90 度 (1 小節が 90 度) 回転する」といった幾何学的な理論式による制御ではなく, より確実な速度追従を実現するため, 幾何学的な縛りを廃止して実測値に基づくルックアップテーブル方式を採用する.

本システムでは, 表 3.49 に示す離散的な 5 段階の BPM 設定値とコード内の実測周期テーブル (ROTATION_PERIOD_LIST) を対応させ, 受信した段階番号に応じた $pwmValue$ を即座に選択・出力する構成とする.

表 3.49 BPM に対する実測周期と出力 PWM 値

段階	目標 BPM	実測周期 [ms]	出力 PWM 値
1	60	230	26
2	90	180	28
3	120	140	30
4	150	110	32
5	180	80	37

ここからは実測値の測定方法について説明する。

まず、レーザーを 2 回読み取るまでの時間差（周期）を測定する。得られた周期から、楽器内に定義された実測周期テーブル ROTATION_PERIOD_LIST[]（あらかじめ PWM 出力に対して測定・予測された値）の中で最も値に近いものをループ処理によって探索する。探索されたインデックスに対応する BPM（一番遅い 60 から一番速い 180 までの 5 段階）を読み取ることで、幾何学的な理論式に依存しない、実際の回転速度に即した確実な BPM 特定を行っている。

3.4.9.1 レーザー光の読み取り

レーザー光を読み取るプログラムで使用する定数と変数と関数をそれぞれ表 3.50, 表 3.51, 表 3.52 に示す。レーザー光の読み取りの処理は SyncManager クラスにより管理される。固定の閾値のみでレーザー通過の立ち上がりを判定すると、周囲光の変化とレーザー光の強弱により誤検知が生じる。そのため、レーザーが当たっていない状態の CdS セルのセンサ値を変数 baseline として継続的に追跡し、レーザーの照射の有無の閾値を変数 baseline からの相対値として動的に補正しながら、レーザーの通過を検知する。具体的な処理手順としては、まず閾値の動的補正について説明する。レーザーが当たっていない時の読み取り値は変数 baseline として継続的に追跡される。変数 baseline は読み取り値が暗くなる方向へは即座に追従し、明るくなる方向へは重み付き移動平均で緩やかに追従するフィルタによって更新され、レーザー通過につられて変数 baseline が誤って増加されることを防いでいる。レーザー通過後、まず個々のピーク値と変数 baseline との差が定数 CDS_MIN_DELTA 未満であるピークはノイズとして無視され、直近のピーク値を保持する配列 peakHistory への登録対象から除外される。peakHistory には最大で定数 CDS_PEAK_HISTORY 件分のピーク値が循環的に記録されるが、記録件数が CDS_PEAK_HISTORY 件に達する

までは、その時点までに記録された件数分のみで平均が算出される。また、記録件数が 3 件未満の間は、後述する閾値の更新自体が行われない。記録件数が 3 件以上になると、関数 registerPeak によって peakHistory 内の平均値と変数 baseline との差 (振幅) が算出され、振幅が定数 CDS_MIN_DELTA 未満である場合は改めてノイズとして無視される。振幅が定数 CDS_MIN_DELTA 以上の場合、変数 onThreshold と変数 offThreshold の目標値を以下の (3.5) 式、(3.6) 式により算出する。

$$\text{onThresholdtarget} = \text{baseline} + \text{振幅} \times \frac{\text{CDS_ON_RATIO}}{100} \quad (3.5)$$

$$\text{offThresholdtarget} = \text{baseline} + \text{振幅} \times \frac{\text{CDS_OFF_RATIO}}{100} \quad (3.6)$$

算出された目標値は 0 以上 1023 以下の範囲に収められ、さらに offThreshold_{target} は onThreshold_{target} を超えないよう制限される。このようにして得られた目標値は、閾値がまだ一度も確定していない場合はそのまま変数 onThreshold、変数 offThreshold へ反映されるが、一度確定した後の更新では、関数 limitStep により変化幅が定数 CDS_THRESHOLD_STEP_LIMIT 以内に制限された上で反映され、急激な閾値の変動を抑えている。また、更新後に offThreshold が onThreshold 以上となった場合は、onThreshold を下回るよう offThreshold が再調整される。補正が一度も行われていない間は、関数 updateFallbackThresholds が変数 baseline から仮の閾値を算出する。次に起動時の誤検知防止として、電源投入後定数 CDS_STARTUP_CALIBRATION_MS の間は、変数 startupCalibrated が false のままとなり、検知を行わず、この間に観測された最小値を変数 startupMin として変数 baseline の初期値に採用する。キャリブレーション終了時点ですでに読み取り値が閾値以上であった場合は、変数 waitingForRelease を true にし、一度読み取り値が変数 releaseThreshold まで下がってレーザーが当たっていない状態まで検知を保留する。続いて、レーザー通過の検知確定について説明する。読み取り値が変数 onThreshold 以上になった瞬間に変数 laserOn が true となり、その回の関数 detectLight が true を返す。その後読み取り値が変数 offThreshold 以下に下がると変数 laserOn が false に戻り、関数 registerPeak が呼び出される。また、変数 lastDetectTime から定数 DEBOUNCE_MS 未満に生じた検知は、関数 update 内で無視され、光量の細かな変動による多重カウントを防止している。

表 3.50 レーザー光読み取り機能で使用する定数の仕様一覧

変数名	型	説明
CDS_PIN	const int	CdS セルを接続するアナログ入力ピン番号を定義
CDS_ON_THRESHOLD	const int	較正が完了するまでの間だけ使用する仮の ON 判定しきい値
CDS_OFF_THRESHOLD	const int	較正が完了するまでの間だけ使用する仮の OFF 判定しきい値
CDS_MIN_DELTA	const int	baseline との差がこの値未満のピークをノイズとして無視する最小振幅
CDS_PEAK_HISTORY	const uint8_t	しきい値較正に用いる直近ピーク値の保持個数を定義
CDS_ON_RATIO	const uint8_t	baseline から振幅の何%を ON 判定しきい値とするかを定義
CDS_OFF_RATIO	const uint8_t	baseline から振幅の何%を OFF 判定しきい値とするかを定義
CDS_THRESHOLD_STEP_LIMIT	const int	1 回の較正でしきい値が変化できる最大幅を定義
CDS_STARTUP_CALIBRATION_MS	const unsigned long	起動直後にレーザー検知を行わないキャリブレーション時間を定義
DEBOUNCE_MS	const unsigned long	直前の検知からこの時間未満の再検知を無視するデバウンス時間を定義

表 3.51 レーザー光読み取り機能（SyncManager クラス）で使用する変数の仕様一覧

変数名	型	説明
baseline	int	暗状態（レーザー非通過時）の読み取り値の追従値を格納
baselineReady	bool	baseline の初期値取得が完了したかを格納
onThreshold	int	現在有効な ON 判定しきい値を格納
offThreshold	int	現在有効な OFF 判定しきい値を格納
laserOn	bool	現在レーザーが CdS セルに当たっている（ON 状態）かどうかを格納
pulsePeak	int	検知中のパルスにおける読み取り値の最大値を一時的に格納
peakHistory[]	int	しきい値較正に用いる直近ピーク値を保持する配列
peakIndex	uint8_t	peakHistory[] 内での現在の書き込み位置を管理するインデックス
peakCount	uint8_t	peakHistory[] に記録済みのピーク数を格納
calibrated	bool	しきい値の較正が一度でも行われたかを格納
startupCalibrated	bool	起動直後のキャリブレーション期間が終了したかを格納
calibrationStartMs	unsigned long	起動キャリブレーションを開始した時刻を格納
startupMin	int	起動キャリブレーション中に観測された最小値（暗状態の候補）を格納
waitingForRelease	bool	起動直後に既にレーザーが当たっていた場合、暗転を待っている状態かを格納
releaseThreshold	int	waitingForRelease 状態を解除するための判定値を格納
lastDetectTime	unsigned long	直前にレーザー検知が確定した時刻を格納

表 3.52 レーザー光読み取り機能（SyncManager クラス）で使用する関数の仕様一覧

関数名	説明
SyncManager()	コンストラクタ. baseline 未取得・仮しきい値の状態に各メンバ変数を初期化する
reset()	全ての内部状態を初期値に戻す. 演奏開始（'S' 受信）時など、再較正が必要な場面で呼び出す
update(cdsValue, nowMs)	CdS の読み取り値と現在時刻を受け取り、しきい値較正・エッジ検出・デバウンス・BPM 判定までを行う公開関数. レーザー通過を検知した回のみ true を返す
detectLight(v, nowMs)	baseline の追従, 起動キャリブレーション, waitingForRelease 状態の解除, ON / OFF エッジの検出を行う
registerPeak(peak)	検知されたパルスのピーク値を peakHistory[] に記録し, baseline との振幅から ON / OFF しきい値を再較正する
updateFallbackThresholds()	しきい値較正が完了する前に使用する仮のしきい値を baseline から算出する
limitStep(current, target, limit)	しきい値の 1 回あたりの変化幅を制限するヘルパー関数
clearPeakHistory()	peakHistory[] を初期化する

3.4.10 LED マトリクスでの BPM 表示

まず、LED マトリクスにおける BPM 表示について説明する。今回のシステムでは表示用のマトリクスとして、Arduino Uno R4 WiFi 付属のマトリクスを使用する。このマトリクスを用いて、中継機から受信した BPM 情報を元に数値変換を行い、独自定義しフォント配列を用いて表示を行う。ここからは、本機能におけるファイル構成、関数設計について説明する。

表 3.53 BPM 表示機能のファイル構成

ファイル名	概要
Display.ino	WiFi 接続・UDP 受信開始の管理
Display.cpp	表示文字の独自フォント定義, LED マトリクスへの描画処理, UDP 受信処理
Display.h	定数定義, 関数宣言, 外部参照

3.4.10.1 ファイル構成

ファイル構成について説明する。また、表 3.53 に BPM 表示機能のファイル構成を示している。はじめに、Display.ino ファイルでは、表 3.53 に示すように WiFi 接続と UDP 接続を行う。今回、指揮機と中継機間の通信形式としては、高速通信を目的とし、UDP 形式で情報を受け取る。また、WiFi 接続の前に WiFi.config(localIP, gateway, subnet) を呼び出し、指揮デバイスの送信先 IP アドレスと一致する静的 IP を設定する。

次に、Display.cpp ファイルでは、表示文字の独自フォント定義、マトリクスへの描画処理、そして通信処理を行う。また、Display.h ファイルでは定数の定義と関数宣言、独自フォント定義などを行う。ここで、今回利用する Arduino のマトリクスは縦 12 横 8 の範囲で構成されている。この時、通常で定義されている文字列では文字の間隔が短くなることや、それに付随した可読性の低下が考えられる。よって、今回は 1 文字を縦 5 横 3 で再定義することにより文字同士の間隔を開け、可読性の担保を目指す。例として数字 0 を以下に示す。

```
{1,1,1,
1,0,1,
1,0,1,
1,0,1,
1,0,1,
1,1,1}
```

また、LED マトリクスの描画処理は、受信した段階番号から BPM 値を参照し、整数を各桁に分割した後、独自定義した 3×5 のフォント配列を用いてフレームバッファに書き込むことで行う。

3.4.10.2 オブジェクト図 (クラス図)

LEDMatrix プログラムのクラス図は以下の図 3.30 の通りである。通信開始を行う Display.ino、通信受信処理と LED マトリクスへの描画処理を行う Display.cpp、および関数宣言やフォント定義を行う Display.h で構成される。各モジュール間の関係に

ついて、Display.ino から Display.cpp への関係は、表示処理を利用する依存関係である。また、Display.cpp から Display.h への関係は、関数宣言およびフォント定義を参照する依存関係である。

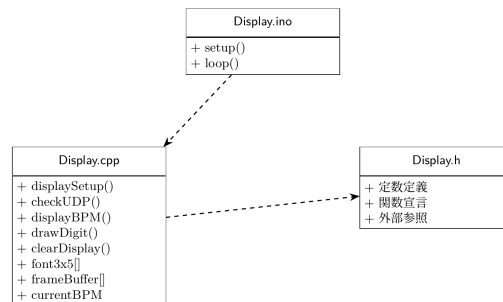


図 3.30 LEDMatrix のクラス図

3.4.11 関数設計

表 3.54 に今回用いる主な処理を行う関数を示している。以降では、各関数の具体的な動作を説明する。

はじめに、displaySetup 関数について説明する。この関数は、表 3.54 に示すように、LEDMatrix の初期化を行い、引数や戻り値はない。matrix.begin() を呼び出した後、消灯フレームを一度描画してから 200 ms 待機する。これは、begin() 直後の最初の loadFrame() は反映されない場合があるための対策である。

次に、checkUDP 関数について説明する。この関数は表 3.54 に示すように、中継機からのパケット確認と BPM 情報の値確認を行い、引数や戻り値はない。まず、中継機からのパケットを Udp.parsePacket() を用いて監視する。パケットが届いていることを確認できれば、uint8_t buffer[2] としてバイナリ形式で 2 バイトを読み取る。buffer[0] をヘッダ文字として判定し、ヘッダが'E' の場合は clearDisplay() を呼び出して LED を消灯する。また、ヘッダが'B' の場合は buffer[1] の段階番号を bpmTable[] で参照して BPM 値に変換し、値が更新されていれば描画更新処理を行う。

次に、displayBPM 関数について説明する。この関数は表 3.54 に示すように、BPM 数値のマトリクス描画を行い、引数としては、checkUDP 関数から保持した数値を用いる。まず、BPM 値を各桁ごとに分割し、drawDigit() を用いてフレームバッファに

描画する。そして、uint32_t[3] 形式にビットパックした後、matrix.loadFrame() で描画を行う。

次に、drawDigit 関数について説明する。この関数は、表 3.54 に示すように、一桁の数字をフレームバッファに描画をし、引数としては描画する数字とフレームバッファ上の横方向開始位置を取る。まず、引数の数字が 0 から 9 の範囲内であるかを確認する。次に、縦方向の中央寄せオフセットを算出して配置する。その後、独自フォント配列から該当する数字のドットパターンを 1 ドットずつ読み取り、フレームバッファの指定位置に書き込む。

最後に、clearDisplay 関数について説明する。この関数は、表 3.54 に示すように、LEDMatrix を全消灯させる処理を行い、引数や戻り値はない。BPM 値を初期値にリセットし、全消灯フレームを matrix.loadFrame() で描画して表示のリセットを行う。

表 3.54 主要な処理を行う関数一覧

関数名	概要	引数	戻り値
displaySetup	LED マトリックスの初期化	なし	なし
checkUDP	UDP パケットの確認、BPM 更新・消灯処理	なし	なし
displayBPM	BPM 数値を LED マトリックスに描画	int bpm	なし
drawDigit	1 桁の数字をフレームバッファに描画	int digit, int x_offset	なし
clearDisplay	LED マトリックスを全消灯	なし	なし

3.4.12 Tick 管理およびシリアル・同期通信

本システムにおいて、複数台のデバイス間でテンポと再生位置を正確に同期させ、輪唱を実現するとともに、Processing（音源）側へ正確な発音指示を送るための制御処理について説明する。まず、同期のシーケンス図を図 3.31 に示す。

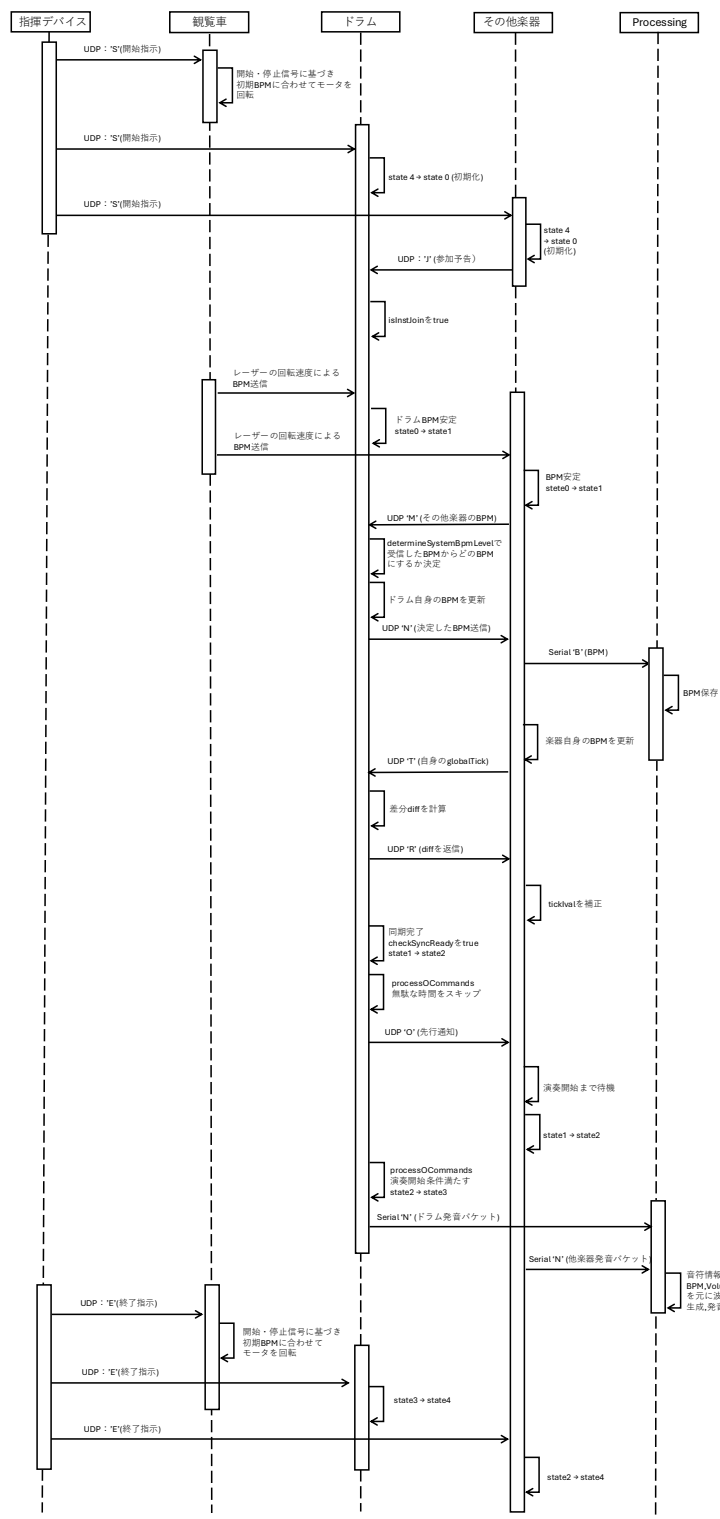


図 3.31 同期・輪唱システムのシーケンス図

本機能では、観覧車のレーザーセンサから取得した BPM の多数決による統一、UDP 通信のネットワーク遅延（レイテンシ）を動的に相殺する Tick（再生位置）の補正、そして計算された Tick に基づくシリアル通信（パケット送信）を統合して行う。

3.4.12.1 ファイル構成

ファイル構成について説明する。表 3.55 に同期・制御機能のファイル構成を示す。

表 3.55 同期・制御機能のファイル構成

ファイル名	概要
drum.ino	マスター機（ドラム）制御。BPM の集約と基準時間の提供、演奏開始管理
notDrum.ino	スレーブ機（他楽器）制御。BPM の提案、通信遅延を考慮した Tick 同期補正

はじめに、Drum.ino ファイルは本システムのマスター機として機能する。参加する各楽器機からの参加表明（J 通信）を受信して動的な編成管理を行うとともに、各楽器から提案された BPM（M 通信）を集約して多数決を行い、システム全体の基準となる確定 BPM（N 通信）を配信する。また、内部に保持した楽譜データに基づき、適切なタイミングで各楽器へ演奏開始指示（O 通信）を送信する司令塔の役割を担う。

次に、notDrum.ino ファイルはスレーブ機として機能する。ドラム機から配信された確定 BPM に従って内部の基本ティック周期を設定し、自律的に Tick を加算しつつドラム機と定期的な通信を行う。通信によって得られた時間差分から、ネットワーク遅延を考慮した高度なフィードバック制御によって再生速度を微調整する。

また、各ファイルの内部ではメモリ制限を考慮して `stdint.h` を用いた構造体（NoteData）を定義し、楽譜データをフラッシュメモリ領域（`const` 配列）に保持している。

3.4.12.2 変数・関数の設計

ドラム

本設計に使用する定数は表 3.56 の通りである。これらは実装において、プログラム内で `const` を使って定義する。

表 3.56 設計に利用する定数一覧

型	名前	説明	メモリ量 [byte]
char[]	SSID	Wi-Fi のネットワーク名 (hackathon001-WPA2)	17
char[]	PASS	その Wi-Fi のパスワード (hackathon001)	12
IPAddress	localIP	ドラム（マスター）の IP アドレス	4
uint16_t	LOCAL_PORT	自身のポート番号	2
int32_t	O_ADVANCE_TICKS	O 通信を演奏開始の何 Tick 前に送るか	4
int	MAX_SONGS	最大の曲数	4
int	MAX_EVENTS	最大の発音開始指示の 上限	4
int	drumInstId	ドラムの ID	5
uint8_t	MAX_INSTRUMENTS	最大楽器数	1
int32_t	SYNC_READY_THRESH- OLD	同期完了チェックの 許容 Tick	4
int	MAX_JUMPS	楽譜の最大ジャンプ（移 動）数	4
int32_t	jumpT- able[MAX_SONGS][MAX_ JUMPS]	曲ごとのジャンプタイ ムとジャンプ先の 指定	960
SongStartData	songStart- Params[MAX_SONGS]	Processing の設定に合 わせた初期値の定義	32

本設計に使用する変数は表 3.57 の通りである。

表 3.57 設計に利用する変数一覧

型	名前	説明	メモリ量 [byte]
uint32_t	globalTick	全体基準の Tick	4
uint32_t	curTick	ドラム楽譜用 Tick	4
uint32_t	lastTick	updateTick 用タイムスタンプ	4
uint32_t	ticklval	1Tick あたりの [μ s]	4
uint16_t	scoreldx	次に発音する予定のデータのインデックス	2
uint8_t	curBpm	現在の BPM	1
uint8_t	curVol	現在の音量	1
int	startSequence[MAX_SONGS][MAX_EVENTS][2]	楽器ごとの演奏開始タの指定	320
int	currentSongId	現在の曲の ID	4
int32_t	elapsedPlayTick	state3 移行後(演奏開始後)の経過 Tick	4
bool	isDrumPlaying	ドラムが演奏中かどうか	1
bool	oSentFlags[MAX_EVENTS]	各イベントの O 通信送信済みか	1
bool	isInstConnected[MAX_INSTRUMENTS]	同期に参加してるかどうか	1
bool	isInstJoin[MAX_INSTRUMENTS]	演奏同期に参加する予告が来たか	1
int32_t	lastReceivedDiff[MAX_INSTRUMENTS]	他の楽器から T 通信で送られる diff を格納	4
uint8_t	receivedBpmLevels[MAX_INSTRUMENTS]	参加している楽器の BPM 段階を集約する(ドラム自身も含む)	1
uint8_t	currentBpmLevel	現在確定している BPM 段階 (前回値)	1
int	state	状態	4
uint8_t	currentJumpIdx	現在のジャンプインデックス	1

本設計に利用する構造体は表 3.58 の通りである。

表 3.58 設計に利用する構造体一覧

型	名前	説明	サイズ [byte]
struct	NoteData	音符データを保持する構造体	8
struct	SongStartData	曲ごとの初期設定データを保持する構造体	8

さらに、構造体の中身を NoteData は表 3.59 に、SongStartData は表 3.60 に示している。

表 3.59 NoteData の中身一覧

型	メンバ名	説明	サイズ [byte]
uint16_t	timing	発音タイミング（楽曲開始からの累積 Tick 数）	2
uint16_t	duration	発音の長さ（Tick 単位）	2
uint8_t	pitch	音高を示す MIDI ノート番号（0～127）	1
uint8_t	startVelocity	発音開始時の強さ（0～127）	1
uint8_t	endVelocity	発音終了時の強さ（0～127）	1

表 3.60 SongStartData の中身一覧

型	メンバ名	説明	サイズ [byte]
uint32_t	startTick	曲の開始時刻（tick）	4
uint16_t	startNoteldx	開始ノート番号	2

ドラム以外の楽器

本設計に使用する定数は表 3.61 の通りである。これらは実装において、プログラム内で const を使って定義する。

表 3.61 設計に利用する定数一覧

型	名前	説明	サイズ [byte]
char[]	SSID	Wi-Fi の ネ ッ ト ワ ー ク 名 (hackathon001- WPA2)	17
char[]	PASS	その Wi-Fi の パ ス ワ ー ド (hackathon001)	12
IPAddress	localIP	楽器の IP アドレス	4
uint16_t	LOCAL_PORT	自 身 の ポ ー ト 番 号	2
uint8_t	MY_INST_ID	楽器の ID	1
uint16_t	note_count	NoteData に 含 ま れ る 音 符 の 数	2
int	MAX_SONGS	最大の 曲 数	4
int	MAX_JUMPS	楽 譜 の 最 大 ジ ャ ン プ (移 動) 数	4
int32_t	jumpTable[MAX_SONGS][MAX_JUMPS][3]	曲 ご と の ジ ャ ン プ タ イ ミ ン グ と ジ ャ ン プ 先 の 指 定	960
SongStartData	songStartParams[MAX_SONGS]	Process- ing の 設 定 に 合 わ せ た 初 期 値 の 定 義	32

本設計に使用する変数は表 3.62 の通りである。

表 3.62 設計に利用する変数一覧

型	名前	説明	メモリ量 [byte]
uint16_t	scoreIdx	次に発音する予定のデータのインデックス	2
uint32_t	globalTick	全体基準の Tick	4
uint32_t	curTick	楽譜用 Tick	4
uint32_t	lastTick	updateTick 用タイムスタンプ	4
uint32_t	baseTickIval	BPM から計算した基本 interval	4
uint32_t	tickIval	実際に使う interval (同期補正あり)	4
int32_t	ticksToCorrect	補正残り Tick 数	4
int32_t	lastDiff	直近の diff (BPM 変化時の再計算用)	4
uint32_t	lastTPacketMicroTime	直近の T パケット送信時の時刻 Micro	4
uint32_t	lastTPacketTick	直近の T パケット送信時の globalTick	4
bool	drumNotStarted	ドラムが演奏していないかどうか	1
uint8_t	curBpm	現在の BPM	1
uint8_t	curVol	現在の音量	1
bool	waitingForStart	O 通信を受信して演奏開始 globalTick を待っている	1
uint32_t	startGlobalTick	ドラムから指定された演奏開始 globalTick	4
int	currentSongId	現在の曲の ID	4
int	state	状態	4
uint8_t	currentJumpIdx	現在のジャンプインデックス	1

本設計に利用する構造体は表 3.63 の通りである。

表 3.63 設計に利用する構造体一覧

型	名前	説明	サイズ [byte]
struct	NoteData	音符データを保持する構造体	8
SongStartData	songStartParams[MAX_SONGS]	Processing の設定に合わせた初期値の定義	32

さらに、構造体の中身を NoteData は表 3.59, SongStartData は表 3.60 に示している (ドラムと同様)。

3.4.12.3 関数設計

ドラム

本設計に使用している関数を表 3.64 に示す。

表 3.64 drum.ino の同期に関する関数一覧

関数名	概要	引数	返り値
updateTick	時間経過による自律 Tick 加算, 定期的な BPM 確定と N 通信の実行	なし	なし
setBpm	BPM の更新および Tick 間隔の再計算, PC へのシリアル送信	uint8_t bpm	なし
sendVolumeValue	音量の更新および PC へのシリアル送信	uint8_t vol	なし
sendPacket	楽譜の走査と発音データの PC へのシリアル送信	なし	なし
processOCommands	楽譜を監視し, 他楽器への演奏開始指示や待機時間のスキップを実行	なし	なし
sendOPacket	他楽器への演奏開始指示 (O 通信) の UDP 送信	uint8_t instId, uint8_t songId, uint32_t start-GlobalTick	なし
checkSyncReady	参加予定の全楽器の接続状態と Tick 誤差の収束判定	なし	bool
sendNPacket	確定したシステム BPM (N 通信) の全参加楽器への UDP 送信	uint8_t bpm	なし
determineSystemBpmLevel	全参加端末の BPM から多数決 (中央値) で基準 BPM を決定	なし	uint8_t

本機能におけるドラム機 (マスター) の各関数の具体的な動作について説明する。ドラム機はシステム全体の時間軸と BPM を統括し, 他楽器への演奏指示を行う司令

塔として機能する。

はじめに、時間管理とシリアル通信に関する関数について説明する。

updateTick 関数は、ドラム機の心臓部として機能する。現在の BPM から算出されたインターバル (tickIval) ごとに、システム全体の絶対時間である globalTick と、ドラム自身の楽譜走査用である curTick を 1 ずつ加算する。さらに、48 Tick 周期で後述の determineSystemBpmLevel 関数を呼び出して BPM を再評価し、sendNPacket によって全端末へ同期させる役割も持つ。演奏状態 (State 3) においては、この関数内から sendPacket 関数を呼び出し、ジャンプ処理 (ループ) を含めた楽譜の進行を行う。

sendPacket 関数は、curTick に到達した音符データをフラッシュメモリ上の楽譜配列から読み出し、音高、長さ、ベロシティ等の情報を Processing へシリアル送信 (N パケット) する。

setBpm 関数および sendVolumeValue 関数は、システム内部の BPM や音量変数を更新し、その変更を即座に Processing へシリアル送信 (B パケット, V パケット) して音源へ反映させる。

次に、システムの BPM 決定と同期準備に関する関数について説明する。

determineSystemBpmLevel 関数は、ドラム自身および他楽器から集められた BPM の値を配列に格納し、バブルソートを用いて昇順に並べ替える。要素数が偶数の場合は、中央の 2 値のうち前回の確定 BPM に近い方を採用する独自ロジックにより、レーザーの誤検知等による急激なテンポ変動を防止し、安定したシステム BPM を算出する。

sendNPacket 関数は、確定した BPM を参加中の全楽器機へ UDP ブロードキャストし、全体の Tick 進行速度を同期させる。

checkSyncReady 関数は、演奏開始前の同期待ち状態 (State 1) において、参加予定 (J 通信を受信済み) の全楽器が接続を完了し、かつドラム機との Tick 誤差が閾値 (6 Tick) 以内に収束しているかを評価し、演奏準備完了を判定する。

最後に、他楽器の演奏をコントロールする輪唱 (先行制御) に関する関数について説明する。

processOCommands 関数は、ドラム機が保持する全体の楽譜シーケンス (startSequence) を監視し、各楽器の演奏開始タイミングを制御する。対象楽器が演奏を開始すべき Tick の 96 Tick 前 (O_ADVANCE_TICKS) に到達した時点で、sendOPacket 関数を呼び出す。また、ドラム自身の演奏開始タイミングの処理や、参

加楽器が存在しない無音区間を動的にスキップしてタイムラインを前倒しする高度な待機調整処理もここで行われる。

`sendOPacket` 関数は、ターゲットとなる楽器機に対して、演奏を開始すべき絶対時間 (`startGlobalTick`) を封入した UDP パケット (O 通信) を送信し、精度の高い輪唱を実現する。

ドラム以外の楽器

本設計に使用している関数を表 3.65 に示す。

表 3.65 notDrum.ino の同期に関する関数一覧

関数名	概要	引数	返り値
updateTick	時間経過による自律 Tick 加算, 同期補正の カウントダウンおよび パケット送信の実行	なし	なし
setBpm	基準 BPM の更新とイン ターバルの再計算, およ び PC へのシリアル送信	uint8_t bpm	なし
applyDiffCorrection	通信遅延を相殺した誤 差 (diff) からの Tick 進 行速度 (tickIval) の伸 縮補正	int32_t diff	なし
sendTPacket	レーザー検知時におけ る自身の globalTick (T 通信) の UDP 送信	なし	なし
sendJPacket	演奏開始時におけるド ラム機への参加予告 (J 通信) の UDP 送信	なし	なし
sendPacket	楽譜の走査と発音デー タの PC へのシリアル 送信	なし	なし
sendMPacket	自身が算出した BPM 提 案値 (M 通信) のドラム 機への UDP 送信	uint8_t bpm	なし

本機能におけるスレーブ機（ドラム以外の楽器）の各関数の具体的な動作について説明する。スレーブ機は、自身のレーザーセンサでテンポを計測しつつも、最終的な時間軸と BPM はマスター機（ドラム機）の指示に従い、ネットワーク遅延を考慮した高度なフィードバック制御によって再生位置を同期させる役割を持つ。

はじめに、時間管理とシリアル通信に関する関数について説明する。

updateTick 関数は、スレーブ機の時間進行を司る。内部タイマーに基づき、動的に変動するインターバル (tickIval) ごとに自身の絶対時間 (globalTick) と楽譜走査用時間 (curTick) を加算する。同時に、後述する同期補正の残り Tick 数 (ticksToCorrect) のカウントダウン処理も行い、補正が完了すると基準の進行速度へ自動的に復帰する。演奏状態 (State 2) では、内部から sendPacket 関数を呼び出して楽譜を進行させる。

sendPacket 関数は、curTick の進行に合わせてフラッシュメモリ上の楽譜配列を走査し、発音タイミングに到達した音符データを Processing へシリアル送信する。

setBpm 関数は、ドラム機から確定 BPM (N 通信) を受信した際に呼び出され、基本となる Tick 間隔 (baseTickIval) を再計算する。この際、もし Tick 補正の実行中であれば、変更後の BPM 比率に合わせて補正スピードを動的にスケールさせる処理を行い、シリアル通信で Processing へ BPM 変更を通知する。

次に、ドラム機との同期および UDP 通信に関する関数について説明する。

sendJPacket 関数は、指揮機から演奏開始指示 (S 通信) を受信した直後に実行され、ドラム機に対して自身のネットワーク参加を知らせる J 通信を送信する。

sendMPacket 関数は、自機のレーザーセンサが回転を検知し、BPM を算出するたびに実行される。算出した BPM を M 通信としてドラム機へ送信し、システム全体の BPM 決定 (多数決) のためのデータを提供する。

sendTPacket 関数は、レーザーを検知するたびに、その瞬間の自身の globalTick をドラム機へ送信する (T 通信)。この送信時刻はマイクロ秒単位で記録され、ドラム機からの返信 (R 通信) を受け取った際の通信遅延 (RTT) 計算の基点となる。

最後に、本システムの同期ロジックの要となる applyDiffCorrection 関数について説明する。

本関数は、ドラム機から R 通信で返信された Tick 誤差 (rawDiff) に対して、計測した通信遅延 (latencyTicks) を差し引いた「真の誤差 (diff)」を引数として受け取る。

算出された誤差が ± 2 Tick 以内の場合はネットワークの揺らぎと判断して補正を行わない (不感帯)。誤差が ± 24 Tick 以上の致命的なズレの場合は、インターバル (tickIval) を一時的に 2 倍 (遅延) または $2/3$ (加速) に設定して強力に補正する。それ以外の通常範囲のズレに対しては、次の 48 Tick をかけて誤差を滑らかに吸収する比例制御式 ($\text{tickIval} = \frac{48 \times \text{baseTickIval}}{48 + \text{diff}}$) を適用する。このロジックにより、時間を直接操作 (ワープ) することなく、演奏のテンポをコンマ数ミリ秒単位で伸縮させる

ことで、音飛びのない極めて自然で高精度な同期を実現している。

3.4.12.4 ヘッダーとステート

本設計に利用しているヘッダーについて、表 3.66 および表 3.67 に示す。

表 3.66 ドラムに利用しているヘッダー

ヘッダー	説明
'S'	指揮機からのブロードキャスト開始指示. state を 0 にして初期化など.
'E'	指揮機からのブロードキャスト終了指示. state を 4 にして初期化など.
'T'	楽器からの同期パケット. R を送信する.
'V'	指揮機からの音量変更指示. 段階番号に応じて変更する.
'J'	楽器からの演奏参加予告. 同期に参加する楽器を読み取る.
'M'	子機からの BPM 提案を受信.

表 3.67 子機に利用しているヘッダー

ヘッダー	説明
'S'	指揮機からのブロードキャスト開始指示. state を 0 にして初期化など.
'E'	指揮機からのブロードキャスト終了指示. state を 4 にして初期化など.
'R'	ドラムからの diff 返信. diff を計算して補正.
'O'	ドラムからの演奏開始指示.
'V'	指揮機からの音量変更指示. 段階番号に応じて変更する.
'N'	マスターからの確定 BPM を受信.

本設計に利用しているステート（状態）について、表 3.68 および表 3.69 に示す。

表 3.68 ドラムで利用している State

State	説明
4	システムが停止していて通信の受付のみを行っている.
0	BPM 安定待ち. レーザーがまだ安定しておらず待っている.
1	BPM が安定し, 楽器と同期待ち.
2	同期が完了し, 演奏開始待ち.
3	演奏中.

表 3.69 子機で利用している State

State	説明
4	システムが停止していて通信の受付のみを行っている.
0	BPM 安定待ち. レーザーがまだ安定しておらず待っている.
1	同期を行っている最中.
2	演奏中.

3.4.12.5 状態遷移ロジック（ステートの遷移タイミング）

本システムが自律的かつ協調的に動作するため, 特定の packets 受信やセンサの検出に基づき, 内部のステートを厳密に遷移させる制御を行っている. 以下に, ドラム機（親機）および他楽器機（子機）における状態遷移の契機（トリガー）を説明する.

ドラム機の状態遷移

ドラム機における各ステート間の遷移タイミングは以下の通りである.

- ・ **任意状態 → State 4 / State 0** : 指揮機から'E' 通信（終了指示）を受信するとシステムは即座に State 4 へ移行し, すべての演奏変数と通信フラグが初期化される. また, 'S' 通信（開始指示）を受信すると State 0 へ移行し, 楽曲 ID の更新を伴う演奏開始シーケンスが始動する.
- ・ **State 0 → State 1 (BPM 安定判定)** : 自身のレーザーセンサによる BPM 算出処理において, 算出値が規定回数 (MIN_STABLE_COUNT = 3 回) 連続して一致した瞬間, BPM 確定とみなして State 1 へ遷移する. この際, 確定したシステム BPM を保持して N 通信を全楽器機へ配信する.

- ・ **State 1 → State 2 (全楽器の同期完了)**: 関数 `checkSyncReady()` により、参加予定 (J 通信受信済み) のすべての他楽器機から T 通信が届き (`isInstConnected`), かつ各楽器機との Tick 誤差 (`diff`) が規定の範囲内 (± 6 Tick 以内) に収束したと判定された瞬間に State 2 へ遷移する.
- ・ **State 2 → State 3 (演奏開始)**: 関数 `process0Commands()` 内での楽譜監視において、前述の「無駄な待機時間の動的スキップ処理」が完了し、ドラム機自身の演奏開始相対タイミング (`elapsedPlayTick  $\geq$  各曲の開始 evTick`) に達した瞬間に State 3 へ遷移し、自律演奏が開始される.

ドラム以外の楽器 (子機) の状態遷移

子機における各ステート間の遷移タイミングは以下の通りである.

- ・ **任意状態 → State 4 / State 0**: ドラム機と同様に、指揮機からの'E' 通信受信によって State 4 (停止・リセット) へ、'S' 通信受信によって State 0 (開始・待機) へ遷移する. State 0 への遷移直後、子機はドラム機に対して参加予告 (J 通信) を即座に UDP 送信する.
- ・ **State 0 → State 1 (自機 BPM の安定)**: センサによる自身の BPM 計測が 3 回連続で安定した瞬間、State 1 へ遷移する. 遷移後は、ドラム機への BPM 提案 (M 通信) を継続しつつ、ドラム機からの確定 BPM (N 通信) を待受ける. また、定期的な Tick 同期要求 (T 通信) の送信と R 通信によるフィードバック補正が有効化される.
- ・ **State 1 → State 2 (輪唱合流・演奏開始)**: ドラム機から演奏開始指示 (O 通信) を受信し、パケットに封入された演奏開始絶対時間 (`startGlobalTick`) に対して自身の `globalTick` が到達、または通信遅延等の理由により既に超過 (オーバーラン) していると判定された瞬間に State 2 へ遷移し、Processing への発音パケット送信 (演奏) を開始する.

3.5 処理フローの設計

3.5.1 全体のフローチャート

今回製作するものの処理全体のフローチャートは以下の図 3.32 の通りである。指揮デバイスの方で初期の BPM の設定を行った後、指揮棒を振った場合中継機デバイスや楽器に演奏情報を送信し、ボタンが押された際に通信開始情報を送信し、観覧車を作動させる。

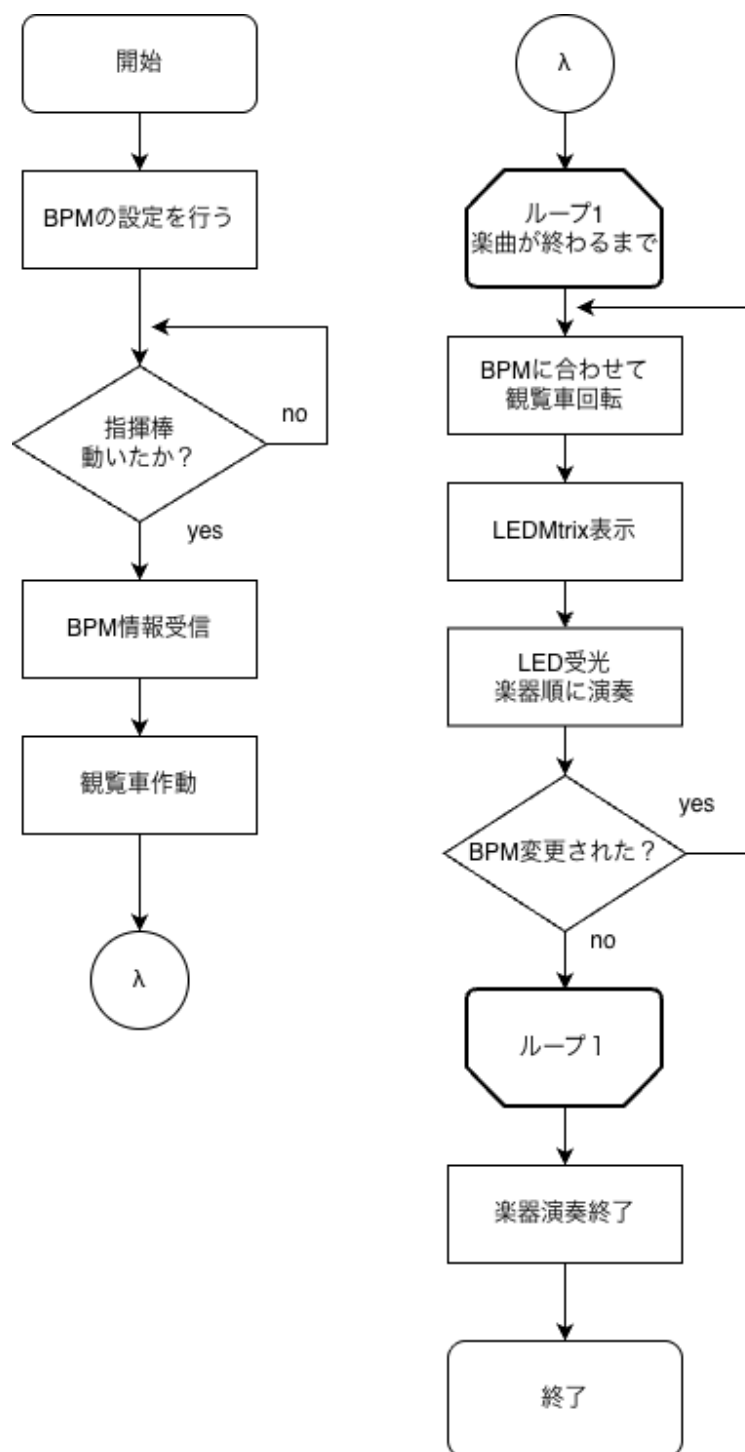


図 3.32 全体のフローチャート図

3.5.2 LEDMatrix の処理フローチャート

次に、LEDMatrix の処理フローについて、図 3.33 を用いて説明する。始めに初期化処理をしたあと、メインループに移行する。メインループは指揮棒から終了信号が来るまで行うループである。

メインループ内では、中継機から送信される UDP パケットを常時監視する。パケットを受信した場合、文字列としてデータを取得し、ヘッダ確認を行う。ここで、ヘッダが規定値と一致しない場合は不正データとして破棄し、受信待機状態へ復帰する。ヘッダが一致した場合は BPM 値を取得し、前回値との比較によって値更新の有無を確認する。値が更新されていない場合は描画処理を行わず、受信待機状態へ戻る。一方、値更新が確認された場合は `displayBPM()` 関数を呼び出し、描画処理を実行する。描画処理では、LED マトリクスの表示状態を初期化した後、BPM 値を各桁へ分割する。その後、各桁に対して独自フォント配列との照合および描画処理を繰り返し実行し、最終的に `renderBitmap()` 関数を用いて LED マトリクスへ反映する。その後、指揮棒が終了合図を送信し、それを受け取るとループから抜け出し、Matrix 表示を終えるという流れである。

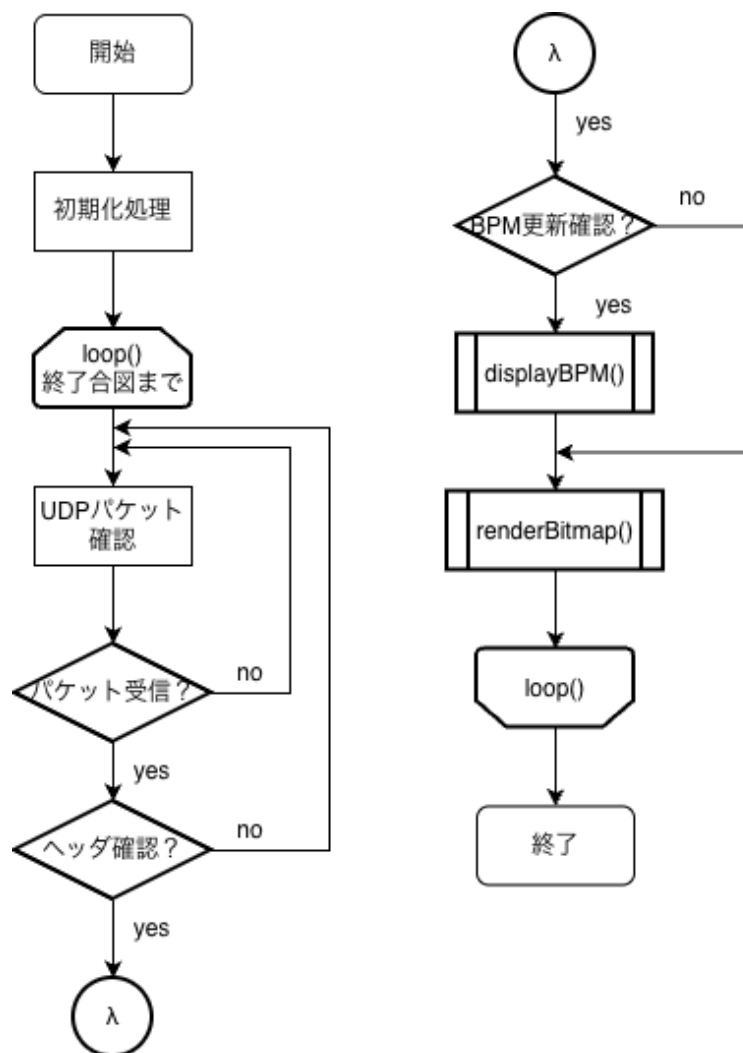


図 3.33 LEDMatrix 内の処理フローチャート図

3.6 テストの設計

3.6.1 音色の再現度テスト

Processing で作成した音色 (ピアノ, フルート, ストリングス, ドラム) の音と, 実際の楽器の録音音声を複数の被験者に聞いてもらい, 似ているかの 5 段階評価のアンケートに回答してもらう. このアンケートの結果の中央値と最頻値を確認する.

3.6.2 ボタン判定の制御

指揮デバイスに接続されたボタンの制御プログラムのテストとして、複数回ボタンを押下し、失敗する確率を測定する。また、連打や長押しをした場合、意図しない動作をしないかを確認する。成否の判定はシリアルモニタに結果を表示するプログラムを追加して行う。

3.6.3 加速度センサの制御

加速度センサの制御プログラムのテストとしては、まず適切な閾値 `accThreshold` が正しく設定されているか確認する。閾値が正しく設定されていないと、一度の振る動作で複数回検知されたり、振っても検知されないことが起きるためである。具体的なテスト方法としてシリアルモニタに振ったと検知したタイミングで表示させるプログラムを追加し、適切に検知されるか確認する。また、振ったと判断されるまでの遅延を計測し、体感として許容範囲であるかを評価する。

3.6.4 BPM・音量読み取り機能

BPM・音量読み取り機能のテストとして、まず移動平均の `sampleSize` が適切かどうか確認する。`sampleSize` が適切でないと、意図せず BPM・音量が変更されたり、移動平均算出にかかる時間が増加し、遅延が増加する可能性がある。テスト方法としては、シリアルモニタに表示して意図せず変更されず、かつ遅延がどのくらいあるかを確認する。

3.6.5 BPM・音量変更機能

BPM・音量変更機能としては、正確に変更された情報が送信され受信できるかを確認する。また、変更操作をしてから受信が完了するまでの時間を確認する。

3.6.6 観覧車の制御

観覧車の制御のテストとしては、設定された BPM に合わせて 4 小節に正確に一回転しているかを確認する。また、最も低い BPM60 の時モータが止まらずにスムーズな回転を維持できているかを確認する。さらに、BPM60 から 180 の急激な BPM 変化で、スムーズに回転速度を変更できるかを確認する。また、表示する LEDMatrix の内容が正しいことを確認する。

3.6.7 BPM 変更機能

BPM 変更機能のテストとしては、CdS セルがレーザー光の通過を正確に読み取れるかを確認する。一度の通過で 2 回以上検知されたり、最高速度の BPM180 でレーザー光を逃さずに検知できるかを確認する。また、観覧車が動かしている BPM と読み取った BPM が一致するかを確認する。

3.6.8 同期精度テスト

ここでは、物理的な動きと楽器の発音タイミングがどの程度一致しているかを検証することを目的とする。具体的には、観覧車のレーザー光が照度センサを通過する物理的なタイミングと、楽器側の発音タイミングが一致し、許容範囲内の遅延におさまっているかを確認する。

3.6.9 負荷テスト

ここでは、システムに高負荷をかけた際の挙動の安定性を検証する。まず、通信負荷をテストするため、BPM や音量の変更パケットを短時間で連続して送信する。次に物理負荷をテストするため、最大速度（180 BPM）で長時間連続運転を行う。これらの操作により、システムが停止したり制御不能に陥ったりしないことを確認する。

参考文献

- [1] 株式会社マーケットリサーチセンター, “音楽出版の日本市場（～2031 年）、市場規模（パフォーマンズ、同期、デジタル収益）・分析レポートを発表,” <https://www.atpress.ne.jp/news/4436457>, 2026/4/22 参照.
- [2] LP Information, “ライブコンサート市場占有率分析レポート,” <https://www.atpress.ne.jp/news/3304316>, 2026/4/22 参照.
- [3] 一般社団法人コンサートプロモーターズ協会, “ライブ市場調査 基礎推移表” <https://www.acpc.or.jp/marketing/transition/>, 2026/5/18 参照.
- [4] Natalia Cooper, Dimitrios Mileounis, Patrick Williams, and Frank P. Brooks, “The effects of substitute multisensory feedback on task performance and the sense of presence in a virtual reality environment,” PLOS ONE, vol.13, no.2, p.e0191846, 2018.
- [5] Martin, R., and Nielsen, N. Enacting Musical Aesthetics: The Embodied Experience of Live Music. Music & Science, 7, 2024
- [6] Yamaha Music Japan, “ENSPIRE PRO,” https://jp.yamaha.com/products/musical_instruments/pianos/disklavier/enspire_pro/features.html#product-tabs, 2026/5/19 参照.
- [7] 関根 伸也, “DMX512 信号システムなどの現状技術,” 電気設備学会誌, vol.41, no.72026, p.382-385, 2021
- [8] ReacTable Legacy, “Welcome|ReacTable Legacy,” <https://reactable.com/>, 2026/5/19 参照.
- [9] H. Haas.: Über den Einfluss Eines Einfachechos auf die Hörsamkeit von Sprache, Acustica, Vol.1, No.2, pp.49-58 (1951).
- [10] KOBESOFT: 稼働率, <https://kobesoft.co.jp/mikata/words/network-cloud/availability-rate/>, (2026/5/19 参照).
- [11] American Customer Satisfaction Index: U.S. Overall Customer Satisfaction, <https://theacsi.org/the-acsi-difference/us-overall-customer-satisfaction/> (参照 2026-05-17).
- [12] 秋月電子通商, “fu650ad5-c6 データシート,” <https://akizukidenshi.com/goodsaffix/fu650ad5-c6.pdf>, (参照 2026-06-30).

- [13] 有世犬, "ティック (tick) - DTM 用語", [https://dtm.conceptmol.com/ティック\(tick\)/](https://dtm.conceptmol.com/ティック(tick)/), (参照 2026-05-22).
- [14] Arduino, "Arduino UNO R4 WiFi Datasheet", <https://docs.arduino.cc/resources/datasheets/ABX00087-datasheet.pdf>, (参照 2026-04-28).
- [15] Control.com, "DC Motor Speed Control," Lessons In Electric Circuits, <https://control.com/textbook/variable-speed-motor-controls/dc-motor-speed-control/>, (参照 2026-05-04).
- [16] リオン株式会社, 「音響・振幅の FFT 解析」, https://svmeas.rion.co.jp/support/p38veq0000000cmg-att/FFT_07881.pdf, (参照 2026-05-27).